

计算理论作业

胡临天

2026 年 5 月 22 日

目录

Chapter 0	2
Chapter 1	6
Chapter 2	42
Chapter 3	69
Chapter 4	77
Chapter 5	87
Chapter 6	105
Chapter 7	118
Chapter 8	157
Chapter 9	168
Chapter 10	175

Chapter 0

0.4

0.4 考察下列集合的形式化描述，先理解它们包括什么样的元素，写一段简短文字描述每一个集合。

- a. $\{1, 3, 5, 7, \dots\}$ 。
- b. $\{\dots, -4, -2, 0, 2, 4, \dots\}$ 。
- c. $\{n \mid \text{对 } \mathcal{N} \text{ 中的某一个 } m, n=2m\}$ 。
- d. $\{n \mid \text{对 } \mathcal{N} \text{ 中的某一个 } m, n=2m, \text{ 并且对 } \mathcal{N} \text{ 中的某个 } k, n=3k\}$ 。
- e. $\{w \mid w \text{ 是 } 0、1 \text{ 字符串, 且 } w \text{ 等于 } w \text{ 的反转}\}$ 。
- f. $\{n \mid n \text{ 是一个整数, 且 } n=n+1\}$ 。

图 1

解答.

- (a) 这是所有正奇数组成的集合，即 $1, 3, 5, 7, \dots$ 。
- (b) 这是所有偶数组成的集合，包括负偶数、0 和正偶数。
- (c) 这是所有能写成 $2m$ 的自然数组成的集合，也就是所有偶自然数。
- (d) 这是所有同时是 2 的倍数和 3 的倍数的自然数组成的集合，也就是所有 6 的倍数。
- (e) 这是所有由 0,1 构成并且满足 $w = w^R$ 的字符串组成的集合，也就是所有二进制回文串。
- (f) 这是空集，因为不存在整数 n 满足 $n = n + 1$ 。

0.7

0.7 对下列每一小题，给出一个满足指定条件的关系。

- a. 自反的和对称的，但不是传递的。
- b. 自反的和传递的，但不是对称的。
- c. 对称的和传递的，但不是自反的。

图 2

案:

案: 可以把关系看成定义在某个顶点集上的一组有向边。这样一来，自反性对应于每个顶点都有一个自环；对称性对应于若存在边 $x \rightarrow y$ ，则也必须存在边 $y \rightarrow x$ ；传递性对应于只要存在路径 $x \rightarrow y \rightarrow z$ ，就必须再有一条边 $x \rightarrow z$ 。因此，从图的视角来看，这道题实际上是在构造满足若干边性质、但不满足另一些边性质的小型有向图。这个视角对于寻找反例和构造例子都很有帮助。

0.9

0.9 试写出下述图的形式化描述

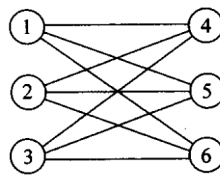


图 3

解答.

$(\{1, 2, 3, 4, 5, 6\}, \{(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)\})$

案：图的形式化描述的方法是 (V, E) ，有此约定能通过顶点和边的二元组来形式化描述一个图，同样通过这个约定能从形式化的描述中画出对应的图。

0.10

0.10 请找出下述证明 $2=1$ 中的错误。

提示：考虑到方程 $a=b$ 。两边同乘以 a ，得到 $a^2=ab$ 。两边同时减去 b^2 ，得到 $a^2-b^2=ab-b^2$ 。对每一边做因式分解，得 $(a+b)(a-b)=b(a-b)$ 。用 $(a-b)$ 除每一边，得 $a+b=b$ 。最后，令 a 和 b 等于 1，得证 $2=1$ 。

图 4

解答. 证明“ $2=1$ ”的错误所在：

原始论证复现

设 $a=b$ ，则：

1. $a=b$ (前提)
2. $a^2=ab$ (两边乘以 a)
3. $a^2-b^2=ab-b^2$ (两边减去 b^2)
4. $(a+b)(a-b)=b(a-b)$ (两边因式分解)
5. $a+b=b$ (两边除以 $(a-b)$) ← 错误在此
6. 令 $a=b=1$ ，得 $2=1$

错误出现在第 4 步到第 5 步的推导中。

由前提 $a=b$ ，可得：

$$a-b=0$$

第 5 步的操作是将等式两边同除以 $(a-b)$ ，即除以 0。

除以零是未定义的运算。在实数域（或任何域）的公理中，乘法逆元的存在性要求：

$$\forall x \in \mathbb{R}, x \neq 0 \implies \exists x^{-1} \text{ 使得 } x \cdot x^{-1} = 1$$

当 $x = 0$ 时，不存在这样的逆元。因此，“两边除以 $(a - b)$ ”这一步骤没有合法性，后续一切推导均无效。

案 该“证明”通过在 $a = b$ 的前提下对 $(a - b) = 0$ 做除法，违反了实数域中除法要求除数非零的基本规则，从而导出了矛盾。错误不在代数变换本身，而在于施加了一个不合法的运算。■

0.11

0.11 请找出下述证明中的错误，它证明所有的马颜色相同。

断言：在任意一个 h 匹马的集合中，所有的马颜色相同。

证明：对 h 作归纳证明。

1) 归纳基础：对于 $h=1$ ，在任何只有一匹马的集合中，显然所有的马颜色相同。

2) 归纳步骤：对于 $k \geq 1$ 。假设命题对于 $h=k$ 为真，要证命题对于 $h=k+1$ 也为真。

令 H 为任意一个有 $k+1$ 匹马的集合，现在要证明这个集合中的所有马颜色相同。

图 5

从这个集合中牵走一匹马，得到集合 H_1 ， H_1 中恰好有 k 匹马。根据归纳假设， H_1 中所有的马颜色相同。现在把牵走的马重新牵回来，再牵走另外一匹马得到集合 H_2 。根据同样的道理， H_2 中所有的马颜色也应相同。因此， H 中所有的马颜色一定相同。

图 6

解答：“所有马颜色相同”的错误证明——严格分析

原始论证复现

断言：在任意一个 h 匹马的集合中，所有的马颜色相同。

“证明”：对 h 作归纳。

1. **归纳基础：** $h = 1$ 时，只有一匹马，显然颜色相同。✓

2. **归纳步骤：**假设对 $h = k$ 命题为真，证明对 $h = k + 1$ 也为真。

令 $H = \{h_1, h_2, \dots, h_{k+1}\}$ 为任意 $k+1$ 匹马的集合。构造：

$$H_1 = \{h_1, h_2, \dots, h_k\}, \quad H_2 = \{h_2, h_3, \dots, h_{k+1}\}$$

由归纳假设， H_1 中所有马颜色相同， H_2 中所有马颜色相同。

而 $H_1 \cap H_2 = \{h_2, \dots, h_k\}$ 非空，故 H_1 和 H_2 的马颜色一致，从而 H 中所有马颜色相同。

错误所在

错误出现在归纳步骤从 $k = 1$ 到 $k = 2$ 的过渡中。

当 $k = 1$ 时，需要证明任意 $k + 1 = 2$ 匹马颜色相同。令 $H = \{h_1, h_2\}$ ，按上述方法构造：

$$H_1 = \{h_1\}, \quad H_2 = \{h_2\}$$

此时：

$$H_1 \cap H_2 = \emptyset$$

交集为**空集**。因此无法通过公共元素将 H_1 和 H_2 的颜色“桥接”起来。归纳步骤所依赖的关键推理

“ $H_1 \cap H_2$ 非空，故两个子集的马颜色一致”

——在 $k = 1$ 时**不成立**。

严格说明

归纳步骤的论证隐含地使用了如下事实：

$$|H_1 \cap H_2| = |H| - 2 = k + 1 - 2 = k - 1$$

这要求 $k - 1 \geq 1$ ，即 $k \geq 2$ ，交集才非空。

因此，归纳步骤仅在 $k \geq 2$ 时有效。而从基础 $h = 1$ 到 $h = 2$ 的这一步恰好是归纳链断裂的地方——整个归纳证明在最关键的第一步就失败了。

案： 该证明的错误是一个**归纳步骤的适用范围错误**：归纳步骤要求两个 k 元子集有非空交集（即 $k \geq 2$ ），但归纳基础仅建立了 $k = 1$ 的情形。从 $k = 1$ 到 $k = 2$ 的推导无法完成，归纳链在起点即断裂。■

0.13

A*0.13 Ramsey 定理。 设 G 是一个图， G 中的**团**为任意两个顶点都有边相连的子图。**反团**又叫做**独立集**，它为任意两个顶点都没有边相连的子图。试证明：所有 n 个顶点的图都包含一个顶点数不少于 $\frac{1}{2} \log_2 n$ 的团或反团。

图 7

解答：

0.13 将空间划分成两类顶点集， A 和 B 。然后扫描整个图，如果顶点 x 度数大于空间中剩余顶点数目的一半，则把 x 加到 A 中，否则将其加到 B 中。如果 x 加到 A 中，则不再考虑所有不与其相连的顶点；如果 x 加到 B 中，则不再考虑所有与其相连的顶点。重复这个步骤，直到处理完所有顶点。由于每一步最多丢弃一半的顶点，因此至少需要经过 $\log_2 n$ 步过程才会终止。由于每步只在 A 或 B 中添加一个顶点，所以在过程结束后 A 或 B 中至少含有 $\frac{1}{2} \log_2 n$ 个顶点。其中， A 中包含的顶点属于团， B 中包含的顶点属于反团。

图 8

案： 这题是通过构造性证明的方式来证明：所有 n 个顶点的图都包含一个顶点数不少于 $\frac{1}{2} \log_2 n$ 的**团或反团**。通过观察构造出来的算法是如何处理 n （节点的个数）证明的这个定理。算法设计用于解决问题，从图灵机的视角来看就是在识别“语言”，反过来分析算法的步骤能得出更多的这个“语言”的性质。如果进一步利用这个性质，或能用于优化这个算法，或能用于优化其他算法。分析做是从另一个视角看一个问题（语言），优化做的是对算法的重新理解。

Chapter 1

1.4

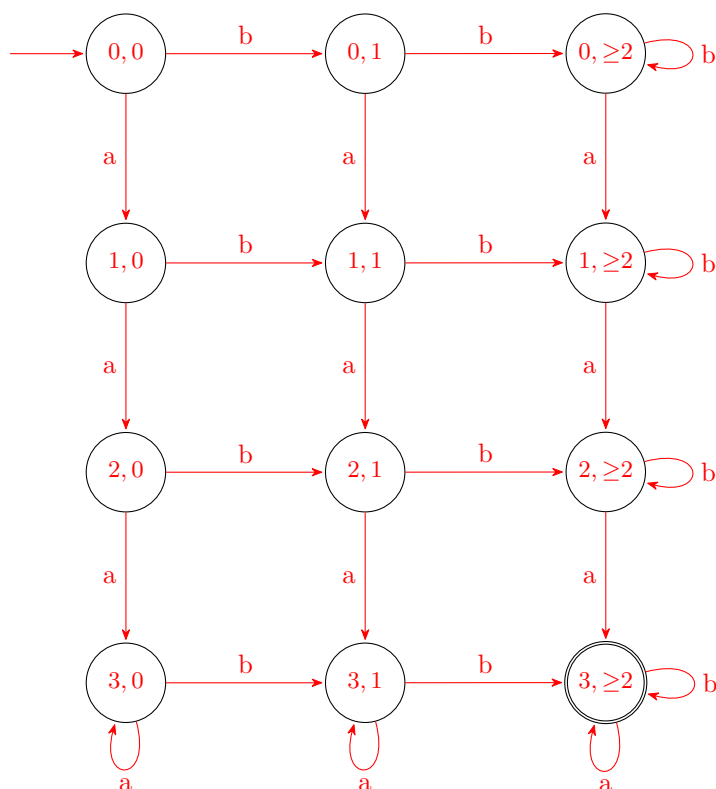
1.4 下面每个语言都是两个简单语言的交。对每一小题先构造简单语言的 DFA，然后按照 1.1.5 节的脚注所讨论的构造方法结合它们以画出给定语言的 DFA 状态图。在所有问题中 $\Sigma = \{a, b\}$ 。

- a. $\{w \mid w \text{ 含有至少 3 个 } a \text{ 和至少 2 个 } b\}$
- ^A b. $\{w \mid w \text{ 含有正好 2 个 } a \text{ 和至少 2 个 } b\}$
- c. $\{w \mid w \text{ 含有偶数个 } a \text{ 和 1 个或 2 个 } b\}$
- ^A d. $\{w \mid w \text{ 含有偶数个 } a \text{ 并且每个 } a \text{ 后都跟有至少一个 } b\}$
- e. $\{w \mid w \text{ 从 } a \text{ 开始并且最多有 1 个 } b\}$
- f. $\{w \mid w \text{ 含有奇数个 } a \text{ 并且以 } b \text{ 结束}\}$
- g. $\{w \mid w \text{ 的长度为偶数并且有奇数个 } a\}$

图 9

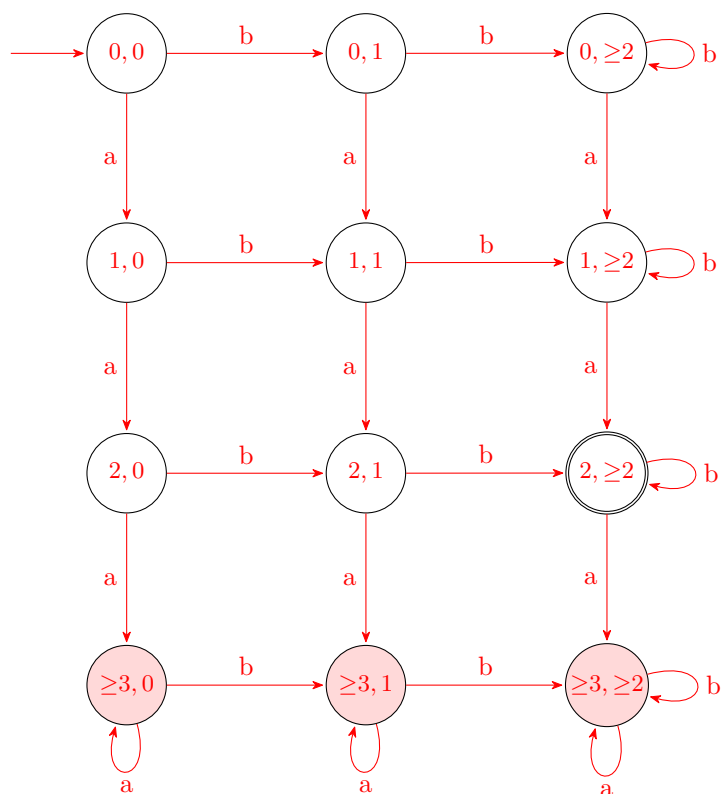
(a) $\{w \mid w \text{ 含至少 3 个 } a \text{ 和至少 2 个 } b\}$

状态 $(i, j): i = a \text{ 的计数 (3 饱和), } j = b \text{ 的计数 (2 饱和)}$ 。接受态 $(3, \geq 2)$ 。



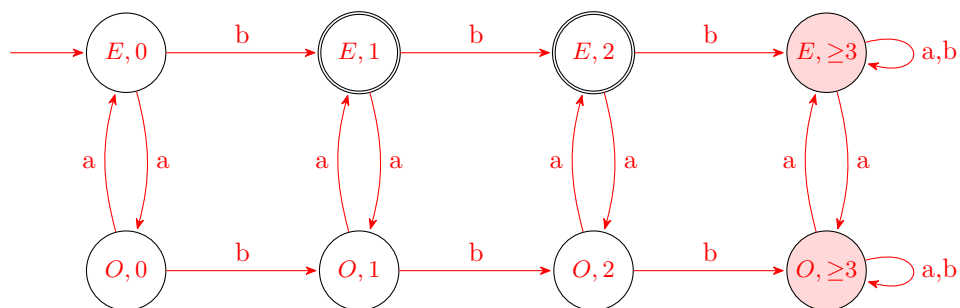
(b) $\{w \mid w \text{ 含正好 2 个 } a \text{ 和至少 2 个 } b\}$

第 4 行 $(\geq 3, \cdot)$ 为死状态 (红色), 接受态仅 $(2, \geq 2)$ 。



(c) $\{w \mid w \text{ 含偶数个 } a \text{ 和 } 1 \text{ 个或 } 2 \text{ 个 } b\}$

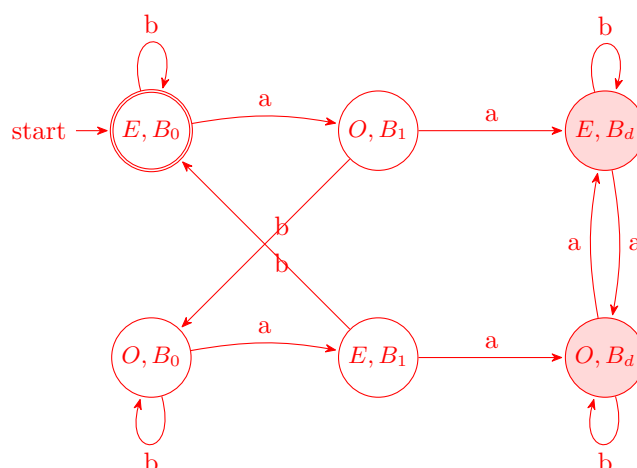
行: a 奇偶 (E/O); 列: b 计数 0, 1, 2, ≥ 3 (死)。接受态 (E, 1) 和 (E, 2)。



注: 死列箭头 a 实际指向另一行的死态, 本图已用两条双向箭头覆盖。

(d) $\{w \mid w \text{ 含偶数个 } a \text{ 且每个 } a \text{ 后跟至少一个 } b\}$

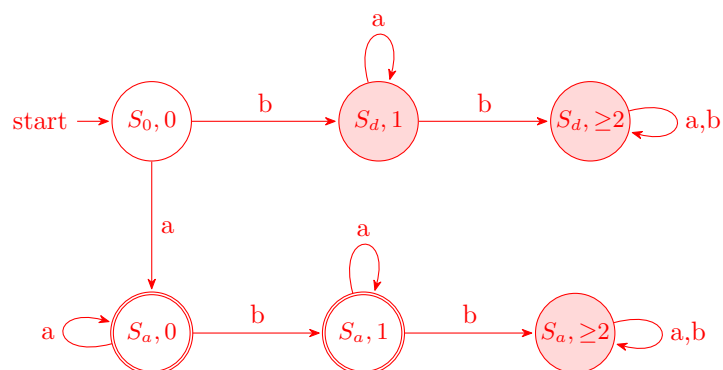
行: a 奇偶 E/O; 列: B_0 (可接受)、 B_1 (刚读 a 等 b)、 B_d 死。唯一接受 (E, B_0)。



案:

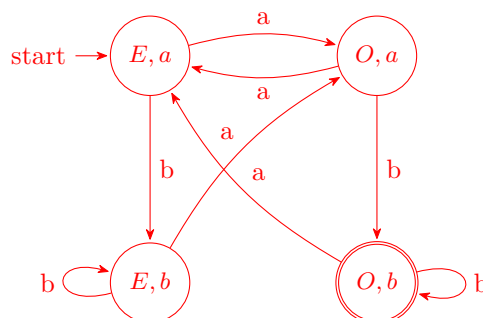
(e) $\{w \mid w \text{ 从 } a \text{ 开始且最多 } 1 \text{ 个 } b\}$

只绘从起始态可达的 6 个状态。接受态 $(S_a, 0)$ 和 $(S_a, 1)$ 。



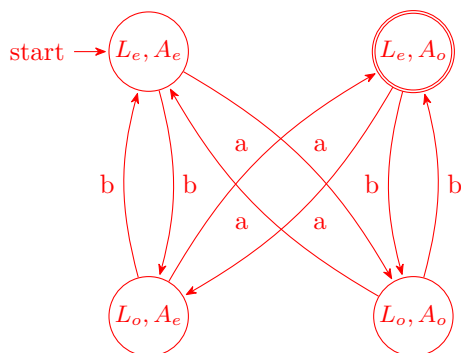
(f) $\{w \mid w \text{ 含奇数个 } a \text{ 且以 } b \text{ 结束}\}$

状态:(a 奇偶, 末字符)。起始态 (E, a) (空串归此类)。接受态 (O, b) 。



(g) $\{w \mid |w| \text{ 为偶数且 } w \text{ 有奇数个 } a\}$

状态:(长度奇偶, a 奇偶)。任何字符都翻转长度;a 额外翻转 a 计数。接受态 (L_e, A_o) 。



观察: 由 $|w| = \#_a + \#_b, |w|$ 偶且 $\#_a$ 奇 $\iff \#_b$ 亦为奇数, 所以此语言等价于 $\{w \mid \#_a, \#_b \text{ 均为奇数}\}$ 。

案: “和”、“并且”等表示的是

1.5

1.5 下述每个语言都是一个简单语言的补。对每一小题先构造简单语言的 DFA, 然后用其画出给定语言的 DFA 状态图。在所有问题中 $\Sigma = \{a, b\}$ 。

^A a. $\{w \mid w \text{ 中不含子串 } ab\}$

图 10

^A b. $\{w \mid w \text{ 中不含子串 } baba\}$

c. $\{w \mid w \text{ 中既不含子串 } ab \text{ 也不包含子串 } ba\}$

d. $\{w \mid w \text{ 是不在 } a^*b^* \text{ 中的任意串}\}$

e. $\{w \mid w \text{ 是不在 } (ab^+)^* \text{ 中的任意串}\}$

f. $\{w \mid w \text{ 是不在 } a^* \cup b^* \text{ 中的任意串}\}$

g. $\{w \mid w \text{ 是恰好不含 2 个 } a \text{ 的任意串}\}$

h. $\{w \mid w \text{ 是除 } a \text{ 和 } b \text{ 外的任意串}\}$

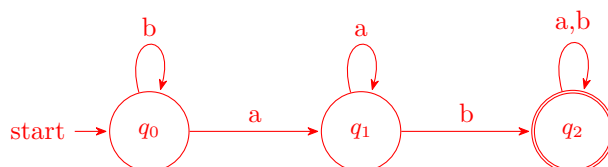
图 11

解答: 方法总结: 对每道题先给简单语言 L_1 构造完备 DFA M (所有状态对所有字符都必须有转移, 必要时添加死状态), 然后将接受态集 F 换成 $Q \setminus F$, 得到识别 $\overline{L_1}$ 的 DFA。转移保持不变。

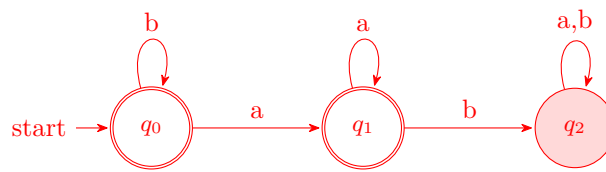
(a) $\{w \mid w \text{ 中不含子串 } ab\}$

简单语言 $L_1 = \{w \mid w \text{ 含子串 } ab\}$ 。

L_1 的 DFA:



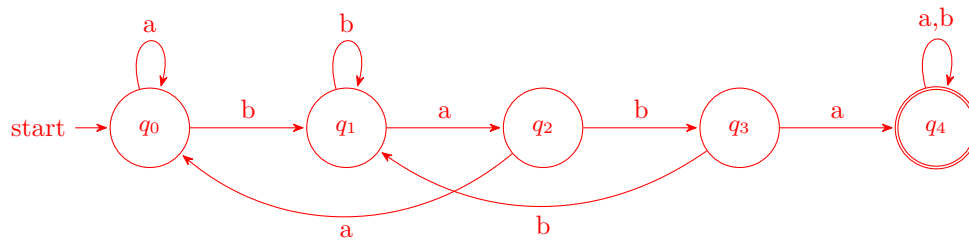
$\overline{L_1}$ 的 DFA(接受态互换):



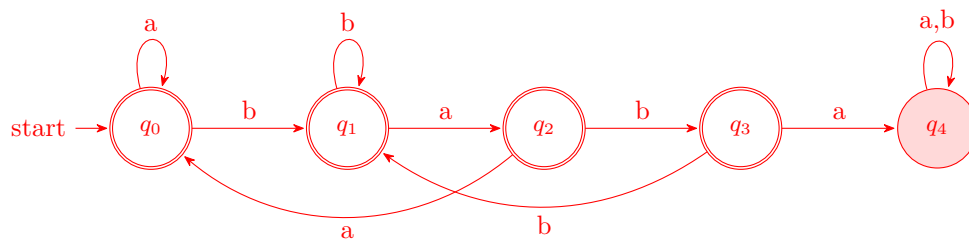
(b) $\{w \mid w \text{ 中不含子串 } baba\}$

简单语言 $L_1 = \{w \mid w \text{ 含子串 } baba\}$ 。失配回退是关键: q_3 读 b 退回 q_1 (新 b 可为新匹配开头); q_2 读 a 退回 q_0 。

L_1 的 DFA:



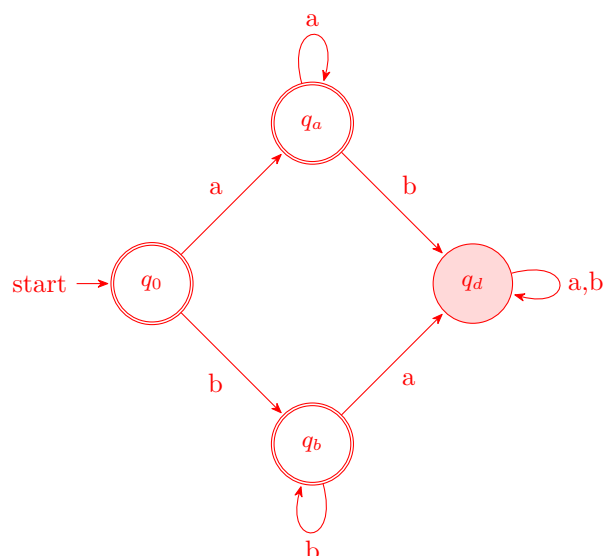
$\overline{L_1}$ 的 DFA:



(c) $\{w \mid w \text{ 中既不含 } ab \text{ 也不含 } ba\}$

等价于 $a^* \cup b^*$ 。直接构造: 4 个状态, q_d 是混合后的死状态。

直接 DFA(此题直接给出比用补集更清晰):

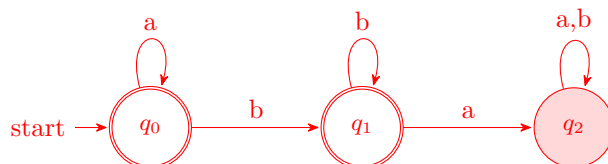


若严格按补集方法: 设 $L_1 = \{w \mid w \text{ 含 } ab \text{ 或含 } ba\}$, 则只需把上图的接受态翻转: q_0, q_a, q_b 变非接受, q_d 变接受即得 L_1 的 DFA。

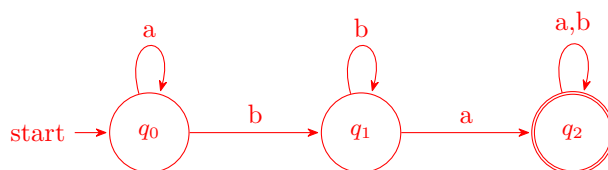
(d) $\{w \mid w \notin a^*b^*\}$

简单语言 $L_1 = a^*b^*$ 。三个状态: q_0 读 a 阶段, q_1 读 b 阶段, q_2 b 后又读 a(死)。

L_1 的 DFA:



$\overline{L_1}$ 的 DFA:

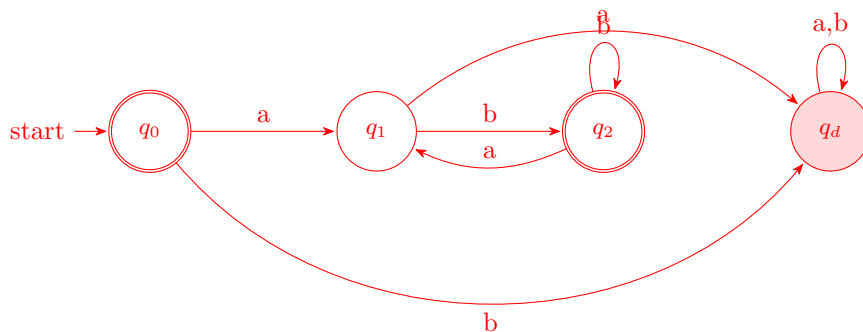


(e) $\{w \mid w \notin (ab^+)^*\}$

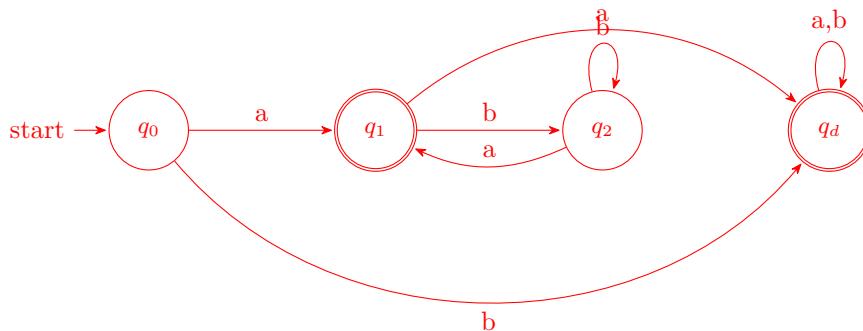
简单语言 $L_1 = (ab^+)^*$ = 零或多个"a 后跟至少一个 b" 块的拼接。

状态: q_0 起始/块完成; q_1 读 a 等 b; q_2 已读 ab(可接受, 可继续读 b 或开新块); q_d 死。

L_1 的 DFA:



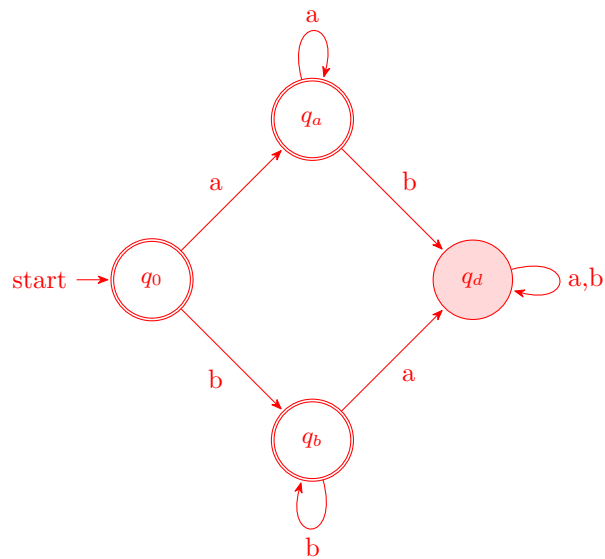
$\overline{L_1}$ 的 DFA:



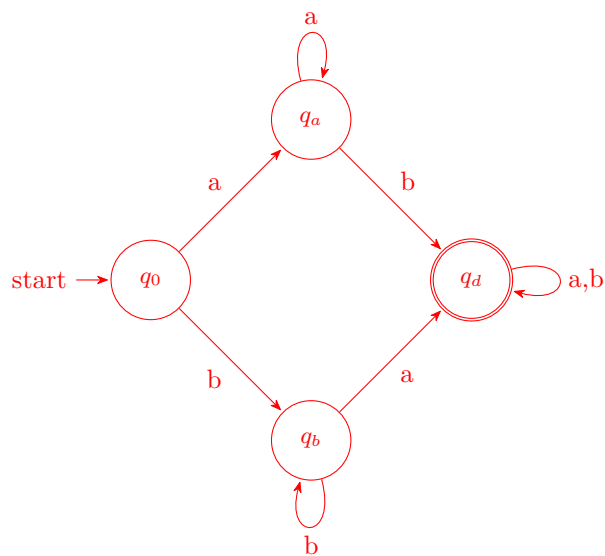
(f) $\{w \mid w \notin a^* \cup b^*\}$

简单语言 $L_1 = a^* \cup b^*$ (全 a 串或全 b 串, 含空串)。结构与 (c) 题相同。

L_1 的 DFA:



$\overline{L_1}$ 的 DFA:

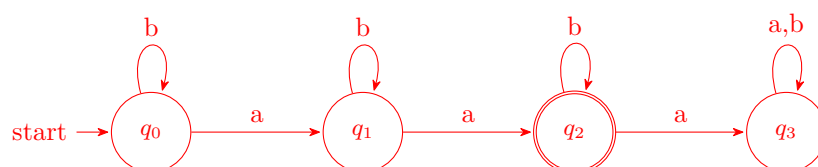


目标语言: a 和 b 都至少出现一次 (顺序不限) 的串。

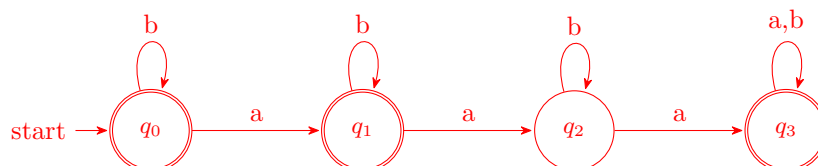
(g) $\{w \mid w \text{ 恰好不含 2 个 a}\}$

即 a 的个数 $\neq 2$ 。简单语言 $L_1 = \{w \mid w \text{ 恰好含 2 个 a}\}$ 。

L_1 的 DFA:



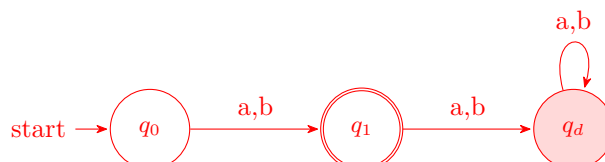
$\overline{L_1}$ 的 DFA:



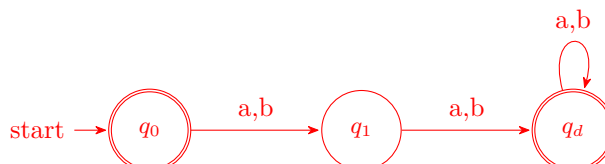
(h) $\{w \mid w \text{ 是除 } a \text{ 和 } b \text{ 外的任意串}\}$

即 $\Sigma^* \setminus \{a, b\}$ ——除了长度为 1 的两个串“a”和“b”以外的所有串 (包括空串!)。简单语言 $L_1 = \{a, b\}$ 。

L_1 的 DFA:



$\overline{L_1}$ 的 DFA:



注: q_0 接受空串 ϵ ; q_d 接受所有长度 ≥ 2 的串。

1.6

1.6 画出识别下述语言的 DFA 状态图。在所有问题中字母表均为 $\{0, 1\}$

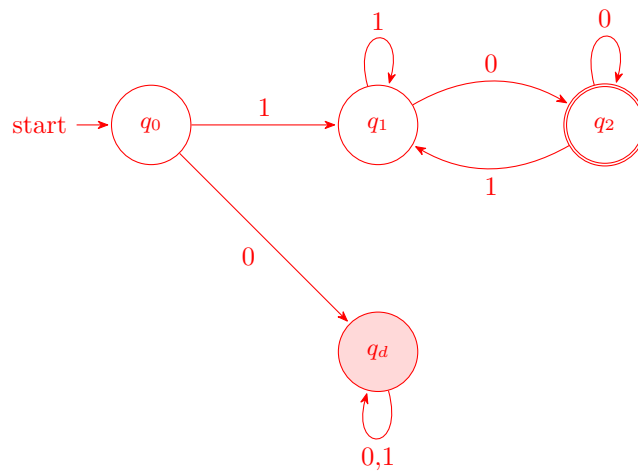
- $\{w \mid w \text{ 从 } 1 \text{ 开始且以 } 0 \text{ 结束}\}$
- $\{w \mid w \text{ 含有至少 } 3 \text{ 个 } 1\}$
- $\{w \mid w \text{ 含有子串 } 0101, \text{ 即对某个 } x \text{ 和 } y, w = x0101y\}$
- $\{w \mid w \text{ 的长度不小于 } 3, \text{ 并且第 } 3 \text{ 个符号为 } 0\}$
- $\{w \mid w \text{ 从 } 0 \text{ 开始且长度为奇数, 或者从 } 1 \text{ 开始且长度为偶数}\}$
- $\{w \mid w \text{ 不含子串 } 110\}$
- $\{w \mid w \text{ 的长度不超过 } 5\}$
- $\{w \mid w \text{ 是除 } 11 \text{ 和 } 111 \text{ 外的任意串}\}$
- $\{w \mid w \text{ 的奇数位置均为 } 1\}$
- $\{w \mid w \text{ 含有至少 } 2 \text{ 个 } 0, \text{ 并且至多含 } 1 \text{ 个 } 1\}$
- $\{\epsilon, 0\}$
- $\{w \mid w \text{ 含有偶数个 } 0 \text{ 或恰好 } 2 \text{ 个 } 1\}$
- 空集
- 除空串外的所有字符串

图 12

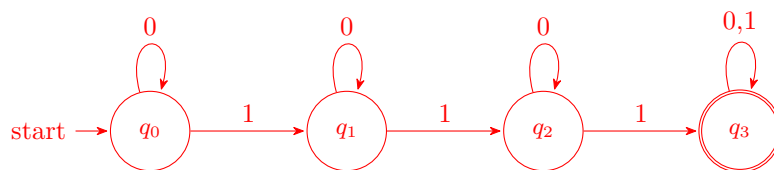
约定: 字母表 $\Sigma = \{0, 1\}$ 。所有 DFA 都是完备的, 死状态用红色底色标出, 接受态用双圆。

(a) $\{w \mid w \text{ 从 } 1 \text{ 开始且以 } 0 \text{ 结束}\}$

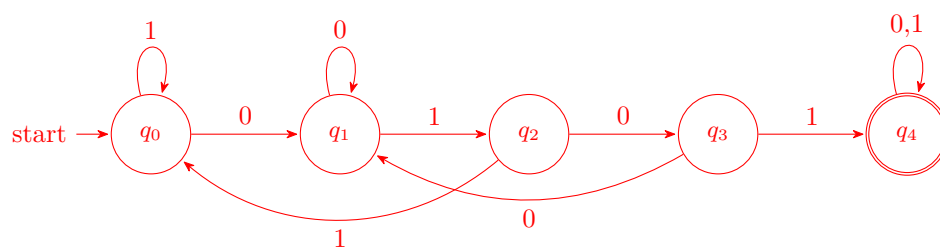
状态: q_0 起始 (未读字符); q_d 首字符为 0 的死状态; q_1 首字符 1, 当前末字符 1 (非接受); q_2 首字符 1, 当前末字符 0 (接受)。

(b) $\{w \mid w \text{ 含有至少 } 3 \text{ 个 } 1\}$

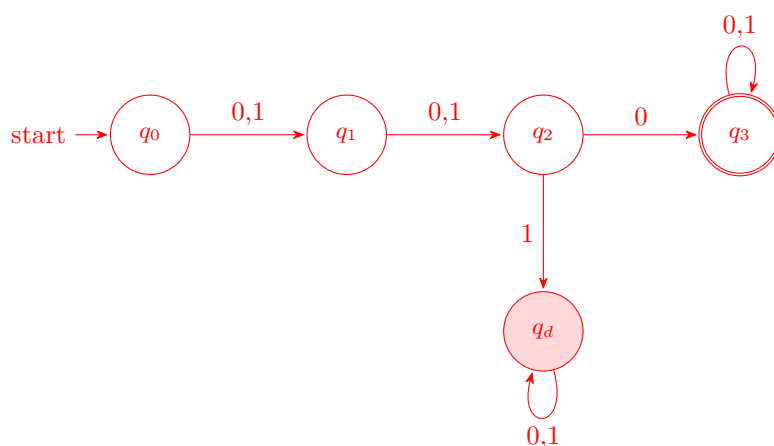
按 1 的计数划分, 3 饱和。

(c) $\{w \mid w \text{ 含子串 } 0101\}$

跟踪匹配进度, 失配时回到最长的已匹配前缀。关键回退: q_1 (已匹配 0) 读 0 保持 q_1 ; q_3 (已匹配 010) 读 0 退回 q_1 (新 0 可能是新匹配开头)。

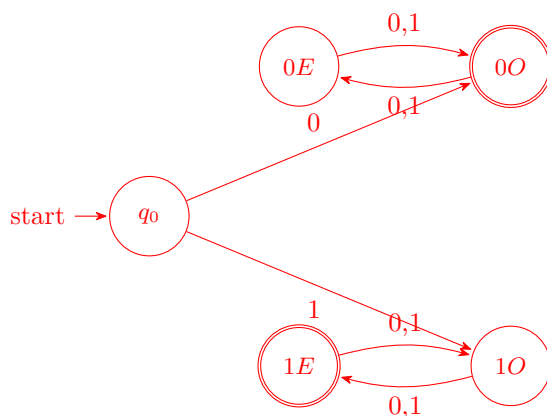
(d) $\{w \mid |w| \geq 3 \text{ 且第 } 3 \text{ 个符号为 } 0\}$

前三步按位置推进, 第 3 个字符决定去向。



(e) $\{w \mid w \text{ 从 } 0 \text{ 开始且长度奇数, 或从 } 1 \text{ 开始且长度偶数} \}$

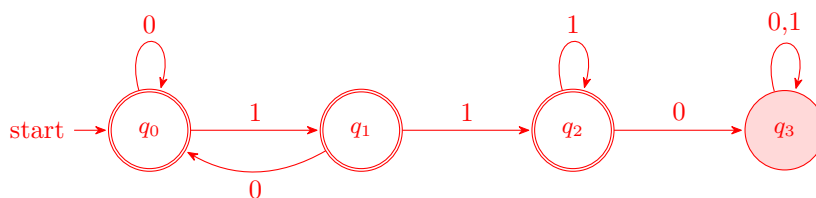
空串不满足任一条件 (空串无首字符)。5 个状态: q_0 起始; $0E, 0O$ (首 0, 当前长度偶/奇); $1E, 1O$ (首 1, 当前长度偶/奇)。接受: $0O$ (首 0 + 奇长)、 $1E$ (首 1 + 偶长, 但长度至少为 2)。



注: 从 q_0 读 0 后长度已为 1 (奇), 直接进入接受态 $0O$; 读 1 后长度为 1 (奇, 不接受), 进入 $1O$, 再读一字符才到 $1E$ 接受。

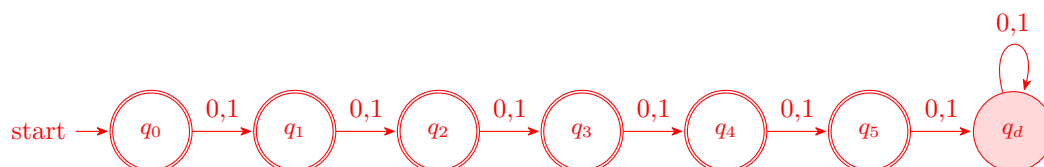
(f) $\{w \mid w \text{ 不含子串 } 110\}$

先画含 110 的 DFA, 再翻转接受态。 q_0 (未匹配)、 q_1 (匹配 1)、 q_2 (匹配 11)、 q_3 (已含 110)。



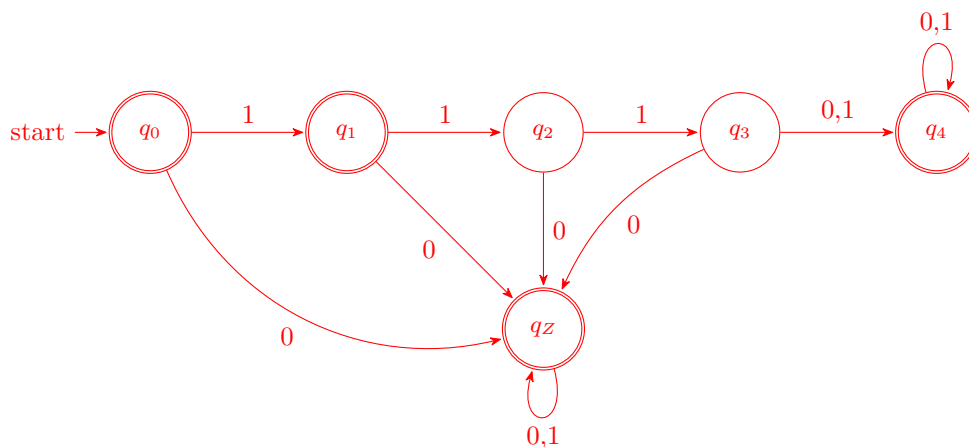
(g) $\{w \mid |w| \leq 5\}$

6 个接受态对应长度 $0, 1, \dots, 5$, 第 7 个状态为死态 (长度 ≥ 6)。



(h) $\{w \mid w \neq 11 \text{ 且 } w \neq 111\}$

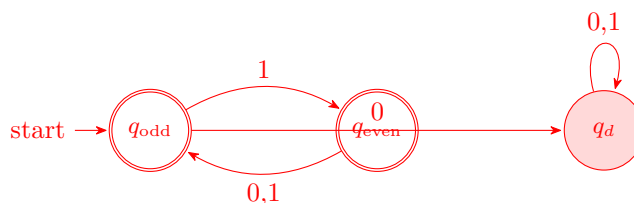
即 $\Sigma^* \setminus \{11, 111\}$ 。先识别 $\{11, 111\}$ 再取补。只需追踪“连续 1 的前缀长度 0/1/2/3”以及“是否已经出现过 0”。



状态含义: $q_0 = \varepsilon$ (接受); $q_1 = 1$ (接受, 不等于 11 或 111); $q_2 = 11$ (不接受); $q_3 = 111$ (不接受); $q_4 =$ 长度 ≥ 4 的纯 1 串或 111 之后还有字符 (接受); $q_z =$ 任何阶段出现过 0, 说明不是纯 1 串, 肯定不等于 11 或 111 (接受)。

(i) $\{w \mid w \text{ 的奇数位置均为 } 1\}$

位置从 1 开始数。奇数位必须是 1, 偶数位随意。空串空位置集满足, 接受。

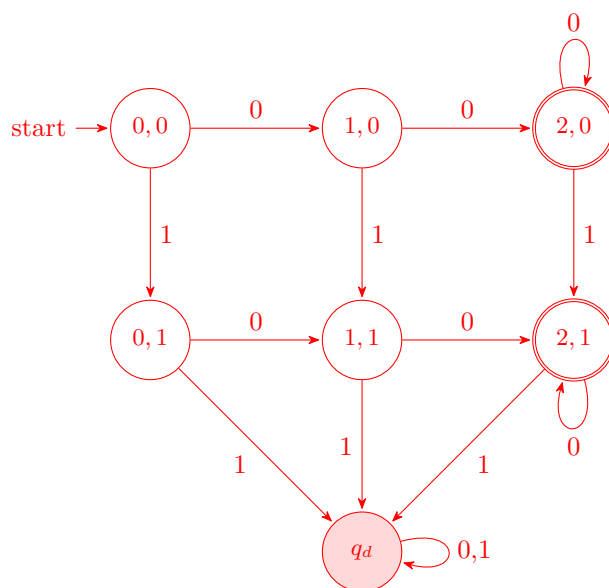


q_{odd} : 下一读入位置是奇数, 必须为 1; q_{even} : 下一位置是偶数, 0/1 都可; 一旦奇数位读到 0 就进入死态。

(j) $\{w \mid w \text{ 含至少 } 2 \text{ 个 } 0 \text{ 且至多 } 1 \text{ 个 } 1\}$

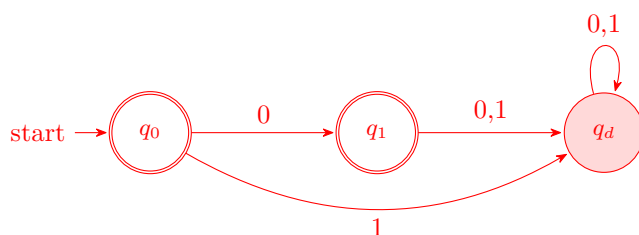
笛卡尔积: 0 计数 $\{0, 1, \geq 2\} \times 1$ 计数 $\{0, 1, \geq 2(\text{死})\}$ 。去掉 1 计数 ≥ 2 那列后只剩 2 列, 加一个共用死态共 $3 \times 2 + 1 = 7$ 个状态 (下面把同为死态的合并)。

状态记作 (i, j) , $i = 0$ 的计数 (0/1/2+), $j = 1$ 的计数 (0/1)。接受 $(2, 0)$ 和 $(2, 1)$ 。



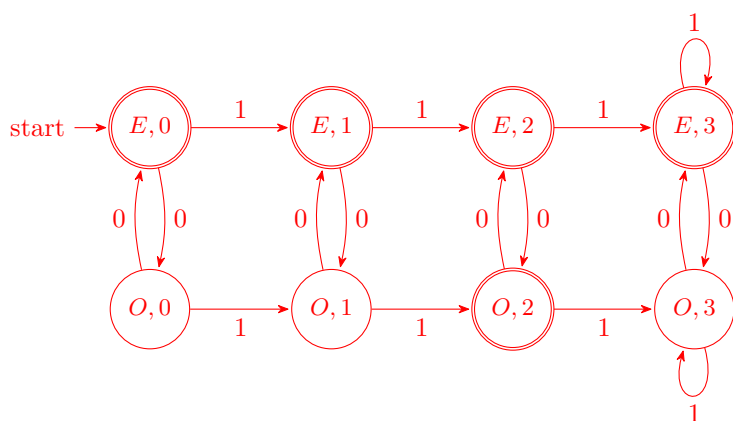
(k) $\{\varepsilon, 0\}$

只接受空串和单字符 0。



(l) $\{w \mid w \text{ 含偶数个 } 0 \text{ 或恰好 } 2 \text{ 个 } 1\}$

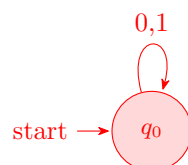
并运算用笛卡尔积: $\{w \mid \#_0 \text{ 偶}\}$ 的 2 状态 DFA $\times \{w \mid \#_1 = 2\}$ 的 4 状态 DFA = 8 状态。接受态: ** 至少一个分量是接受态 ** (并运算)。状态记为 (p, k) , $p \in \{E, O\}$ (0 的奇偶), $k \in \{0, 1, 2, 3\}$ (1 的计数, 3 代表 ≥ 3)。



接受规则 (并): E 行全部接受 (因为 0 偶); O 行中只有 $(O, 2)$ 接受 (1 恰好 2 个)。

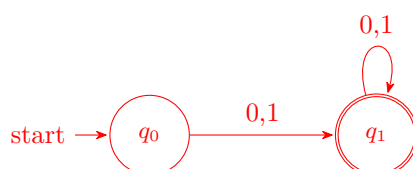
(m) 空集 $\emptyset$

不接受任何串。只需一个非接受态带所有字符自环。



(n) 除空串外的所有串 Σ^+

起始态不接受 (用于排除空串), 读任一字符就进入永久接受态。



1.7

1.7 给出识别下述语言的 NFA, 并且符合规定的状态数。在所有问题中字母表均为 $\{0, 1\}$ 。

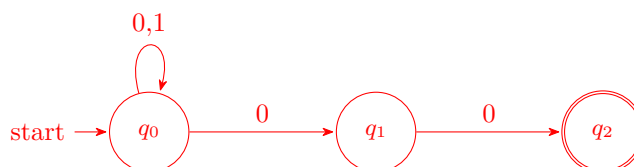
- ^A a. 语言 $\{w \mid w \text{ 以 } 00 \text{ 结束}\}$, 3 个状态。
- b. 练习 1.6c 中的语言, 5 个状态。
- c. 练习 1.61 中的语言, 6 个状态。
- d. 语言 $\{0\}$, 2 个状态。
- e. 语言 $0^* 1^* 0^+$, 3 个状态。
- ^A f. 语言 $1^* (001^+)^*$, 3 个状态。
- g. 语言 $\{\epsilon\}$, 1 个状态。
- h. 语言 0^* , 1 个状态。

图 13

方法总结: NFA 允许不完备的转移函数、对同一符号有多个去向、以及 ϵ 转移。构造时常用技巧: (1) 用起始态自环“等待”匹配时机, 由非确定性“猜”何时进入识别阶段; (2) 用 ϵ 转移实现并运算和 Kleene 星号; (3) 缺失的转移即“拒绝当前分支”。

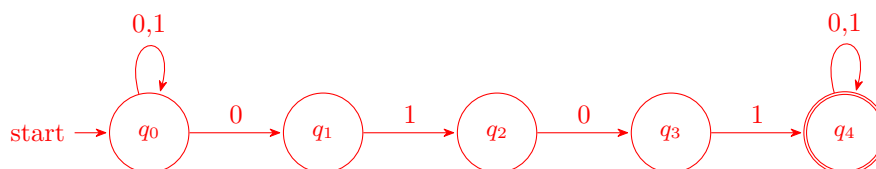
(a) $\{w \mid w \text{ 以 } 00 \text{ 结束}\}$, 3 个状态

q_0 靠自环消耗任意前缀, 非确定性地“猜”最后两个字符是 00 并分支到 q_1, q_2 。



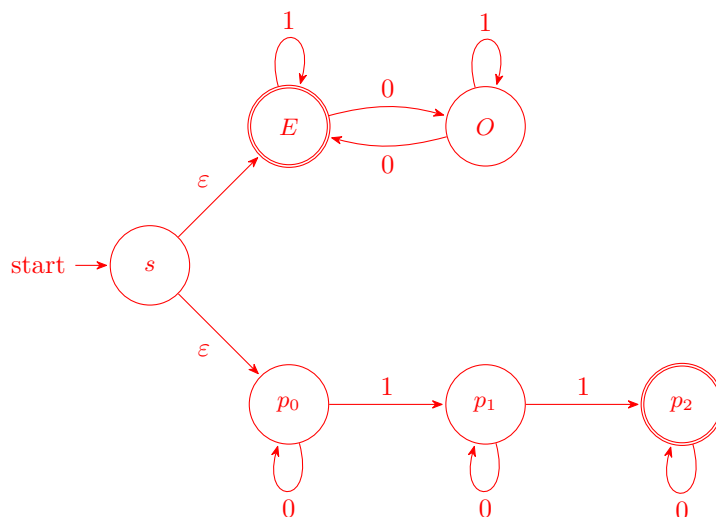
(b) $\{w \mid w \text{ 含子串 } 0101\}$, 5 个状态

与 1.6c 的 DFA 相比, NFA 无需设计失配回退—— q_0 自环“等”匹配时机。



(c) 练习 1.61 中的语言: 偶数个 0 或恰好 2 个 1, 6 个状态

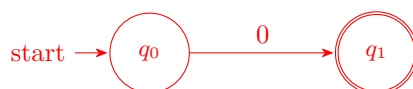
并运算用两条 ε 转移从起始态分叉到两个独立小 NFA。



上分支: E = 已读偶数个 0 (接受), O = 奇数个 0。下分支: p_k = 已读 k 个 1, p_2 接受; 一旦再读 1 就无路可走 (被拒绝)。

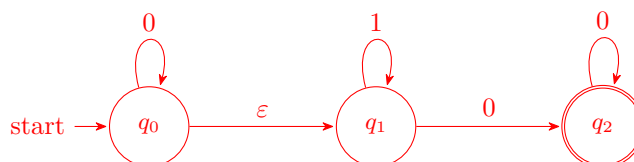
(d) 语言 $\{0\}$, 2 个状态

只接受单字符 0。读 1 或超出 1 字符都无路可走。



(e) 语言 $0^*1^*0^+$, 3 个状态

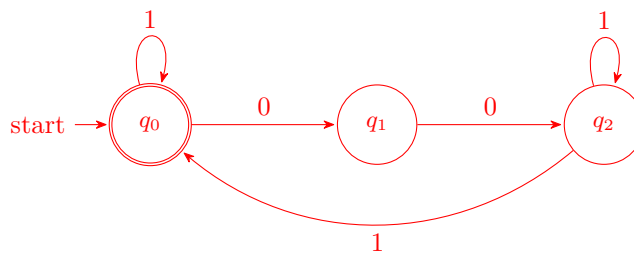
三个阶段对应三个状态。注意 0^+ 要求至少一个 0, 所以 q_2 必须 ** 读到 0 才能进入 ** (不能用 ε 跳进)。



q_0 消耗 0^* 前缀; ε 进入 q_1 消耗 1^* 中间; 读一个 0 进入 q_2 接受, q_2 再自环消耗剩余 0。

(f) 语言 $1^*(001^+)^*$, 3 个状态

q_0 即接受态 (对应已完成一个合法串, 包括空串)。 q_0 自环吃掉任意多个 1; 读 '00' 经由 q_1 到 q_2 ; q_2 必须读至少一个 1 才能返回 q_0 , 从而保证每个 001 块末尾的 1^+ 至少有一个 1。



关键: q_2 只能通过“读 1”离开(到 q_0 或自身), 这保证了每个 ‘00’ 之后至少跟一个 1。NFA 的非确定性体现在: q_2 读到一个 1 时, 既可选择自环留在 q_2 (继续延长当前块的 1^+), 也可选择转移回 q_0 (结束当前块或整个串)。

(g) 语言 $\{\epsilon\}$, 1 个状态

只接受空串: 起始态即接受态, 无任何转移。



(h) 语言 0^* , 1 个状态

起始态即接受态, 加一个 0 自环。读到 1 无路可走。



1.11

^A 1.11 证明每一台 NFA 都能够被转换成只有一个接受状态的等价 NFA。

图 14

1.11 Let $N = (Q, \Sigma, \delta, q_0, F)$ be any NFA. Construct an NFA N' with a single accept state that recognizes the same language as N . Informally, N' is exactly like N except it has ϵ -transitions from the states corresponding to the accept states of N , to a new accept state, q_{accept} . State q_{accept} has no emerging transitions. More formally, $N' = (Q \cup \{q_{\text{accept}}\}, \Sigma, \delta', q_0, \{q_{\text{accept}}\})$, where for each $q \in Q$ and $a \in \Sigma_\epsilon$

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{if } a \neq \epsilon \text{ or } q \notin F \\ \delta(q, a) \cup \{q_{\text{accept}}\} & \text{if } a = \epsilon \text{ and } q \in F \end{cases}$$

and $\delta'(q_{\text{accept}}, a) = \emptyset$ for each $a \in \Sigma_\epsilon$.

图 15

案: 中文版此处翻译错误, δ' 的第二行中的两个条件应该为“且”

设 $N=(Q,\Sigma,\delta,q_0,F)$ 为任一 NFA。构造一个只有一个接受状态的 NFA N' ，它与 N 接受同样的语言。非形式地讲， N' 与 N 很相近，但是 N' 从相对于 N 是接受状态的状态到新的接受状态 q_{accept} 有 ϵ 转移。状态 q_{accept} 上没有新的转移。形式地讲， $N'=(Q\cup\{q_{\text{accept}}\},\Sigma,\delta',q_0,\{q_{\text{accept}}\})$ ，其中对每一个 $q\in Q$ 和 $a\in\Sigma_\epsilon$ ，

$$\delta'(q,a)=\begin{cases} \delta(q,a) & \text{如果 } a\neq\epsilon \text{ 或 } q\notin F \\ \delta(q,a)\cup\{q_{\text{accept}}\} & \text{如果 } a=\epsilon \text{ 或 } q\in F \end{cases}$$

并且对每一个 $a\in\Sigma_\epsilon$ ， $\delta'(q_{\text{accept}},a)=\emptyset$ 。

图 16

1.13

1.13 令 F 是 $\{0,1\}$ 上所有串构成的语言，并且 F 中任意两个 1 之间间隔的符号数都不是奇数。画出识别 F 的 5 状态的 DFA 图。（先找一个 4 状态的 NFA 作 F 的补集可能有助于求解问题。）

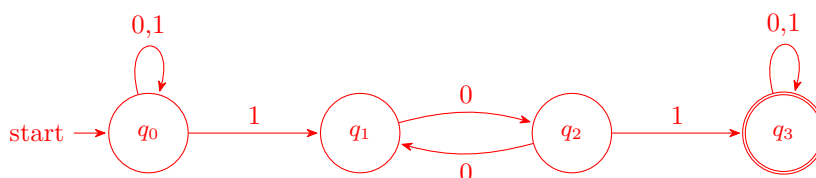
图 17

1.13 $F = \{w \mid w \text{ 中任意两个 1 之间间隔的符号数都不是奇数} \}$

题意分析：要求 F 中的串，其中任意两个 1 之间的 0 的个数都是偶数（包括 0 个）。没有 1 或只有一个 1 的串自动满足（空真）。

策略：按提示先构造识别 $\bar{F}$ 的 4 状态 NFA，再用子集构造转为 DFA，最后取补。

第一步：识别 $\bar{F}$ 的 4 状态 NFA。 $\bar{F}$ = 存在某对 1，它们之间的 0 的个数是奇数。NFA 利用非确定性猜违反对的第一个 1，然后验证其后读了奇数个 0 再读到 1。



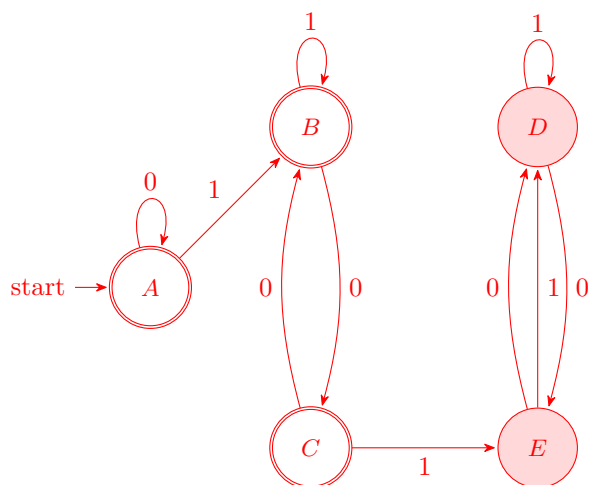
状态含义： q_0 = 读任意前缀； q_1 = 从猜到的第一个 1 起，已读偶数个 0； q_2 = 已读奇数个 0； q_3 = 奇数个 0 后读到第二个 1，违反对确认，接受。

第二步：子集构造转 DFA。

DFA 状态	读 0	读 1
$A = \{q_0\}$	$\{q_0\} = A$	$\{q_0, q_1\} = B$
$B = \{q_0, q_1\}$	$\{q_0, q_2\} = C$	$\{q_0, q_1\} = B$
$C = \{q_0, q_2\}$	$\{q_0, q_1\} = B$	$\{q_0, q_1, q_3\} = D$
$D = \{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\} = E$	$\{q_0, q_1, q_3\} = D$
$E = \{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\} = D$	$\{q_0, q_1, q_3\} = D$

得到 5 个可达状态。 $\bar{F}$ 的接受态是含 q_3 的状态： D, E 。

第三步: 取补得 F 的 5 状态 DFA。 将接受态翻转—— A, B, C 接受 (未含违反对), D, E 拒绝 (已确认违反)。



状态语义:

- A : 尚未读到 1 (空串 ε 或全 0 串)
- B : 上个 1 之后读了偶数个 0 (含 0 个)——若串此刻结束, 前面所有的 1-对间隔都合规
- C : 上个 1 之后读了奇数个 0——尚未确认违反 (若后面不再有 1 也可接受)
- D : 已经确认违反 (在 C 状态时读到过 1), 当前距最后的 1 读了偶数个 0
- E : 已经确认违反, 当前距最后的 1 读了奇数个 0

关键转移: $C \xrightarrow{1} E$ 是唯一“发生违反”的转移——在距上一个 1 奇数个 0 后又读到了一个 1。一旦进入 $\{D, E\}$ 就永远困在其中 (死区)。

1.14

- 1.14** a. 证明: 若 M 是一台识别语言 B 的 DFA, 交换 M 的接受状态与非接受状态得到一台新的 DFA, 则这台新 DFA 识别 B 的补集。因而, 正则语言类在补运算下封闭。
- b. 举例说明: 若 M 是一台识别语言 C 的 NFA, 交换 M 的接受状态与非接受状态, 得到一台新 NFA, 这台新 NFA 不一定识别 C 的补集。NFA 识别的语言类在补运算下封闭吗? 并解释回答。

图 18

解答

先证明对 DFA 成立, 再说明为什么对 NFA 直接交换接受状态与非接受状态并不正确。

DFA 情形. 设

$$M = (Q, \Sigma, \delta, q_0, F)$$

识别语言 B , 即

$$B = L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}.$$

构造新的 DFA

$$M' = (Q, \Sigma, \delta, q_0, Q \setminus F),$$

也就是只交换接受状态与非接受状态，而保持转移函数不变。

对任意字符串 $w \in \Sigma^*$ ，由于 DFA 的计算路径唯一，

$$w \in L(M') \iff \delta^*(q_0, w) \in Q \setminus F \iff \delta^*(q_0, w) \notin F.$$

另一方面，

$$\delta^*(q_0, w) \notin F \iff w \notin L(M) \iff w \notin B.$$

因此

$$L(M') = \Sigma^* \setminus B = \overline{B}.$$

所以，只要把一个识别 B 的 DFA 的接受状态与非接受状态互换，就得到一个识别 $\overline{B}$ 的 DFA。于是，若 B 是正则语言，则 $\overline{B}$ 也是正则语言。换言之，正则语言类在补运算下封闭。

NFA 情形. 对于 NFA，直接交换接受状态与非接受状态不一定得到补语言。下面给出一个反例。

设字母表

$$\Sigma = \{a\}.$$

构造 NFA

$$N = (Q, \Sigma, \delta, q_0, F),$$

其中

$$Q = \{q_0, q_1\}, \quad F = \{q_1\},$$

转移为

$$\delta(q_0, a) = \{q_0, q_1\}, \quad \delta(q_1, a) = \{q_1\}.$$

这个 NFA 识别的语言是

$$C = \{a^n \mid n \geq 1\} = a^+.$$

原因是：只要输入非空串，就可以在某一步从 q_0 转到 q_1 ，并最终停在接受状态。

现在交换接受状态与非接受状态，得到

$$F' = Q \setminus F = \{q_0\}.$$

设新 NFA 为

$$N' = (Q, \Sigma, \delta, q_0, F').$$

由于对任意 a^n 都可以始终停留在状态 q_0 ，所以

$$L(N') = a^*.$$

但

$$\overline{C} = \Sigma^* \setminus a^+ = \{\varepsilon\}.$$

显然

$$L(N') = a^* \neq \{\varepsilon\} = \overline{C}.$$

因此，对 NFA 直接交换接受状态与非接受状态，并不一定得到原语言的补集。

原因分析. DFA 的接受基于唯一计算路径, 因此

$$w \in L(M) \iff \text{这唯一一条计算路径接受 } w.$$

所以把接受态与非接受态交换后, 恰好就把“接受”变成“拒绝”, 把“拒绝”变成“接受”。

但 NFA 的接受条件是

$$w \in L(N) \iff \text{存在一条计算路径接受 } w.$$

交换接受状态后, 新 NFA 接受的是

存在一条计算路径停在原来的非接受状态。

而补语言要求的是

$$w \in \overline{L(N)} \iff \text{不存在任何一条计算路径接受 } w,$$

也就是

$\forall$ 计算路径, 都不接受。

这里“存在一条路径”与“所有路径都不接受”并不等价, 所以不能通过简单交换终态来取补。

结论. 虽然对 NFA 不能直接交换接受状态来取补, 但 NFA 所识别的语言类仍然在补运算下封闭。因为任意 NFA 都可以先通过子集构造法转化为等价的 DFA, 再对该 DFA 交换接受状态与非接受状态, 即可得到补语言的识别器。■

1.15

1.15 给出一个反例, 说明下述构造不能证明定理 1.24, 即正则语言类在星号运算下封闭^①。

设 $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ 识别 A_1 。如下构造 $N = (Q, \Sigma, \delta, q, F)$ 。 N 应该识别 A_1^* 。

- N 的状态集是 N_1 的状态集。
- N 的起始状态与 N_1 的起始状态相同。
- $F = \{q_1\} \cup F_1$ 。

F 的接受状态是原来的接受状态加上它的起始状态。

- 定义 δ 如下: 对每一个 $q \in Q$ 和每一个 $a \in \Sigma_\epsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \notin F_1 \text{ 或 } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \notin F_1 \text{ 或 } a = \epsilon \end{cases}$$

(建议: 把这个构造转换成图, 如图 1-26 所示。)

图 19

这题中文版的第二版和第三版均存在翻译错误, 下图20 是英文版的题干

1.15 Give a counterexample to show that the following construction fails to prove Theorem 1.49, the closure of the class of regular languages under the star operation.⁷ Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 . Construct $N = (Q_1, \Sigma, \delta, q_1, F)$ as follows. N is supposed to recognize A_1^* .

- a. The states of N are the states of N_1 .
- b. The start state of N is the same as the start state of N_1 .
- c. $F = \{q_1\} \cup F_1$.
The accept states F are the old accept states plus its start state.
- d. Define δ so that for any $q \in Q_1$ and any $a \in \Sigma_\varepsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \notin F_1 \text{ or } a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \varepsilon. \end{cases}$$

(Suggestion: Show this construction graphically, as in Figure 1.50.)

图 20

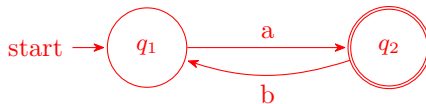
解答

这份证明的问题在于：所构造的新 NFA 并不一定识别 A_1^* 。下面给出一个反例。

反例的选择。 取

$$A_1 = a(ba)^*.$$

令 N_1 是识别 A_1 的 NFA，其状态图如下：



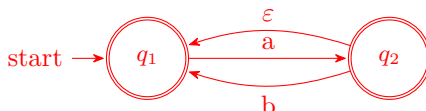
其中 q_1 是开始状态， q_2 是唯一接受状态。因此

$$L(N_1) = \{a(ba)^k \mid k \geq 0\} = a(ba)^* = A_1.$$

按题中错误方法构造新 NFA。 按照题目中的构造，新 NFA N 的接受状态变为

$$F = \{q_1\} \cup F_1 = \{q_1, q_2\},$$

并且因为 $q_2 \in F_1$ ，所以还要加入一条 ε -转移 $q_2 \rightarrow q_1$ 。于是得到下图：



说明该构造出错。 在这个新 NFA 中，

$$q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_1,$$

而 q_1 已经是接受状态，所以

$$ab \in L(N).$$

但是

$$ab \notin A_1^*.$$

这是因为 $A_1 = a(ba)^*$ 中的每个非空字符串都以 a 开头并以 a 结尾，因此 A_1^* 中每个非空块也都以 a 结尾，从而任意非空拼接结果仍以 a 结尾。字符串 ab 以 b 结尾，所以不可能属于 A_1^* 。

因此

$$L(N) \neq A_1^*.$$

这就说明题目中的构造不能用来证明正则语言类对 Kleene 星号运算封闭。

错误的本质. 问题出在它直接把原开始状态 q_1 设成了接受状态。这样一来，只要某条路径在“尚未完整读完一个 A_1 -块”时返回到了 q_1 ，机器就可能提前接受，从而识别出不属于 A_1^* 的字符串。

正确的做法是：新建一个**新的开始接受状态**，并从它向旧开始状态加一条 ε -边；同时从每个旧接受状态向旧开始状态加 ε -边。这样才能正确识别 A_1^* 。■

1.17

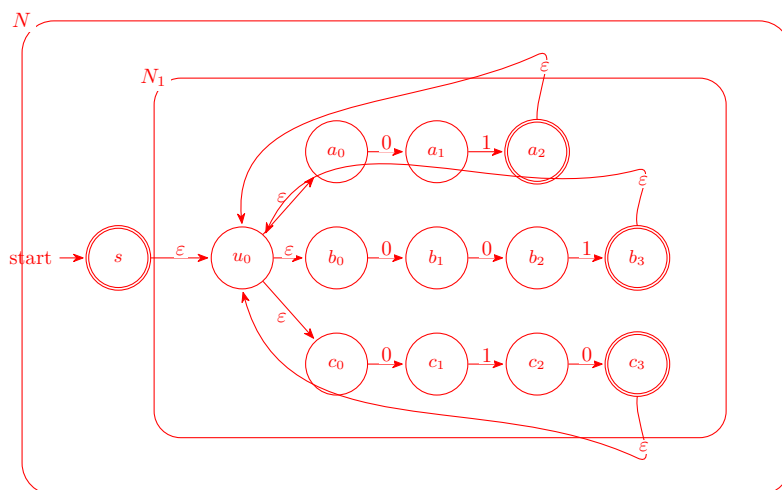
1.17 a. 给出识别语言 $(01 \cup 001 \cup 010)^*$ 的 NFA。

b. 将给出的 NFA 转换成等价的 DFA。给出在该 DFA 开始状态可达的部分即可。

图 21

解答

识别 $(01 \cup 001 \cup 010)^*$ 的 NFA. 按照正则表达式到 NFA 的标准构造，先分别构造识别 01、001、010 的三个小 NFA，再用 ε 边做并运算得到 N_1 ，最后在外层按星号构造得到 N 。



其中 N_1 识别 $01 \cup 001 \cup 010$ ，上、中、下三条分支分别识别 01、001、010。外层 N 的新开始状态 s 同时是接受状态，因此接受空串；从 s 到 u_0 的 ε 边进入 N_1 ，从 N_1 的接受状态回到 u_0 的 ε 边允许重复读下一个块。

子集构造得到的等价 DFA. 为了得到较小的 DFA，可以先把上面的构造 NFA 化简为只记录“当前匹配到块内哪个前缀”的四状态 NFA。设该 NFA 的状态为 q_0, q_1, q_2, q_3 ，其中 q_0 是开始状态和接受状态，并有如下含义：

状态	含义
q_0	当前正好停在块边界，可以接受；
q_1	已经读到某个块的前缀 0；
q_2	已经读到某个块的前缀 00；
q_3	已经读到某个块的前缀 01，还可继续读 0 形成 010。

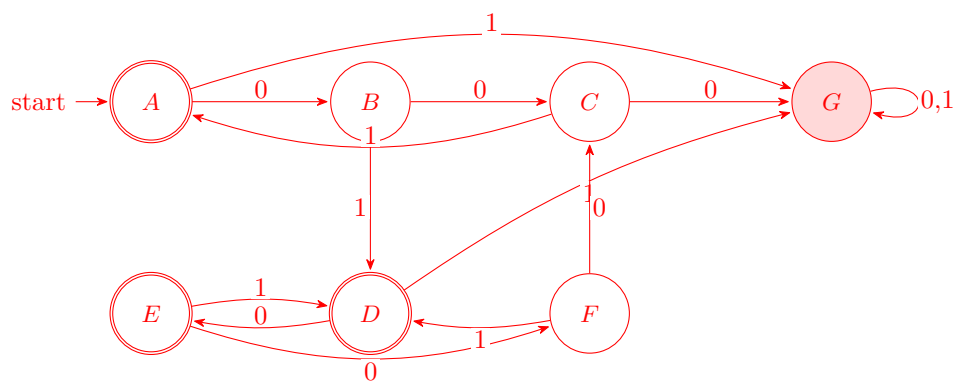
对这个 NFA 做子集构造时，DFA 的每个状态都是一个 NFA 状态集合。只保留从开始集合 $\{q_0\}$ 出发能够到达的集合，得到：

DFA 状态	NFA 状态集合	为什么会出现
A	$\{q_0\}$	开始集合
B	$\{q_1\}$	A 读 0
C	$\{q_2\}$	B 读 0
D	$\{q_0, q_3\}$	B 或 F 读 1
E	$\{q_0, q_1\}$	D 读 0
F	$\{q_1, q_2\}$	E 读 0
G	$\emptyset$	无合法转移时进入的死状态

它们之间的转移由逐点取并得到：

状态	0	1
A	B	G
B	C	D
C	G	A
D	E	G
E	F	D
F	C	D
G	G	G

因为原 NFA 的唯一接受态是 q_0 ，所以 DFA 中所有包含 q_0 的状态都是接受状态，即 A, D, E ； $G = \emptyset$ 是死状态。对应的 DFA 如下。



1.19

1.19 对于下述每一个语言，给出 4 个字符串，其中 2 个是这个语言的成员，2 个不是这个语言

图 22

的成员。这里假设字母表 $\Sigma = \{a, b\}$ 。

- | | |
|-------------------|--|
| a. a^*b^* | e. $\Sigma^*a\Sigma^*b\Sigma^*a\Sigma^*$ |
| b. $a(ba)^*b$ | f. $aba \cup bab$ |
| c. $a^* \cup b^*$ | g. $(\epsilon \cup a)b$ |
| d. $(aaa)^*$ | h. $(a \cup ba \cup bb)\Sigma^*$ |

图 23

解答

下面给出每个语言的两个成员和两个非成员。这里约定字母表为 $\Sigma = \{a, b\}$ 。

题号	语言	属于该语言的字符串	不属于该语言的字符串
(a)	a^*b^*	$\epsilon, aaabbb$	$ba, abab$
(b)	$a(ba)^*b$	$ab, abab$	ϵ, aab
(c)	$a^* \cup b^*$	aaa, bb	ab, ba
(d)	$(aaa)^*$	$\epsilon, aaaaaa$	a, aa
(e)	$\Sigma^*a\Sigma^*b\Sigma^*a\Sigma^*$	$aba, baaba$	ϵ, aab
(f)	$aba \cup bab$	aba, bab	ϵ, ab
(g)	$(\epsilon \cup a)b$	b, ab	ϵ, a
(h)	$(a \cup ba \cup bb)\Sigma^*$	a, ba	ϵ, b

其中 (e) 要求字符串中存在一个按顺序出现的子序列 a, b, a ；例如 $baaba$ 中第 2, 4, 5 个字符可取为 a, b, a 。而 (h) 表示字符串必须以 a 、 ba 或 bb 开头，所以空串和单独的 b 都不属于该语言。

1.30

- 1.30** 指出下述关于 0^*1^* 不是正则语言的“证明”的错误(由于 0^*1^* 是正则语言所以“证明”存在错误)：用反证法证明。假设 0^*1^* 是正则的。设 p 是泵引理给出的关于 0^*1^* 的泵长度。选择 s 为串 0^p1^p 。已知 s 是 0^*1^* 的一个成员，但例 1.38 表明 s 不能被抽取。这样得到矛盾。因此 0^*1^* 不是正则的。

图 24

解答

该证明的错误在于混淆了两个不同的语言。语言

$$0^*1^* = \{0^m1^n \mid m, n \geq 0\}$$

本身由正则表达式 0^*1^* 描述，因而是正则语言，不可能由泵引理推出其非正则性。

错误发生在把例 1.38 中关于

$$B = \{0^n1^n \mid n \geq 0\}$$

的结论用于 0^*1^* 。对于 B ，字符串 0^p1^p 不能被泵，是因为泵动会破坏 0 与 1 的数量相等；但 0^*1^* 并不要求二者数量相等，只要求所有 0 在所有 1 之前。

事实上，取

$$s = 0^p1^p \in 0^*1^*,$$

可作分解

$$x = \epsilon, \quad y = 0, \quad z = 0^{p-1}1^p.$$

于是对任意 $i \geq 0$,

$$xy^iz = 0^{p-1+i}1^p \in 0^*1^*.$$

因此该字符串在语言 0^*1^* 中是可以被泵的。

泵引理用于证明非正则时，必须证明：存在某个选定的 $s \in A$ ，使得对所有满足条件的分解 $s = xyz$ ，都能找到 $i \geq 0$ 令 $xy^iz \notin A$ 。题中证明只借用了 $B = \{0^n1^n \mid n \geq 0\}$ 的结论，而该结论不能迁移到约束不同的语言 0^*1^* 。■

方法整理

证明一个语言正则，通常采用构造法。也就是说，给出下列任意一种等价描述即可：

$$\text{正则表达式} \iff \text{NFA} \iff \text{DFA}.$$

因此，若能直接写出正则表达式，或构造出识别该语言的 NFA/DFA，就已经证明了该语言正则。例如 0^*1^* 由正则表达式本身描述，所以正则。

另一种常用方法是使用正则语言的闭包性质。已知正则语言类在并、连接、星号、补、交、差、逆、同态和逆同态等运算下封闭。因此，如果目标语言可由已知正则语言经过这些运算得到，也可推出其正则性。

证明一个语言非正则，则通常用反证法：先假设该语言正则，再推出矛盾。常用工具有：

- (1) 泵引理：设语言正则，取足够长且结构受限的字符串 s ，证明对任意合法分解 $s = xyz$ ，总能选择某个 i 使 xy^iz 离开该语言。
- (2) 闭包性质反证：若假设 A 正则，则通过与某个正则语言求交、取差、同态等操作得到一个已知非正则语言，从而矛盾。
- (3) 区分串方法：构造无限多个两两可区分的前缀，说明任何 DFA 都需要无限多个状态，因此语言非正则。

需要注意，泵引理给出的是正则语言的必要条件，而不是充分条件。其逻辑形式是

$$A \text{ 正则} \implies A \text{ 满足泵引理}.$$

因此可以使用逆否命题

$$A \text{ 不满足泵引理} \implies A \text{ 非正则},$$

但不能反过来认为

$$A \text{ 满足泵引理} \implies A \text{ 正则}.$$

所以，泵引理是证明非正则的工具，而不是证明正则的工具。证明正则时应给出 RE、NFA、DFA，或把语言化归为正则语言闭包运算的结果。

1.32

1.32 设

$$\Sigma_3 = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \dots, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\}$$

Σ_3 包含所有高度为 3 的 0 和 1 的列。 Σ_3 上的字符串给出三行 0 和 1。把每一行看作一个二进制数，令

$$B = \{w \in \Sigma_3^* \mid w \text{ 最下面的一行等于上面两行的和}\}$$

例如，

$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \in B, \text{ 但是 } \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \notin B$$

证明 B 是正则的 (提示: 证明 B^R 是正则的会容易一些。假设已知问题 1.31 中的结果)。

图 25

证明

二进制加法应从最低位向最高位检查。故先证明反转语言 B^R 正则，再由正则语言对反转封闭推出 B 正则。

设输入字母为三位列

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix}, \quad a, b, c \in \{0, 1\},$$

分别表示两个加数与和的对应二进制位。在 B^R 中，读入顺序正是从最低位到最高位，所以只需记录当前进位。

构造 DFA

$$M = (Q, \Sigma_3, \delta, q_0, F),$$

其中

$$Q = \{q_0, q_1, q_d\}, \quad F = \{q_0\}.$$

状态 q_0, q_1 分别表示当前进位为 0, 1，而 q_d 为死状态。对 $r \in \{0, 1\}$ 与任意列字母

$$\sigma = \begin{bmatrix} a \\ b \\ c \end{bmatrix},$$

定义转移如下：若

$$c \equiv a + b + r \pmod{2},$$

则

$$\delta(q_r, \sigma) = q_{\lfloor (a+b+r)/2 \rfloor};$$

否则令 $\delta(q_r, \sigma) = q_d$ 。并且对一切 $\sigma \in \Sigma_3$,

$$\delta(q_d, \sigma) = q_d.$$

该 DFA 恰好逐位验证二进制加法，并把进位传递到下一列。输入结束时接受当且仅当最终进位为 0，即当前状态为 q_0 。因此 B^R 是正则语言。由于正则语言对反转封闭，

$$B = (B^R)^R$$

也是正则语言。■

解释. 这个证明的核心是找出二进制加法检查中真正需要记住的信息。若从右向左读，即从最低位向最高位读，那么已经读过的低位对后续检查的影响只体现为当前进位。进位只有两种可能，0 或 1，所以这是有限信息，可以由 DFA 的有限状态记录。

具体地，DFA 处在 q_r 时表示“到目前为止读过的低位都合法，且传给下一位的进位为 r ”。读入一列

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

后，DFA 检查 c 是否等于 $a + b + r$ 的末位。如果不是，就进入死状态；如果是，就把状态更新为新的进位

$$\left\lfloor \frac{a + b + r}{2} \right\rfloor.$$

因此 DFA 并不需要记住已经读过的全部数字，只需要记住一个有限状态：进位是否为 1。这正是该语言正则的原因。

1.34

1.34 设 Σ_2 与问题 1.33 中的相同。把每一行看作是一个二进制数，并且设

$$D = \{w \in \Sigma_2^* \mid w \text{ 上一行表示的数大于下一行表示的数}\}$$

例如，

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in D, \quad \text{但是} \quad \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \notin D$$

证明 D 是正则的。

图 26

证明

设字母表为

$$\Sigma_2 = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

字符串

$$w = \begin{bmatrix} a_1 \\ b_1 \end{bmatrix} \begin{bmatrix} a_2 \\ b_2 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ b_n \end{bmatrix}$$

表示两行二进制数 $a_1 a_2 \cdots a_n$ 与 $b_1 b_2 \cdots b_n$ 。从左向右比较二者时，大小关系由最左边第一个不同位决定。

构造 DFA

$$M = (Q, \Sigma_2, \delta, q_-, F),$$

其中

$$Q = \{q_-, q_+, q_-\}, \quad F = \{q_+\}.$$

状态 q_- 表示目前为止两行相等, q_+ 表示已经确定上面一行较大, q_- 表示已经确定上面一行较小。

在状态 q_- 时, 定义

$$\begin{aligned} \delta\left(q_-, \begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) &= q_-, & \delta\left(q_-, \begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) &= q_-, \\ \delta\left(q_-, \begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) &= q_+, & \delta\left(q_-, \begin{bmatrix} 0 \\ 1 \end{bmatrix}\right) &= q_-. \end{aligned}$$

一旦已经确定大小关系, 后续位不再改变结果。因此对任意 $\sigma \in \Sigma_2$,

$$\delta(q_+, \sigma) = q_+, \quad \delta(q_-, \sigma) = q_-.$$

该 DFA 恰好记录从左至右比较时所需的有限信息: 尚未分出大小、已经上大于下、已经上小于下。最终接受当且仅当最左边第一个不同位为

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

即上面一行表示的二进制数大于下面一行。因此 $L(M) = D$, 故 D 是正则语言。■

1.35

1.35 设 Σ_2 与问题 1.33 中的相同。把上、下行都看作是 0 和 1 的字符串, 并且设

$$E = \{w \in \Sigma_2^* \mid w \text{ 的下一行是上一行的反转}\}$$

证明 E 不是正则的。

图 27

证明

设

$$\Sigma_2 = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

若

$$w = \begin{bmatrix} a_1 \\ b_1 \end{bmatrix} \begin{bmatrix} a_2 \\ b_2 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ b_n \end{bmatrix},$$

则 $w \in E$ 当且仅当

$$b_1 b_2 \cdots b_n = a_n a_{n-1} \cdots a_1.$$

用泵引理反证。假设 E 正则, 令 p 为其泵长度。取

$$s = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^p \begin{bmatrix} 1 \\ 0 \end{bmatrix}^p.$$

其上行是 $0^p 1^p$, 下行是 $1^p 0^p = (0^p 1^p)^R$, 故 $s \in E$, 且 $|s| = 2p \geq p$ 。

由泵引理, 存在分解 $s = xyz$, 满足

$$|xy| \leq p, \quad |y| > 0,$$

并且对所有 $i \geq 0$, 有 $xy^iz \in E$ 。由于 $|xy| \leq p$, 子串 y 完全位于前 p 个字母中, 所以

$$y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^k \quad (k > 0).$$

令 $i = 0$, 得到

$$xz = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^{p-k} \begin{bmatrix} 1 \\ 0 \end{bmatrix}^p.$$

此时上行是 $0^{p-k}1^p$, 下行是 $1^{p-k}0^p$ 。然而

$$(0^{p-k}1^p)^R = 1^p0^{p-k} \neq 1^{p-k}0^p.$$

故 $xz \notin E$, 与泵引理要求 $xy^0z \in E$ 矛盾。因此 E 不是正则语言。■

解释. 该语言要求下行等于上行的反转。也就是说, 自动机读到前面的上行信息后, 必须在后面以相反顺序逐位比较。随着输入长度增加, 需要保存的前缀可以任意长; 而 DFA 只有有限多个状态, 不能保存任意长的前缀信息。这就是 E 非正则的本质原因。

这里并不是只考察了某一种特殊分解。泵引理要求对任意满足 $|xy| \leq p$ 、 $|y| > 0$ 的分解都能泵。我们选取的

$$s = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^p \begin{bmatrix} 1 \\ 0 \end{bmatrix}^p$$

有一个关键性质: 它的前 p 个符号全都是

$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

因此无论合法分解 $s = xyz$ 怎样选, 只要 $|xy| \leq p$ 且 $|y| > 0$, 子串 y 都必然具有形式

$$y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^k \quad (k > 0).$$

所以证明中令 $y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^k$ 并不是任取一种情况, 而是概括了所有合法分解。随后取 $i = 0$, 对任意这样的 $k > 0$ 都会破坏上下两行的反转关系, 故得到的矛盾满足泵引理反证所需的量词结构。

1.36

1.36 设 $B_n = \{a^k \mid k \text{ 是 } n \text{ 的整数倍}\}$ 。证明: 对每一个 $n \geq 1$, 语言 B_n 是正则的。

图 28

证明

固定任意 $n \geq 1$ 。构造 DFA

$$M_n = (Q, \{a\}, \delta, q_0, F),$$

其中

$$Q = \{q_0, q_1, \dots, q_{n-1}\}, \quad F = \{q_0\}.$$

状态 q_i 表示目前已经读入的 a 的个数模 n 余 i 。转移函数定义为

$$\delta(q_i, a) = q_{i+1 \bmod n}, \quad 0 \leq i \leq n-1.$$

于是对任意 $k \geq 0$,

$$\delta^*(q_0, a^k) = q_{k \bmod n}.$$

因此

$$a^k \in L(M_n) \iff \delta^*(q_0, a^k) = q_0 \iff k \equiv 0 \pmod{n}.$$

也就是说,

$$L(M_n) = \{a^k \mid k \text{ 是 } n \text{ 的整数倍}\} = B_n.$$

所以对每个 $n \geq 1$, B_n 都由一个有限状态自动机识别, 故 B_n 是正则语言。■

解释. 这里 DFA 需要记住的不是完整的长度 k , 而只是 k 除以 n 的余数。余数只有

$$0, 1, \dots, n-1$$

这 n 种可能, 因此可以用有限多个状态记录。每读入一个 a , 余数加 1 并对 n 取模; 最终回到余数 0 的状态, 正好表示长度是 n 的整数倍。

1.40

1.40 如果存在字符串 z 使得 $xz=y$, 则称字符串 x 是字符串 y 的**前缀**(prefix)。如果 x 是 y 的前缀且 $x \neq y$, 则称 x 是 y 的**真前缀**(proper prefix)。下面每小题目定义一个语言 A 上的运算。证明: 正则语言类在每个运算下封闭。

- ^A a. $\text{NOPREFIX}(A) = \{w \in A \mid w \text{ 任一真前缀都不是 } A \text{ 的元素}\}$
 b. $\text{NOEXTEND}(A) = \{w \in A \mid w \text{ 不是 } A \text{ 中任何字符串的真前缀}\}$

图 29

证明

设 A 是正则语言, 令

$$M = (Q, \Sigma, \delta, q_0, F)$$

为识别 A 的 DFA。

(a) 先证明 $\text{NOPREFIX}(A)$ 正则。

构造 DFA

$$M_1 = (Q \cup \{q_d\}, \Sigma, \delta_1, q_0, F),$$

其中 q_d 是死状态。定义

$$\delta_1(q_d, a) = q_d \quad (a \in \Sigma),$$

且对 $q \in Q$,

$$\delta_1(q, a) = \begin{cases} q_d, & q \in F, \\ \delta(q, a), & q \notin F. \end{cases}$$

这个构造的含义是：一旦读到某个已经属于 A 的前缀，而输入还没有结束，则该前缀就是真前缀，于是进入死状态。只有当第一次到达接受状态恰好发生在输入结束时，才接受。因此

$$L(M_1) = \text{NOPREFIX}(A),$$

所以 $\text{NOPREFIX}(A)$ 正则。

(b) 再证明 $\text{NOEXTEND}(A)$ 正则。

关键是筛掉那些“还能继续走到接受态”的状态。把 M 看成一个有限有向图：状态是顶点，每条转移边

$$q \xrightarrow{a} \delta(q, a)$$

是一条带标号的有向边。定义

$$P = \{q \in Q \mid \text{从 } q \text{ 出发存在一条长度至少为1的路径通向 } F\}.$$

等价地，

$$P = \{q \in Q \mid \exists x \in \Sigma^+, \delta^*(q, x) \in F\}.$$

这里并不是枚举无穷多个字符串 x ，而是在有限状态图上做可达性判断。

具体地，先在反向图中从所有接受状态 F 出发做 BFS 或 DFS，得到

$$R = \{q \in Q \mid \text{从 } q \text{ 出发可以经零步或多步到达 } F\}.$$

然后取

$$P = \{q \in Q \mid \exists a \in \Sigma, \delta(q, a) \in R\}.$$

这样正好保证从 q 到接受态至少走了一步；特别地，若某个接受状态有自环或能绕一圈回到接受状态，它也会被放入 P 。

构造 DFA

$$M_2 = (Q, \Sigma, \delta, q_0, F'), \quad F' = F \setminus P.$$

也就是说，状态、字母表、转移函数、开始状态全部不变，只把接受状态改为 $F \setminus P$ 。

若 w 被 M_2 接受，设

$$r = \delta^*(q_0, w).$$

则 $r \in F' = F \setminus P$ 。所以 $r \in F$ ，从而 $w \in A$ ；同时 $r \notin P$ ，说明从 r 出发不可能再经非空路径到达任何接受状态。于是不存在非空字符串 z 使得 $wz \in A$ ，故 w 不是 A 中任何字符串的真前缀。

反过来，若 $w \in A$ 且 w 不是 A 中任何字符串的真前缀，令 $r = \delta^*(q_0, w)$ 。由于 $w \in A$ ，有 $r \in F$ ；又因为 w 不能继续扩展成 A 中的字符串，从 r 出发不存在通向接受状态的非空路径，所以 $r \notin P$ 。于是 $r \in F \setminus P = F'$ ，故 M_2 接受 w 。

因此

$$L(M_2) = \text{NOEXTEND}(A),$$

所以 $\text{NOEXTEND}(A)$ 正则。

综上，正则语言类在 NOPREFIX 与 NOEXTEND 运算下封闭。■

解释。 $\text{NOPREFIX}(A)$ 要求“在接受当前串之前，不能已经接受过某个真前缀”，所以 DFA 只需在读入过程中检查是否提前进入过接受状态；若提前接受后还有输入，就进入死状态。

这里使用的 δ^* 是 DFA 转移函数 δ 的扩展。若

$$\delta : Q \times \Sigma \rightarrow Q$$

表示读入一个字符后的状态转移, 则

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

表示从某个状态出发, 读完整个字符串后到达的状态。它由

$$\delta^*(q, \varepsilon) = q, \quad \delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

递归定义, 其中 $w \in \Sigma^*$, $a \in \Sigma$ 。因此 $\delta^*(q, z) \in F$ 的意思是: 从状态 q 出发读完整个字符串 z 后到达接受状态。这是自动机理论中的标准记号。

NOEXTEND(A) 要求 “当前串虽然在 A 中, 但不能继续扩展成另一个 A 中的串”。从改 DFA 的角度看, 就是检查每个接受状态后面是否还存在通往某个接受状态的非空路径: 若存在, 就说明停在该状态的字符串还能继续扩展成 A 中更长的字符串, 因此这个状态不能保留为接受状态; 若不存在, 才保留它。这个过程完全发生在有限状态图上, 所以是有限的构造。

什么叫封闭。一般地, 称一类语言 $\mathcal{C}$ 在某个运算 T 下封闭, 是指对 $\mathcal{C}$ 中的语言施行该运算后, 所得结果仍属于 $\mathcal{C}$ 。若 T 是一元运算, 则要求

$$A \in \mathcal{C} \implies T(A) \in \mathcal{C};$$

若 T 是二元运算, 则要求

$$A, B \in \mathcal{C} \implies T(A, B) \in \mathcal{C}.$$

在本章里, $\mathcal{C}$ 就是正则语言类。因此 1.40 的结论是: 若 A 正则, 则 NOPREFIX(A) 与 NOEXTEND(A) 仍正则。证明这类封闭性命题的常见方法, 就是从识别原语言的 DFA 或 NFA 出发, 直接构造识别新语言的自动机。

1.42

1.42 对语言 A 和语言 B , 设 A 和 B 的完全间隔交叉(perfect shuffle)为:

$$\{w \mid w = a_1 b_1 \cdots a_k b_k, \text{ 其中 } a_1 \cdots a_k \in A \text{ 并且 } b_1 \cdots b_k \in B, \text{ 任一 } a_i, b_i \in \Sigma\}$$

证明: 正则语言类在完全间隔交叉下封闭。

图 30

证明

设 $A, B \subseteq \Sigma^*$ 都是正则语言。取识别它们的 DFA

$$M_A = (Q_A, \Sigma, \delta_A, q_A, F_A), \quad M_B = (Q_B, \Sigma, \delta_B, q_B, F_B).$$

我们构造 DFA

$$M = (Q_A \times Q_B \times \{0, 1\}, \Sigma, \delta, (q_A, q_B, 0), F),$$

其中第三个分量用来记录当前读入的位置奇偶:

$$0: \text{下一位应送入 } M_A, \quad 1: \text{下一位应送入 } M_B.$$

接受状态取为

$$F = F_A \times F_B \times \{0\}.$$

这表示输入已经恰好读完若干对符号，并且奇数位形成的串被 M_A 接受，偶数位形成的串被 M_B 接受。

转移函数定义为

$$\delta((p, q, 0), a) = (\delta_A(p, a), q, 1), \quad \delta((p, q, 1), b) = (p, \delta_B(q, b), 0),$$

其中 $p \in Q_A$, $q \in Q_B$, $a, b \in \Sigma$ 。

下面证明 $L(M)$ 正是 A 与 B 的完全间隔交叉。

若

$$w = a_1 b_1 a_2 b_2 \cdots a_k b_k$$

且

$$a_1 a_2 \cdots a_k \in A, \quad b_1 b_2 \cdots b_k \in B,$$

则 M 读入奇数位时只更新第一个分量，读入偶数位时只更新第二个分量。因此读完整个 w 后到达状态

$$(\delta_A^*(q_A, a_1 a_2 \cdots a_k), \delta_B^*(q_B, b_1 b_2 \cdots b_k), 0).$$

由于两行分别属于 A 与 B ，前两个分量落在 F_A 与 F_B 中；又因为 w 长度为偶数，第三个分量回到 0。故 $w \in L(M)$ 。

反过来，若 $w \in L(M)$ ，则第三个分量最终为 0，所以 w 的长度必为偶数。于是可唯一写成

$$w = a_1 b_1 a_2 b_2 \cdots a_k b_k.$$

设

$$x = a_1 a_2 \cdots a_k, \quad y = b_1 b_2 \cdots b_k.$$

根据转移定义， M 读完 w 后的前两个分量分别正是

$$\delta_A^*(q_A, x), \quad \delta_B^*(q_B, y).$$

又因为 w 被接受，这两个状态分别属于 F_A 与 F_B ，故

$$x \in A, \quad y \in B.$$

因此 w 正是由 $x \in A$ 与 $y \in B$ 按完全间隔交叉得到的字符串。

综上，

$$L(M) = \{w \mid w \text{ 是 } A \text{ 与 } B \text{ 的完全间隔交叉}\}.$$

所以正则语言类在完全间隔交叉运算下封闭。■

解释. 这个构造的核心很直接：DFA 一边读输入，一边把奇数位送给识别 A 的自动机，把偶数位送给识别 B 的自动机；额外的一个二值状态只负责记录“当前该轮到哪一边读”。由于需要记住的信息只有两个分自动机的当前状态以及当前位置的奇偶性，所以总状态数仍然是有限的。

就 1.42 而言，这里讨论的运算是“完全间隔交叉”：输入是两个语言 A, B ，输出是由 A 中字符串与 B 中字符串按奇偶位交错得到的所有字符串组成的语言。所谓“正则语言类在这个运算下封闭”，意思正是：只要 A 与 B 都正则，那么它们的完全间隔交叉所得语言也仍然正则。上面的乘积 DFA 就给出了这一点的构造性证明。

1.52

- 1.52 Myhill-Nerode 定理。**参见问题 1.51, 设 L 是一个语言, X 是一个字符串集合。如果 X 中的任意两个不同的字符串都是用 L 可区分的, 则称 X 是用 L 两两可区分的。定义 L 的指数为用 L 两两可区分的集合中的元素个数的最大值。 L 的指数可能是有穷的或无穷的。
- 证明: 如果 L 被一台有 k 个状态的 DFA 识别, 则 L 的指数不超过 k 。
 - 证明: 如果 L 的指数是一个有穷数 k , 则它被一台有 k 个状态的 DFA 识别。
 - 由此得到: L 是正则的当且仅当它有有穷的指数。而且, 它的指数是识别它的最小 DFA 的大小。

图 31

证明

记

$$x \equiv_L y$$

表示字符串 $x, y \in \Sigma^*$ 用 L 不可区分, 即

$$\forall z \in \Sigma^*, \quad xz \in L \iff yz \in L.$$

若存在某个 $z \in \Sigma^*$ 使得

$$xz \in L, \quad yz \notin L$$

或反过来, 则称 x, y 可由 L 区分。(a) 设 L 被一台有 k 个状态的 DFA

$$M = (Q, \Sigma, \delta, q_0, F)$$

识别, 并且 $|Q| = k$ 。任取一个用 L 两两可区分的集合

$$X \subseteq \Sigma^*.$$

对每个 $x \in X$, 看它从初态出发到达的状态 $\delta^*(q_0, x)$ 。若存在不同的 $x, y \in X$ 满足

$$\delta^*(q_0, x) = \delta^*(q_0, y),$$

则对任意 $z \in \Sigma^*$, 都有

$$\delta^*(q_0, xz) = \delta^*(\delta^*(q_0, x), z) = \delta^*(\delta^*(q_0, y), z) = \delta^*(q_0, yz).$$

因而

$$xz \in L \iff yz \in L.$$

这说明 x, y 用 L 不可区分, 与 X 中任意两个不同字符串都应可区分矛盾。因此, X 中不同字符串必须到达不同状态。于是映射

$$x \mapsto \delta^*(q_0, x)$$

从 X 到 Q 是单射, 所以

$$|X| \leq |Q| = k.$$

故 L 的指数不超过 k 。

- (b) 设 L 的指数有限, 且等于 k 。因为 L 的指数为 k , 所以关系 $\equiv_L$ 将 Σ^* 分成 k 个等价类。以这些等价类为状态, 构造 DFA

$$M = (Q, \Sigma, \delta, q_0, F).$$

令

$$Q = \{[x] \mid x \in \Sigma^*\},$$

其中

$$[x] = \{y \in \Sigma^* \mid y \equiv_L x\}.$$

因为指数是 k , 所以 $|Q| = k$ 。

初始状态定义为

$$q_0 = [\varepsilon].$$

转移函数定义为

$$\delta([x], a) = [xa].$$

需要说明它是良定义的。若

$$[x] = [y],$$

即 $x \equiv_L y$, 则对任意 $z \in \Sigma^*$, 因为 $x \equiv_L y$ 对字符串 az 也成立, 所以

$$xaz \in L \iff yaz \in L.$$

也就是

$$(xa)z \in L \iff (ya)z \in L.$$

因此 $xa \equiv_L ya$, 从而

$$[xa] = [ya].$$

所以转移函数定义良好。

接受状态定义为

$$F = \{[x] \mid x \in L\}.$$

这同样是良定义的。若 $[x] = [y]$, 即 $x \equiv_L y$, 取 $z = \varepsilon$, 则

$$x \in L \iff y \in L.$$

因而同一个等价类中的字符串要么全在 L 中, 要么全不在 L 中, 所以接受状态集合定义良好。

下面证明该 DFA 识别 L 。对任意字符串

$$w = a_1 a_2 \cdots a_n,$$

由转移定义可归纳得到

$$\delta^*([\varepsilon], w) = [w].$$

因此

$$w \in L(M) \iff \delta^*([\varepsilon], w) \in F \iff [w] \in F \iff w \in L.$$

所以

$$L(M) = L.$$

因而 L 被一台有 k 个状态的 DFA 识别。

(c) 由 (a) 可知, 若 L 被某台 k 状态 DFA 识别, 则 L 的指数不超过 k 。因此, 若 L 是正则语言, 则存在有限状态 DFA 识别它, 从而 L 的指数有限。

由 (b) 可知, 若 L 的指数有限且等于 k , 则可构造一台有 k 个状态的 DFA 识别 L 。所以

$$L \text{ 正则} \iff L \text{ 的指数有限.}$$

此外, 因为任何识别 L 的 DFA 的状态数都至少是 L 的指数, 而我们又能构造一台恰有指数个状态数的 DFA, 所以

$$L \text{ 的指数} = \text{识别 } L \text{ 的最小 DFA 的状态数.}$$

■

解释. Myhill–Nerode 定理的核心是: DFA 的一个状态只代表“从这里继续往后读, 未来会发生什么”。若两个前缀 x, y 对所有后缀 z 都表现完全一样, 即

$$xz \in L \iff yz \in L,$$

那么自动机没有必要把它们分开; 反之, 只要存在某个后缀能把它们区分开, 自动机就必须给它们不同的状态。因此, 最小 DFA 的状态本质上就是这些“未来行为”不同的等价类。

1.53

设 $\Sigma = \{0, 1, +, =\}$ 以及

$$ADD = \{x = y + z \mid x, y, z \text{ 是二进制整数, 并且 } x \text{ 是 } y \text{ 与 } z \text{ 的和}\}$$

图 32

证明

设

$$ADD = \{x = y + z \mid x, y, z \text{ 是二进制整数, 且 } x = y + z\}.$$

我们用闭包性质反证。令

$$R = \{10^i = 10^j + 0 \mid i, j \geq 0\}.$$

显然 R 是正则语言, 因为它可由正则表达式

$$10^* = 10^* + 0$$

描述。

若 ADD 是正则语言, 则

$$L = ADD \cap R$$

也应是正则语言。但在 R 中, 字符串具有形式

$$10^i = 10^j + 0.$$

其表示的等式为

$$2^i = 2^j + 0.$$

该等式成立当且仅当 $i = j$ 。因此

$$L = \{10^n = 10^n + 0 \mid n \geq 0\}.$$

下面证明 L 非正则。假设 L 正则，令 p 为其泵长度。取

$$s = 10^p = 10^p + 0 \in L.$$

由泵引理，存在分解 $s = xyz$ ，满足

$$|xy| \leq p, \quad |y| > 0,$$

且对所有 $k \geq 0$ ，都有 $xy^kz \in L$ 。

由于 $|xy| \leq p$ ，子串 y 完全落在第一个 10^p 中。令 $k = 0$ ，即删除 y 。这样得到的 xz 会改变等号左边的二进制数，而等号右边的 $10^p + 0$ 不变。因此 xz 不可能仍具有形式

$$10^n = 10^n + 0.$$

所以

$$xz \notin L,$$

这与泵引理矛盾。故 L 非正则。

于是若 ADD 正则，则正则语言 R 与它的交 L 应正则；但 L 非正则，矛盾。因此

ADD

不是正则语言。■

解释. 这个证明的关键是用正则语言 R 把 ADD 限制到一种很简单的加法：

$$10^i = 10^j + 0.$$

此时加法本身没有复杂性，因为右边只是加 0；唯一要检查的是左右两个二进制数是否完全相同，即是否 $i = j$ 。这就把问题化为

$$\{10^n = 10^n + 0 \mid n \geq 0\},$$

而有限自动机无法记住等号左边有多少个 0，再在等号右边逐一比较。因此该语言非正则，从而 ADD 也不可能正则。

Chapter 2

2.4

2.4 给出产生下述语言的上下文无关文法：

- ^A a. $\{w \mid w \text{ 至少包含 3 个 } 1\}$ 。
- b. $\{w \mid w \text{ 以相同的符号开始和结束}\}$ 。
- c. $\{w \mid w \text{ 的长度为奇数}\}$ 。
- ^A d. $\{w \mid w \text{ 的长度为奇数且正中间的符号是 } 0\}$ 。
- e. $\{w \mid w = w^R, \text{ 即 } w \text{ 是一个回文}\}$ 。
- f. 空集

图 33

解答

以下各小题默认字母表为 $\Sigma = \{0, 1\}$ 。

(a) 取文法

$$S \rightarrow T1T1T1T, \quad T \rightarrow 0T \mid 1T \mid \varepsilon.$$

其中 T 生成任意二进制串，因此 S 恰好生成至少含有 3 个 1 的字符串。

(b) 取文法

$$S \rightarrow 0T0 \mid 1T1 \mid 0 \mid 1, \quad T \rightarrow 0T \mid 1T \mid \varepsilon.$$

这恰好生成首尾符号相同的所有非空字符串。

(c) 取文法

$$S \rightarrow 0E \mid 1E,$$

$$E \rightarrow 00E \mid 01E \mid 10E \mid 11E \mid \varepsilon.$$

其中 E 生成所有偶长度字符串，因此 S 生成所有奇长度字符串。

(d) 取文法

$$S \rightarrow 0 \mid 0S0 \mid 0S1 \mid 1S0 \mid 1S1.$$

基础情形是中间符号单独为 0；每次在左右两侧各添一个任意符号，故长度始终为奇数且中间符号保持为 0。

(e) 取文法

$$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon.$$

这正是回文串的标准上下文无关文法。

(f) 取文法

$$G = (\{S\}, \{0, 1\}, \emptyset, S).$$

即没有任何产生式，因此从开始变量 S 无法推出任何终结字符串，故 $L(G) = \emptyset$ 。

2.6

2.6 给出产生下述语言的上下文无关文法：

^A a. 字母表 $\{a, b\}$ 上 a 多于 b 的所有字符串组成的集合。

b. 语言 $\{a^n b^n \mid n \geq 0\}$ 的补集。

^A c. $\{w \# x \mid w, x \in \{0, 1\}^* \text{ 且 } w^R \text{ 是 } x \text{ 的子串}\}$ 。

d. $\{x_1 \# x_2 \# \cdots \# x_k \mid k \geq 1, \text{ 每一个 } x_i \in \{a, b\}^*, \text{ 且存在 } i \text{ 和 } j \text{ 使得 } x_i = x_j^R\}$ 。

图 34

解答

(a) 取文法

$$S \rightarrow EaE \mid EaS,$$

$$E \rightarrow aEbE \mid bEaE \mid \varepsilon.$$

其中 E 生成所有 a, b 个数相同的字符串。于是 S 每次额外保留一个尚未被配平的 a ，故恰好生成所有 a 的个数多于 b 的字符串。

(b) 这里补集是相对于 $\{a, b\}^*$ 而言。取文法

$$S \rightarrow X \mid Y \mid Z,$$

$$X \rightarrow aXb \mid aA, \quad A \rightarrow aA \mid \varepsilon,$$

$$Y \rightarrow aYb \mid B, \quad B \rightarrow bB \mid b,$$

$$Z \rightarrow TbaT, \quad T \rightarrow aT \mid bT \mid \varepsilon.$$

其中 X 生成所有形如 $a^i b^j$ 且 $i > j$ 的串， Y 生成所有形如 $a^i b^j$ 且 $i < j$ 的串， Z 生成所有包含子串 ba 的串，也就是所有不属于 $a^* b^*$ 的串。因此 S 生成的正是

$$\{a, b\}^* \setminus \{a^n b^n \mid n \geq 0\}.$$

(c) 取文法

$$S \rightarrow RA,$$

$$R \rightarrow 0R0 \mid 1R1 \mid \#B,$$

$$A \rightarrow 0A \mid 1A \mid \varepsilon, \quad B \rightarrow 0B \mid 1B \mid \varepsilon.$$

其中 R 生成所有形如

$$w \# u w^R$$

的字符串，而 A 再在右侧补上任意后缀。因此 S 恰好生成所有

$$w \# x \quad (w^R \text{ 是 } x \text{ 的子串})$$

的字符串。

(d) 设 T 生成任意 $\{a, b\}$ -串：

$$T \rightarrow aT \mid bT \mid \varepsilon.$$

再令

$$L \rightarrow \varepsilon \mid T\#L, \quad R \rightarrow \varepsilon \mid \#TR,$$

分别生成所选块之前与之后的任意若干块；令

$$M \rightarrow T \mid T\#M$$

生成两块之间的任意若干中间块。定义

$$C \rightarrow aCa \mid bCb \mid \# \mid \#M\#,$$

则 C 生成形如

$$x\#x^R \quad \text{或} \quad x\#y_1\#\cdots\#y_m\#x^R$$

的串，即存在两个互为逆序的块。又令

$$P \rightarrow aPa \mid bPb \mid a \mid b \mid \varepsilon,$$

它生成所有回文块。于是取开始变量

$$S \rightarrow LCR \mid LPR.$$

第一部分 LCR 处理存在 $i \neq j$ 且 $x_i = x_j^R$ 的情形；第二部分 LPR 处理 $i = j$ 时某一块本身就是回文的情形。故该文法生成的正是题目所给语言。

2.8

2.8 证明上下文无关语言类在并、连结和星号三种正则运算下封闭。

图 35

证明

设 A, B 是上下文无关语言。分别取生成它们的上下文无关文法

$$G_A = (V_A, \Sigma, R_A, S_A), \quad G_B = (V_B, \Sigma, R_B, S_B).$$

满足

$$L(G_A) = A, \quad L(G_B) = B.$$

不妨先把变量重命名，使

$$V_A \cap V_B = \emptyset.$$

(a) 并运算。

构造新文法

$$G_{\cup} = (V_A \cup V_B \cup \{S\}, \Sigma, R, S),$$

其中

$$R = R_A \cup R_B \cup \{S \rightarrow S_A \mid S_B\}.$$

从开始变量 S 出发，可先选择导出 A 中的串或 B 中的串，因此

$$L(G_{\cup}) = A \cup B.$$

(b) 连接运算。

构造新文法

$$G_{\circ} = (V_A \cup V_B \cup \{S\}, \Sigma, R', S),$$

其中

$$R' = R_A \cup R_B \cup \{S \rightarrow S_A S_B\}.$$

于是从 S 出发，先导出 A 中某个字符串，再导出 B 中某个字符串，因此

$$L(G_{\circ}) = AB.$$

(c) 星号运算。

设

$$G_A = (V, \Sigma, R_A, S_A)$$

满足 $L(G_A) = A$ 。构造新文法

$$G_* = (V \cup \{S\}, \Sigma, R'', S),$$

其中

$$R'' = R_A \cup \{S \rightarrow \varepsilon \mid S_A S\}.$$

产生式 $S \rightarrow \varepsilon$ 对应零次连接，产生式 $S \rightarrow S_A S$ 对应先取一个 A 中的串，再继续重复。因此

$$L(G_*) = A^*.$$

综上，上下文无关语言类在并、连接与星号运算下封闭。■

解释. 这类闭包证明的核心思路是：已知原语言能由 CFG 生成，就直接在 CFG 上做拼装。并运算是在新开始变量处“二选一”；连接运算是让新开始变量先展开成两个旧开始变量；星号运算则是在新开始变量上加入“停止”与“继续接一个块”两种选择。由于这些改动都只引入有限多个新变量和产生式，所得仍是上下文无关文法。

2.10

2.10 给出产生语言

$$A = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ 且 } i = j \text{ 或 } j = k\}$$

图 36

的上下文无关文法。你给出的文法是有歧义的吗？为什么？

图 37

解答

将该语言写成两个较简单语言的并：

$$A = \{a^i b^j c^k \mid i = j\} \cup \{a^i b^j c^k \mid j = k\}.$$

也就是

$$A = \{a^n b^n c^k \mid n, k \geq 0\} \cup \{a^i b^n c^n \mid i, n \geq 0\}.$$

取文法

$$\begin{aligned} S &\rightarrow XC \mid AY, \\ X &\rightarrow aXb \mid \varepsilon, \quad C \rightarrow cC \mid \varepsilon, \\ A &\rightarrow aA \mid \varepsilon, \quad Y \rightarrow bYc \mid \varepsilon. \end{aligned}$$

其中

$$L(X) = \{a^n b^n \mid n \geq 0\}, \quad L(C) = \{c^k \mid k \geq 0\},$$

所以

$$L(XC) = \{a^n b^n c^k \mid n, k \geq 0\}.$$

又

$$L(A) = \{a^i \mid i \geq 0\}, \quad L(Y) = \{b^n c^n \mid n \geq 0\},$$

所以

$$L(AY) = \{a^i b^n c^n \mid i, n \geq 0\}.$$

因此

$$L(S) = L(XC) \cup L(AY) = A.$$

这个文法是有歧义的。原因是

$$\{a^n b^n c^n \mid n \geq 0\} \subseteq L(XC) \cap L(AY),$$

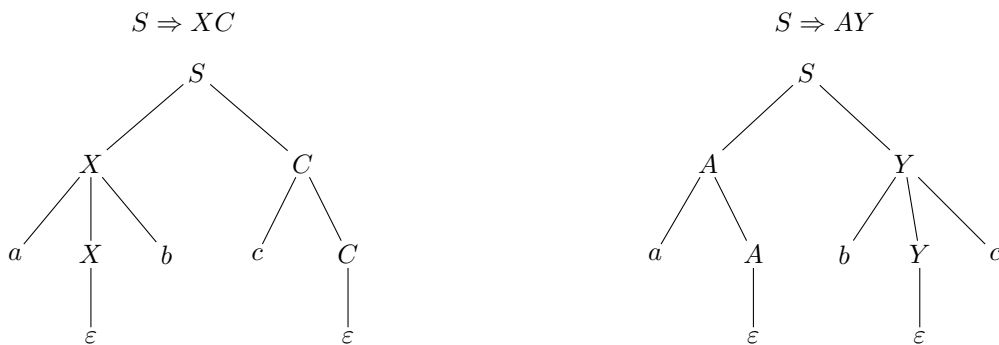
故每个形如 $a^n b^n c^n$ 的字符串都可以从开始变量 S 的两条不同分支导出。以 abc 为例，有两种不同的最左推导：

$$S \Rightarrow XC \Rightarrow aXbC \Rightarrow abC \Rightarrow abcC \Rightarrow abc,$$

以及

$$S \Rightarrow AY \Rightarrow aAY \Rightarrow aY \Rightarrow abYc \Rightarrow abc.$$

因此该文法是歧义文法。

图 38: 字符串 abc 对应的两棵不同语法树

解释. 证明一个文法有歧义, 只需找出一个字符串 $w \in L(G)$, 使它有俩棵不同的语法树, 或等价地, 有两种不同的最左推导 (也可用最右推导)。常见做法是先观察文法的不同分支是否会生成重叠的子语言; 若文法形如并集拼装

$$S \rightarrow S_1 \mid S_2,$$

则优先去找

$$L(S_1) \cap L(S_2)$$

中的字符串, 因为这样的字符串往往天然会对应两种推导。需要注意的是, 证明“这个文法有歧义”只说明当前这套产生式有两种分析方式, 并不自动推出该语言本身是固有歧义的; 后者要求证明任何文法都会歧义, 难度更高。

2.14

2.14 设文法 $G = (V, \Sigma, R, S)$, 其中 $V = \{S, T, U\}$, $\Sigma = \{0, \#\}$, R 是下述规则的集合:

$$S \rightarrow TT \mid U$$

$$T \rightarrow 0T \mid T0 \mid \#$$

$$U \rightarrow 0U00 \mid \#$$

- 用普通的语言描述 $L(G)$ 。
- 证明 $L(G)$ 不是正则的。

图 39

解答

(a) 先分别描述变量 T, U 所生成的语言。

由

$$T \rightarrow 0T \mid T0 \mid \#$$

可知, T 最终先变成一个符号 $\#$, 随后每用一次产生式 $0T$, 就在其左边添一个 0; 每用一次产生式 $T0$, 就在其右边添一个 0。因此

$$L(T) = \{0^i \# 0^j \mid i, j \geq 0\}.$$

又由

$$U \rightarrow 0U00 \mid \#$$

可知, U 从 $\#$ 出发, 每一步都在左边添一个 0, 同时在右边添两个 0, 故

$$L(U) = \{0^n \# 0^{2n} \mid n \geq 0\}.$$

于是

$$L(TT) = \{0^i \# 0^j \mid i, j \geq 0\} \{0^k \# 0^\ell \mid k, \ell \geq 0\} = \{0^i \# 0^m \# 0^j \mid i, m, j \geq 0\},$$

因为连接

$$0^i \# 0^j \cdot 0^k \# 0^\ell = 0^i \# 0^{j+k} \# 0^\ell.$$

故该文法生成的语言为

$$L(G) = \{0^i \# 0^m \# 0^j \mid i, m, j \geq 0\} \cup \{0^n \# 0^{2n} \mid n \geq 0\}.$$

用普通语言描述, 即:

$L(G)$ 由两类字符串组成: 一类是恰含两个符号 $\#$ 、其余位置全为 0 的字符串; 另一类是恰含一个符号 $\#$, 且其右边 0 的个数恰为左边 0 个数两倍的字符串。

(b) 证明 $L(G)$ 不是正则语言。

取正则语言

$$R = 0^* \# 0^*.$$

它恰好由所有只含一个符号 $\#$ 的字符串组成。于是

$$L(G) \cap R = \{0^n \# 0^{2n} \mid n \geq 0\}.$$

下证

$$K = \{0^n \# 0^{2n} \mid n \geq 0\}$$

不是正则语言。若 K 正则, 则设其泵长度为 p 。取

$$s = 0^p \# 0^{2p} \in K.$$

对任意分解

$$s = xyz, \quad |xy| \leq p, \quad |y| > 0,$$

由于前 p 个符号全是 0, 故必有

$$y = 0^t \quad (t > 0).$$

令 $i = 0$, 则

$$xz = 0^{p-t} \# 0^{2p}.$$

但此时左边 0 的个数为 $p - t$, 右边 0 的个数为 $2p$, 显然

$$2p \neq 2(p - t),$$

故

$$xz \notin K,$$

与 pumping lemma 矛盾。因此 K 不是正则语言。

若 $L(G)$ 是正则语言, 则因正则语言对交运算封闭, $L(G) \cap R$ 也应正则; 但这与上面的结论矛盾。故

$L(G)$ 不是正则语言。

解释. 这题的关键是把文法分成两部分看。变量 T 只是在一个固定的 $\#$ 左右自由添 0, 所以它生成的是

$$0^* \# 0^*.$$

而变量 U 每次递归都“左边加 1 个 0、右边加 2 个 0”, 因此它精确记录了比例关系 1:2, 生成

$$\{0^n \# 0^{2n} \mid n \geq 0\}.$$

证明非正则时, 不必直接处理整个并集; 只要与正则语言 $0^* \# 0^*$ 相交, 就能把前一部分滤掉, 只剩下后一部分, 再对这一部分应用 pumping lemma 即可。

2.16

2.16 给出一个反例证明下述构造不能证明上下文无关语言类在星运算下封闭。设 CFL A 由 CFG $G = (V, \Sigma, R, S)$ 产生, 将 G 增加一条新规则 $S \rightarrow SS$ 并称其为 G' , 这个语法可产生语言 A^* 。

图 40

解答

取

$$A = \{a\}.$$

它显然是上下文无关语言。可取文法

$$G = (\{S\}, \{a\}, \{S \rightarrow a\}, S),$$

满足

$$L(G) = A.$$

按照题目中的错误构造, 在 G 上增加一条规则

$$S \rightarrow SS,$$

得到新文法

$$G' = (\{S\}, \{a\}, \{S \rightarrow a \mid SS\}, S).$$

这个文法生成的语言是

$$L(G') = a^+.$$

因为每次使用 $S \rightarrow SS$ 都会把一个 a -块再分成两个非空部分, 所以可以生成任意正长度的 a 串; 但由于文法中没有推出 ε 的规则, 故

$$\varepsilon \notin L(G').$$

另一方面,

$$A^* = \{a\}^* = a^*,$$

其中必然包含 ε 。因此

$$L(G') = a^+ \neq a^* = A^*.$$

所以，仅仅给原文法增加一条规则 $S \rightarrow SS$ ，并不能保证得到 A^* 的文法，因而这个构造不能用来证明上下文无关语言类在星号运算下封闭。■

解释. 这个构造失败的根本原因是：星号允许取零次连接，因此结果语言必须包含 ε ；而在原文法不生成 ε 时，单加一条 $S \rightarrow SS$ 只能实现“重复一次以上”，做不到“零次重复”。正确构造通常要新建一个开始变量，并显式加入

$$S_0 \rightarrow \varepsilon$$

这一分支。

2.18

- ^ 2.18** a. 设 C 是上下文无关语言， R 是正则语言。证明语言 $C \cap R$ 是上下文无关的。
b. 利用 a 证明语言

$A = \{w \mid w \in \{a, b, c\}^*, \text{且含有数目相同的 } a, b \text{ 和 } c\}$
不是上下文无关的。

图 41

证明

- (a) 设 C 是上下文无关语言， R 是正则语言。取识别 C 的 PDA

$$P = (Q_P, \Sigma, \Gamma, \delta_P, q_P, F_P)$$

及识别 R 的 DFA

$$M = (Q_M, \Sigma, \delta_M, q_M, F_M).$$

构造新的 PDA

$$P' = (Q_P \times Q_M, \Sigma, \Gamma, \delta', (q_P, q_M), F_P \times F_M).$$

它的思想是：读入每个符号时，有限控制同时模拟 P 与 M ；栈操作完全照搬 P ，而 DFA 的状态只记录在控制状态中。

按照课本中 PDA 的记法，

$$\delta_P : Q_P \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q_P \times \Gamma_\varepsilon).$$

因此对任意 $p \in Q_P$ 、 $r \in Q_M$ 、 $x \in \Gamma_\varepsilon$ ，定义 δ' 如下。

若 $a \in \Sigma$ ，则

$$\delta'((p, r), a, x) = \{((p', \delta_M(r, a)), u) \mid (p', u) \in \delta_P(p, a, x)\}.$$

若进行 ε -转移，则

$$\delta'((p, r), \varepsilon, x) = \{((p', r), u) \mid (p', u) \in \delta_P(p, \varepsilon, x)\}.$$

也就是说：若

$$(p', u) \in \delta_P(p, a, x),$$

则在 P' 中有

$$((p', \delta_M(r, a)), u) \in \delta'((p, r), a, x),$$

而若

$$(p', u) \in \delta_P(p, \varepsilon, x),$$

则

$$((p', r), u) \in \delta'((p, r), \varepsilon, x).$$

于是 P' 在栈上与 P 做完全相同的操作，同时在状态第二分量中记录读入前缀在 DFA M 中所到达的状态。故对任意 $w \in \Sigma^*$,

$$w \in L(P') \iff w \in C \text{ 且 } w \in R.$$

即

$$L(P') = C \cap R.$$

因此

$$C \cap R$$

是上下文无关语言。■

(b) 设

$$A = \{w \in \{a, b, c\}^* \mid \#_a(w) = \#_b(w) = \#_c(w)\}.$$

取正则语言

$$R = a^*b^*c^*.$$

则

$$A \cap R = \{a^n b^n c^n \mid n \geq 0\}.$$

这是因为在 R 中，所有 a 都在前，所有 b 居中，所有 c 都在后；再加上三种字母数目相同，就只能得到形如 $a^n b^n c^n$ 的字符串。

已知

$$\{a^n b^n c^n \mid n \geq 0\}$$

不是上下文无关语言。若 A 是上下文无关语言，则由 (a) 可知 $A \cap R$ 也应是上下文无关语言，这就得到矛盾。故

$$A = \{w \in \{a, b, c\}^* \mid \#_a(w) = \#_b(w) = \#_c(w)\} \text{ 不是上下文无关语言。}$$

解释. 这题的核心是：正则语言所需的“记忆”是有限的，因此可以直接并入 PDA 的有限控制中；栈仍只负责上下文无关那部分信息。于是“CFL 条件”与“regular 条件”可以同时检查。第 (b) 问则是反过来用这个封闭性：若一个复杂语言真是 CFL，那么与合适的正则语言相交后也应仍是 CFL；只要把它切出一个已知非 CFL 的子语言，就得到反证。

2.19

*2.19 设有 CFG G :

$$S \rightarrow aSb \mid bY \mid Ya$$

$$Y \rightarrow bY \mid aY \mid \epsilon$$

用通常的语言给出 $L(G)$ 的简单描述，并利用这个描述，给出 $L(G)$ 的补集 $\overline{L(G)}$ 的 CFG。

解答

先注意

$$Y \rightarrow aY \mid bY \mid \varepsilon$$

生成的正是

$$L(Y) = \{a, b\}^*.$$

因此

$$S \rightarrow bY$$

生成所有以 b 开头的字符串，而

$$S \rightarrow Ya$$

生成所有以 a 结尾的字符串。规则

$$S \rightarrow aSb$$

则是在某个已生成字符串外侧再包上一层首字母 a 和末字母 b 。

由此可见， $L(G)$ 的简单描述应为

$$L(G) = \{a, b\}^* \setminus \{a^n b^n \mid n \geq 0\}.$$

用通常的语言描述，即：

$L(G)$ 由所有不属于 $\{a^n b^n \mid n \geq 0\}$ 的 $\{a, b\}$ -串组成；也就是说，它恰好包含所有不是“先若干个 a 、再若干个 b ，且两段长度相等”这种形式的字符串。

这里并不是“猜”补集，而是先分析补集中字符串被迫具有的结构。设

$$C = \{a, b\}^* \setminus L(G).$$

若 $w \in C$ ，则：

(i) w 不能以 b 开头；否则由 $S \rightarrow bY$ 可得 $w \in L(G)$ 。

(ii) w 不能以 a 结尾；否则由 $S \rightarrow Ya$ 可得 $w \in L(G)$ 。

因此 w 只能是 ε ，或可写成

$$w = axb$$

的形式。再者，若 $w = axb \in C$ ，则必有

$$x \in C,$$

因为一旦 $x \in L(G)$ ，就可由规则

$$S \rightarrow aSb$$

推出 $axb \in L(G)$ ，矛盾。于是 C 满足递归描述

$$C = \{\varepsilon\} \cup aCb.$$

这已经强制说明： C 中的字符串只能是

$$\varepsilon, ab, aabb, aaabbb, \dots,$$

即

$$C \subseteq \{a^n b^n \mid n \geq 0\}.$$

后面的两步正式证明再进一步说明反向包含也成立，从而得到

$$C = \{a^n b^n \mid n \geq 0\}.$$

下面证明这一点。

先证 $L(G) \subseteq \{a, b\}^* \setminus \{a^n b^n \mid n \geq 0\}$. 用结构归纳即可。若最外层使用

$$S \rightarrow bY,$$

则所得串以 b 开头，不可能属于 $\{a^n b^n \mid n \geq 0\}$ ；若最外层使用

$$S \rightarrow Ya,$$

则所得串以 a 结尾，也不可能属于该集合。若最外层使用

$$S \rightarrow aSb,$$

且中间部分 $x \notin \{a^n b^n \mid n \geq 0\}$ ，则

$$axb \notin \{a^n b^n \mid n \geq 0\},$$

因为若 $axb = a^n b^n$ ，则必有

$$x = a^{n-1} b^{n-1},$$

矛盾。故一切由该文法生成的字符串都不属于 $\{a^n b^n \mid n \geq 0\}$ 。

再证反包含。 对字符串长度作归纳。设

$$w \in \{a, b\}^* \setminus \{a^n b^n \mid n \geq 0\}.$$

若 w 以 b 开头，则由 $S \rightarrow bY$ 可得 $w \in L(G)$ ；若 w 以 a 结尾，则由 $S \rightarrow Ya$ 可得 $w \in L(G)$ 。

剩下的情形是： w 以 a 开头并以 b 结尾。记

$$w = axb.$$

此时 x 仍不属于 $\{a^n b^n \mid n \geq 0\}$ ；否则若

$$x = a^n b^n,$$

则

$$w = a^{n+1} b^{n+1},$$

与假设矛盾。又因为 $|x| < |w|$ ，由归纳假设得

$$x \in L(G).$$

于是利用产生式

$$S \rightarrow aSb$$

便知

$$w = axb \in L(G).$$

综上,

$$L(G) = \{a, b\}^* \setminus \{a^n b^n \mid n \geq 0\}.$$

因此其补集正是

$$\overline{L(G)} = \{a^n b^n \mid n \geq 0\}.$$

一个识别该补集的 CFG 可以取为

$$S_0 \rightarrow aS_0b \mid \varepsilon.$$

解释. 这个文法的两条非递归分支已经覆盖了“以 b 开头”与“以 a 结尾”的所有串; 递归分支 $S \rightarrow aSb$ 只是在外面再加一层配对的 a, b . 因此, 它恰好漏掉的就是那些既不以 b 开头、也不以 a 结尾、并且不断去掉首尾配对后最终只剩空串的字符串, 也就是

$$a^n b^n.$$

一旦看出这一点, 补集文法就是标准的

$$a^n b^n$$

文法。

2.22

***2.22** 设 $C = \{x \# y \mid x, y \in \{0, 1\}^*, \text{ 且 } x \neq y\}$, 证明 C 是上下文无关语言。

图 43

证明

由定理“一个语言是上下文无关语言, 当且仅当某个 PDA 识别它”, 只需构造一个识别

$$C = \{x \# y \mid x, y \in \{0, 1\}^*, x \neq y\}$$

的非确定下推自动机即可。

下面给出一个非正式的 PDA 描述。机器在开始时非确定地分成两种情况。

情况 1: 检验 $|x| \neq |y|$. 在读入左半段 x 时, 对每个 0 或 1 都向栈中压入一个记号 X 。读到分隔符 $\#$ 后, 开始读入右半段 y , 每读一个符号就弹出一个 X 。

(i) 若在 y 尚未读完时栈已空, 则说明 $|y| > |x|$, 接受。

(ii) 若输入已经读完而栈中仍有 X , 则说明 $|x| > |y|$, 接受。

(iii) 只有当输入恰好读完且栈也恰好清空时, 才说明 $|x| = |y|$, 此分支拒绝。

情况 2: 检验 $|x| = |y|$ 且某个对应位置符号不同. 这一分支并不是“猜第一个失配位置”, 而只是非确定地猜一个失配见证: 若 $x \neq y$, 总存在某个对应位置符号不同; 机器只要有一条分支猜中这个位置即可。为了便于用栈对齐, 我们选择“从右边数的某个失配位置”作为见证。

在读入 x 时, 机器在每个位置都有两种转移选择: 要么继续向右扫描而什么也不记; 要么把当前位置选作待比较位置, 并把该符号是 0 还是 1 记在有限控制中。从这以后, 每再读入 x 的一个符号, 就向栈中压入一个记号 X 。因此, 当读完整个 x 时, 栈中记下的正是“该位置右边还剩多少个符号”。

读到 # 后, 机器在 y 中也先通过非确定分支任意跳过一个前缀; 随后在某个位置停下, 把当前符号作为与前面所记位置对应的比较对象, 并要求这个符号与有限控制中记下的符号不同。此后机器继续读完 y 的剩余部分, 并对每个剩余符号弹出一个 X 。只有当输入恰好读完且栈也恰好清空时才接受。

这一条件意味着: 在 x 中被选中的位置与在 y 中被选中的位置, 右边所剩符号个数相同, 也就是说它们是从右边数起的对应位置; 而这两个位置上的符号又不同, 所以

$$x \neq y.$$

正确性.

- (i) 若 $x \neq y$ 且 $|x| \neq |y|$, 则情况 1 接受。
- (ii) 若 $x \neq y$ 且 $|x| = |y|$, 则必存在某个对应位置上的符号不同。取其中任意一个, 尤其可取从右边数的某个失配位置。情况 2 总有一条分支能在 x 和 y 中恰好选中这一对位置, 于是接受。
- (iii) 反之, 若某分支接受, 则要么情况 1 证明了长度不同, 要么情况 2 找到了一个从右边数对应、但符号不同的位置, 因此一定有 $x \neq y$ 。

故该 PDA 恰好识别语言 C , 从而 C 是上下文无关语言。■

解释. 你指出的那个问题本质上在于: CFG 很难直接保证“第 i 个位置”对应“第 i 个位置”。而 PDA 用栈天然适合从右往左对齐, 所以更自然的做法不是去找“第一次失配”, 而是去找“从右边数的某个失配见证”。这里所谓“猜”并不是额外魔法, 而正是非确定自动机的标准工作方式: 当机器读到某个位置时, 它可以沿着多条合法转移同时分支, 例如“一条分支把当前位置记为候选失配位置, 另一条分支继续向后扫描”。若输入串确实满足 $x \neq y$, 那么总存在至少一个真正的失配位置, 于是至少有一条分支会恰好选中这个位置并最终验证成功; 若输入串不满足条件, 则所有分支最终都会失败。

2.24

2.24 设 $E = \{a^i b^j \mid i \neq j \text{ 且 } 2i \neq j\}$, 证明 E 是上下文无关语言。

图 44

证明

将

$$E = \{a^i b^j \mid i \neq j, 2i \neq j\}$$

按 j 与 $i, 2i$ 的大小关系拆成三部分:

$$E_1 = \{a^i b^j \mid j < i\},$$

$$E_2 = \{a^i b^j \mid i < j < 2i\},$$

$$E_3 = \{a^i b^j \mid j > 2i\}.$$

显然

$$E = E_1 \cup E_2 \cup E_3.$$

下面分别为这三个语言构造 CFG。

情形 $j < i$. 取文法

$$S_1 \rightarrow aS_1b \mid A, \quad A \rightarrow aA \mid a.$$

前一条规则每次同时增加一个 a 和一个 b , 后一条规则额外产生至少一个未配对的 a , 故

$$L(S_1) = E_1.$$

情形 $j > 2i$. 取文法

$$S_3 \rightarrow aS_3bb \mid B, \quad B \rightarrow bB \mid b.$$

前一条规则每次增加一个 a 和两个 b , 后一条规则额外产生至少一个未配对的 b , 故

$$L(S_3) = E_3.$$

情形 $i < j < 2i$. 取文法

$$S_2 \rightarrow aS_2b \mid aTbb,$$

$$T \rightarrow aTb \mid aTbb \mid ab.$$

说明其生成的正是 E_2 。

若某次推导中, 规则

$$S_2 \rightarrow aS_2b$$

共用了 m 次, 然后转入

$$S_2 \rightarrow aTbb,$$

而在变量 T 中, 规则

$$T \rightarrow aTb$$

用了 n 次, 规则

$$T \rightarrow aTbb$$

用了 t 次, 最后以

$$T \rightarrow ab$$

结束, 则最终得到的字符串为

$$a^i b^j,$$

其中

$$i = m + n + t + 2, \quad j = m + n + 2t + 3.$$

于是

$$j - i = t + 1 > 0,$$

且

$$2i - j = m + n + 1 > 0.$$

所以

$$i < j < 2i.$$

这说明

$$L(S_2) \subseteq E_2.$$

反过来, 任取

$$a^i b^j \in E_2, \quad i < j < 2i.$$

令

$$t = j - i - 1 \geq 0, \quad m = 2i - j - 1 \geq 0.$$

则

$$i = m + t + 2, \quad j = m + 2t + 3.$$

于是让 $S_2 \rightarrow aS_2b$ 使用 m 次, 随后使用一次 $S_2 \rightarrow aTbb$; 再在 T 中使用 $T \rightarrow aTbb$ 共 t 次, 最后用 $T \rightarrow ab$ 收尾, 便得到恰好

$$a^i b^j.$$

故

$$E_2 \subseteq L(S_2).$$

于是

$$L(S_2) = E_2.$$

最后令

$$S \rightarrow S_1 \mid S_2 \mid S_3.$$

则

$$L(S) = E_1 \cup E_2 \cup E_3 = E.$$

因此 E 是上下文无关语言。■

解释. 这题本质上是把平面上所有点 (i, j) 去掉两条直线

$$j = i, \quad j = 2i$$

以后, 剩下的区域分成三块: 下方、中间、上方。每一块都可以用一个简单 CFG 描述, 再用并集拼起来。中间那块

$$i < j < 2i$$

稍微麻烦一点, 但只要把每个 a 对应产生 1 个或 2 个 b , 并且强制这两种情况都至少出现一次, 就正好落在两条直线之间。

2.26

2.26 设 G 是一个乔姆斯基范式 CFG, 证明: 对于任一长度为 $n(n \geq 1)$ 的字符串 $w \in L(G)$, 通过 CFG G 将其派生出来恰好需要 $2n-1$ 步。

图 45

证明。设某个派生

$$S \Rightarrow^* w$$

生成了长度为 $n \geq 1$ 的字符串 w 。记其中使用形如

$$A \rightarrow BC$$

的产生式共 m 次, 使用形如

$$A \rightarrow a$$

的产生式共 ℓ 次。

因为 G 是乔姆斯基范式, 且 $w \neq \varepsilon$, 所以派生中只能使用这两类产生式。
先求 ℓ 。每次使用 $A \rightarrow a$, 恰好产生一个终结符; 最终得到的字符串长度为 n , 故

$$\ell = n.$$

再求 m 。在派生过程中, 开始时只有 1 个非终结符 S 。每次使用

$$A \rightarrow BC$$

会使当前非终结符个数增加 1; 每次使用

$$A \rightarrow a$$

会使当前非终结符个数减少 1。最终得到终结符串时, 非终结符个数为 0, 因此

$$1 + m - \ell = 0.$$

代入 $\ell = n$, 得

$$m = n - 1.$$

故总派生步数为

$$m + \ell = (n - 1) + n = 2n - 1.$$

因此, 对任意长度为 $n \geq 1$ 的 $w \in L(G)$, 由 G 派生出 w 恰好需要

$$2n - 1$$

步。■

解释. 这题的关键是: CNF 中只有两种规则。一种把一个非终结符变成两个, 因此“未展开的非终结符数”加 1; 另一种把一个非终结符变成一个终结符, 因此这个数减 1。终结符一共要产生 n 个, 所以第二类规则用了 n 次; 从 1 个非终结符降到 0 个, 就反推出第一类规则必用 $n - 1$ 次。

2.32

2.32 对于字母表 $\Sigma = \{1, 2, 3, 4\}$ 上的语言 $C = \{w \in \Sigma^* \mid \text{在 } w \text{ 中, } 1 \text{ 与 } 2 \text{ 的个数相同, } 3 \text{ 与 } 4 \text{ 的个数相同}\}$, 证明 C 不是上下文无关语言。

图 46

证明。反设

$$C = \{w \in \Sigma^* \mid \#_1(w) = \#_2(w), \#_3(w) = \#_4(w)\}$$

是上下文无关语言。由上下文无关语言的泵引理, 存在常数 p , 使得任意满足 $|s| \geq p$ 的 $s \in C$ 都可写成

$$s = uvxyz,$$

其中

$$|vxy| \leq p, \quad |vy| > 0,$$

并且对所有 $i \geq 0$,

$$uv^i xy^i z \in C.$$

取

$$s = 1^p 3^p 2^p 4^p.$$

显然 $s \in C$, 因为

$$\#_1(s) = \#_2(s) = p, \quad \#_3(s) = \#_4(s) = p.$$

由于每一段

$$1^p, 3^p, 2^p, 4^p$$

的长度都恰为 p , 而 $|vxy| \leq p$, 故子串 vxy 至多落在一段中, 或至多跨越两段相邻的块。于是只有以下几种情形。

1. vxy 完全落在某一块中。

若 vxy 落在

$$1^p, 3^p, 2^p, 4^p$$

中的某一块里, 则抽引 $i = 0$ 或 $i = 2$ 只会改变这一类符号的个数, 而其对应需要配平的另一类符号个数不变, 因此必有

$$\#_1 \neq \#_2 \quad \text{或} \quad \#_3 \neq \#_4.$$

故所得字符串不在 C 中。

2. vxy 跨越两段相邻的块。

若跨越

$$1^p \text{ 与 } 3^p,$$

则抽引只会改变 1 和 3 的个数, 而 2, 4 的个数保持为 p 。由于 $|vy| > 0$, 至少有一种符号个数发生变化, 于是必有

$$\#_1 \neq \#_2 \quad \text{或} \quad \#_3 \neq \#_4.$$

跨越

$$3^p \text{ 与 } 2^p$$

或跨越

$$2^p \text{ 与 } 4^p$$

时同理。抽引后, 至多只改变两种相邻符号的个数, 而至少有一个配平条件被破坏, 因此所得字符串也不在 C 中。

综上, 对任意满足泵引理条件的分解 $s = uvxyz$, 总可取 $i = 0$ 或 $i = 2$ 使得

$$uv^i xy^i z \notin C,$$

与泵引理矛盾。

因此 C 不是上下文无关语言。■

解释. 这题和

$$\{a^n b^n c^n \mid n \geq 0\}$$

一样, 核心是同时维持两组独立的计数约束。选取

$$1^p 3^p 2^p 4^p$$

以后, 泵引理中的短窗口最多只会碰到相邻两块, 因此一旦抽引, 就不可能同时保持

$$\#_1 = \#_2, \quad \#_3 = \#_4.$$

2.33

* 2.33 证明语言 $F = \{a^i b^j \mid \text{存在正整数 } k, \text{ 有 } i \neq kj\}$ 不是上下文无关的。

图 47

说明. 按题意, 这里应理解为证明

$$F = \{a^i b^j \mid i = kj \text{ for some positive integer } k\}$$

不是上下文无关语言。若写成 $i \neq kj$, 则该语言几乎就是整个 $a^* b^*$, 命题不成立。

证明。反设 F 是上下文无关语言。由泵引理, 存在泵长度 p , 使得任意满足 $|s| \geq p$ 的 $s \in F$ 都可写成

$$s = uvxyz,$$

其中

$$|vxy| \leq p, \quad |vy| > 0,$$

并且对所有 $i \geq 0$,

$$uv^i xy^i z \in F.$$

取素数 q , 满足

$$q > p^2 + p.$$

令

$$s = a^{q^2} b^q.$$

因为

$$q^2 = q \cdot q,$$

故 $s \in F$ 。

对任意满足条件的分解 $s = uvxyz$, 记

$$\alpha = \#_a(vy), \quad \beta = \#_b(vy).$$

则

$$0 \leq \alpha, \beta \leq p, \quad \alpha + \beta > 0.$$

下面分类讨论。

1. $\beta = 0$ 。

此时 vy 只含字母 a 。取 $i = 0$, 得

$$uv^0 xy^0 z = a^{q^2 - \alpha} b^q.$$

因为

$$0 < \alpha \leq p < q,$$

而 $q \mid q^2$, 故

$$q \nmid (q^2 - \alpha).$$

因此新的 a 的个数不是 q 的整数倍, 从而

$$uv^0xy^0z \notin F.$$

2. $\alpha = 0$ 。

此时 vy 只含字母 b 。取 $i = 2$, 得

$$uv^2xy^2z = a^{q^2}b^{q+\beta}.$$

由于 q 是素数, q^2 的正因子只有

$$1, q, q^2.$$

而

$$q < q + \beta < 2q < q^2,$$

故

$$q + \beta \nmid q^2.$$

因此

$$uv^2xy^2z \notin F.$$

3. $\alpha > 0$ 且 $\beta > 0$ 。

取 $i = 2$, 得

$$uv^2xy^2z = a^{q^2+\alpha}b^{q+\beta}.$$

若它仍在 F 中, 则必须有

$$q + \beta \mid q^2 + \alpha.$$

但模 $q + \beta$ 计算,

$$q \equiv -\beta \pmod{q + \beta},$$

故

$$q^2 + \alpha \equiv \beta^2 + \alpha \pmod{q + \beta}.$$

又因为

$$0 < \beta^2 + \alpha \leq p^2 + p < q < q + \beta,$$

所以

$$\beta^2 + \alpha$$

是一个严格介于 0 与 $q + \beta$ 之间的余数, 不可能为 0。因此

$$q + \beta \nmid q^2 + \alpha,$$

从而

$$uv^2xy^2z \notin F.$$

三种情形都得到矛盾, 因此 F 不是上下文无关语言。■

解释. 这题的关键不是“比例关系难描述”，而是要找到一个对抽引非常敏感的样本。取

$$a^{q^2} b^q$$

以后，原来的比例恰好是整数倍；但只要把前面的 a 或后面的 b 稍微改动一点，这种整除关系就会立刻失效。选素数 q 的作用，就是让“谁能整除 q^2 ”这件事非常刚性。

2.44

2.44 给定语言 A, B ，有 $A \diamond B = \{xy \mid x \in A, y \in B, \text{且 } |x| = |y|\}$ ，证明：如果 A, B 是正则语言，则 $A \diamond B$ 上下文无关。

图 48

证明。设 A, B 是正则语言。取识别它们的 DFA

$$M_A = (Q_A, \Sigma, \delta_A, q_A, F_A),$$

$$M_B = (Q_B, \Sigma, \delta_B, q_B, F_B).$$

我们构造一台 NPDA 来识别

$$A \diamond B = \{xy \mid x \in A, y \in B, |x| = |y|\}.$$

这台 NPDA 的思路如下。

1. 在读入前半段 x 时，它在有限控制中模拟 M_A ，并且每读入一个符号就向栈中压入一个记号 X 。
2. 在某个时刻，它通过一次 ϵ -转移非确定地猜测：“这里就是分界点，接下来开始读 y 。”
3. 在读入后半段 y 时，它在有限控制中模拟 M_B ，并且每读入一个符号就从栈中弹出一个 X 。
4. 当输入恰好读完、栈恰好回到底符号，并且两个 DFA 都处于接受态时，这台 NPDA 接受。

形式化地，令栈字母表为

$$\Gamma = \{X, \$\},$$

其中 $\$$ 是栈底符号。状态集取为

$$Q = Q_A \times Q_B \times \{1, 2\} \cup \{q_{\text{acc}}\}.$$

其中第三个分量表示当前处于第一阶段还是第二阶段。开始状态为

$$(q_A, q_B, 1),$$

初始栈为 $\$$ 。

转移规则如下。

第一阶段：若当前状态为 $(p, r, 1)$ ，读入 $a \in \Sigma$ ，则转到

$$(\delta_A(p, a), r, 1),$$

并向栈中压入一个 X 。

阶段切换：对任意 $(p, r, 1)$ ，允许一条 ε -转移到

$$(p, r, 2),$$

栈不变。

第二阶段：若当前状态为 $(p, r, 2)$ ，读入 $a \in \Sigma$ ，且栈顶为 X ，则转到

$$(p, \delta_B(r, a), 2),$$

并弹出这个 X 。

接受：若输入已读完，当前状态为 $(p, r, 2)$ ，其中

$$p \in F_A, \quad r \in F_B,$$

且栈顶为 $\$$ ，则通过一条 ε -转移进入接受态 q_{acc} 。

证明这台 NPDA 识别的正是 $A \diamond B$ 。

若某串 w 被该 NPDA 接受，则它在某一条接受分支上被分成两段

$$w = xy.$$

第一阶段恰好读完 x ，故有限控制中的 M_A 模拟表明

$$x \in A.$$

第二阶段恰好读完 y ，故 M_B 的模拟表明

$$y \in B.$$

又因为第一阶段每读一个符号压入一个 X ，第二阶段每读一个符号弹出一个 X ，并且接受时栈恰好回到底符号，所以

$$|x| = |y|.$$

于是

$$w \in A \diamond B.$$

反过来，若

$$w = xy \in A \diamond B,$$

其中

$$x \in A, \quad y \in B, \quad |x| = |y|,$$

则这台 NPDA 可以在恰好读完 x 时猜中分界点，随后在第二阶段读入 y 。由于

$$x \in A, \quad y \in B,$$

最后两台 DFA 都处于接受态；又因为

$$|x| = |y|,$$

栈中压入与弹出的 X 个数恰好相同，因此最终栈回到底符号，从而接受 w 。

故

$$L(P) = A \diamond B.$$

因此 $A \diamond B$ 是上下文无关语言。■

解释. 这题的本质是把“属于 A ”“属于 B ”和“前后两段长度相等”三件事同时检查。前两件事因为 A, B 正则，只需在有限控制中模拟两个 DFA；第三件事则由栈完成，用“前半段压栈、后半段弹栈”的方式记录长度差。非确定性只用来猜分界点。

2.45

2.45 设 $A = \{wtw^R \mid w, t \in \{0, 1\}^* \text{ 且 } |w| = |t|\}$ ，证明 A 不是上下文无关语言。

图 49

证明。设

$$A = \{wtw^R \mid w, t \in \{0, 1\}^*, |w| = |t|\}.$$

反设 A 是上下文无关语言。

考虑正则语言

$$R = 0^*10^*10^*.$$

由于上下文无关语言对与正则语言求交封闭，故

$$L = A \cap R$$

也应当是上下文无关语言。

下面先刻画 L 。

若

$$s \in A \cap R,$$

则

$$s = wtw^R, \quad |w| = |t|,$$

并且 s 恰好含有两个 1，其形式为

$$0^*10^*10^*.$$

因此 t 中不能含有 1，否则整个串中至少有三个 1；同理， w 中也只能含有一个 1，否则 w^R 中也会贡献至少两个 1，整个串仍会超过两个 1。于是必有

$$w = 0^n1$$

对某个 $n \geq 0$ 成立。因为

$$|w| = n + 1 = |t|,$$

而 t 只能由 0 组成，所以

$$t = 0^{n+1}.$$

从而

$$s = 0^n10^{n+1}10^n.$$

反过来，对任意 $n \geq 0$,

$$0^n10^{n+1}10^n = (0^n1)0^{n+1}(0^n1)^R$$

且

$$|0^n 1| = n + 1 = |0^{n+1}|,$$

故它属于 A ；又显然属于 R 。于是

$$L = \{0^n 1 0^{n+1} 1 0^n \mid n \geq 0\}.$$

现在用上下文无关语言的泵引理证明 L 不是上下文无关语言。设 p 是泵长度，取

$$s = 0^p 1 0^{p+1} 1 0^p \in L.$$

任取分解

$$s = uvxyz,$$

满足

$$|vxy| \leq p, \quad |vy| > 0.$$

由于两个 1 之间相隔

$$p + 1$$

个 0，故长度不超过 p 的子串 vxy 至多包含一个 1。

1. 若 v 或 y 中含有 1，则抽引后字符串中 1 的个数会改变。取 $i = 0$ 或 $i = 2$ ，得到的新串中 1 的个数不是 2，因此不可能仍属于

$$\{0^n 1 0^{n+1} 1 0^n \mid n \geq 0\}.$$

2. 若 v 和 y 都不含 1，则它们只由 0 组成。

若它们完全落在同一个 0 块中，则抽引只会改变三段 0 中的一段长度，而语言 L 要求首尾两段长度相等且中间一段恰好多 1，故取 $i = 0$ 即离开 L 。

若它们分布在第一个 1 的两侧，则抽引后得到

$$0^{p-\alpha} 1 0^{p+1-\beta} 1 0^p$$

其中 $\alpha + \beta > 0$ 。但属于 L 的串必须满足首尾两段 0 长度相等，于是必须有

$$p - \alpha = p,$$

从而 $\alpha = 0$ ；同时中间一段还必须恰好多 1，于是又需

$$p + 1 - \beta = p + 1,$$

从而 $\beta = 0$ 。这与

$$\alpha + \beta > 0$$

矛盾。因此该串不在 L 中。

若它们分布在第二个 1 的两侧，同理，抽引后得到

$$0^p 1 0^{p+1-\alpha} 1 0^{p-\beta},$$

也不可能仍满足 L 的长度关系。

综上，对任意满足泵引理条件的分解，总可取 $i = 0$ 或 $i = 2$ 使得

$$uv^i xy^i z \notin L,$$

与泵引理矛盾。因此 L 不是上下文无关语言。

但若 A 是上下文无关语言，则 $L = A \cap R$ 也应当是上下文无关语言，矛盾。故

A

不是上下文无关语言。■

解释。 原语言里“前一段”和“后一段的反转”之间没有分隔符，直接泵会比较别扭。和正则语言

$$0^*10^*10^*$$

相交以后，结构被压成

$$0^n10^{n+1}10^n,$$

这时需要维持的关系就非常清楚了：首尾两段必须一样长，中间那段必须恰好多 1。任何局部抽引都会破坏这个关系。

总结：如何证明语言是或不是上下文无关

本章中，判断一个语言是否为上下文无关语言，通常分成两类问题：

1. 证明一个语言是上下文无关的。
2. 证明一个语言不是上下文无关的。

这两类题的证明方法和写作结构并不相同，最好严格区分。

一、证明语言是上下文无关的 最常见的方法有三种。

1. 直接构造一个 CFG。

也就是写出文法 G ，再证明

$$L(G) = A.$$

规范写法通常分成两步：

$$L(G) \subseteq A, \quad A \subseteq L(G).$$

第一步往往用结构归纳，分析任意由文法生成的串一定具有题目要求的形式；第二步则对目标语言中的任意串，按照其参数或长度归纳，说明它确实可以由该文法推导出来。

2. 直接构造一台 PDA 或 NPDA。

若题目的约束更像“边读边检查”，例如需要比较两部分长度、记录某个中间信息，往往 PDA 比 CFG 更自然。此时做法是构造自动机 P ，再证明

$$L(P) = A.$$

证明同样最好写成两个方向：接受的串一定满足条件；满足条件的串一定存在一条接受计算路径。

3. 利用闭包性质。

若一个语言可以拆成若干个已经知道是上下文无关的语言，再通过并、连接、星号，或与正则语言相交等运算得到，那么可以先引用闭包性，再把问题化归为已知语言的构造。

因此，证明一个语言是上下文无关时，最标准的套路是：
先决定用 CFG 还是 PDA 更自然；构造对象以后，不要只写“显然成立”，而应明确证明

$$L(\text{构造出的对象}) = A.$$

若是 CFG，就证明“文法生成的串恰是目标语言”；若是 PDA，就证明“自动机接受的串恰是目标语言”。

二、证明语言不是上下文无关的 常见方法主要有两种。

1. 用上下文无关语言的泵引理。

其标准结构是：

先反设 A 是上下文无关语言。设 p 为泵长度。选取一个特殊的

$$s \in A, \quad |s| \geq p.$$

然后对任意分解

$$s = uvxyz,$$

满足

$$|vxy| \leq p, \quad |vy| > 0,$$

证明总能找到某个 $i \geq 0$ ，使得

$$uv^i xy^i z \notin A.$$

于是与泵引理矛盾。

这里最关键的是量词顺序：

$$\exists s \forall (u, v, x, y, z) \exists i.$$

也就是说，必须证明对任意合法分解都能把它抽坏，而不是只挑一种分解。

2. 先用闭包性质把语言化简，再对化简后的语言使用泵引理。

这在本章里非常常见。原语言本身可能结构太复杂，不容易直接抽引；这时可以把它与一个正则语言相交，把无关部分滤掉，得到一个更标准的语言，再证明后者不是上下文无关。由于上下文无关语言对与正则语言求交封闭，若化简后的语言不是上下文无关，则原语言也不可能是上下文无关。

三、写证明时的常见误区 1. 不能用泵引理证明“一个语言是上下文无关的”。

泵引理是上下文无关语言的必要条件，不是充分条件。满足泵引理，并不能推出语言是上下文无关的；因此它只能用来证明“不是上下文无关”，不能用来证明“是上下文无关”。

2. 用泵引理时，不能只分析一种分解。

若只说明“某一种分解不能抽引”，这远远不够。必须覆盖所有满足

$$|vxy| \leq p, \quad |vy| > 0$$

的分解。

3. 构造 CFG 以后，不能只写出产生式而不证明正确性。

正确的写法应当说明：文法不会生成多余的串，而且目标语言中的每个串都能生成。

四、一个规范的证明模板 若要证明 A 是上下文无关语言，可以按下面格式写：

“构造 CFG（或 PDA） G （或 P ）。下面证明

$$L(G) = A \quad (\text{或 } L(P) = A).$$

先证

$$L(G) \subseteq A,$$

再证

$$A \subseteq L(G).$$

因此 A 是上下文无关语言。”

若要证明 A 不是上下文无关语言，可以按下面格式写：

“反设 A 是上下文无关语言。设 p 为泵长度。取

$$s \in A, \quad |s| \geq p.$$

任取分解

$$s = uvxyz, \quad |vxy| \leq p, |vy| > 0.$$

分类讨论 vxy 所在的位置，证明总可取某个 i 使得

$$uv^i xy^i z \notin A.$$

与泵引理矛盾，故 A 不是上下文无关语言。”

如果直接抽引太难，就先把语言与适当的正则语言相交，再对交出来的语言应用同一模板。

解释. 简言之：要证明“是 CFL”，就去构造；要证明“不是 CFL”，就去找不可能由单栈有限控制完成的全局约束，再用泵引理或闭包性把这种不可能性写成严格矛盾。前者的关键词是“构造并证明等于”，后者的关键词是“反设、选串、分类、抽坏”。

Chapter 3

3.6

3.6 定理 3.13 证明了一个语言是图灵可识别的当且仅当有枚举器枚举它，为什么不能用下列更简单的算法作为证明？像以前一样， $s_1, s_2, \dots$ 是 Σ^* 中的所有串。

$E =$ “忽略输入，

1) 对于 $i=1, 2, 3, \dots$ ，重复下列步骤。

图 50

2) 在 s_i 上运行 M 。

3) 如果接受，则打印出 s_i 。”

图 51

证明。不能用题目中给出的更简单算法，是因为 M 只是图灵可识别器，不一定对每个输入都停机。若按题目中的方法枚举，到了某个字符串 s_i 时，步骤 2 会在 s_i 上一直运行 M 。如果

M 在 s_i 上循环而不停机，

则枚举器就会永远卡在这一步，再也不会继续处理

$s_{i+1}, s_{i+2}, \dots$

这样一来，即使后面某个字符串 s_j 满足

$s_j \in L(M)$,

它也永远不会被打印出来。因此这个算法不能保证枚举出语言中的所有字符串。

这正是定理 3.13 正向证明里必须采用交错模拟 (dovetailing) 的原因：在第 t 轮同时把前 t 个输入都只模拟有限步，从而不会因为某一个输入上的无限循环而阻塞整个枚举过程。■

解释。 枚举器最怕“某个分支把全局卡死”。对图灵可识别器来说，某些输入上可能永远不停机，所以不能把全部时间都压在单个 s_i 上；必须把时间公平地分给所有候选输入。

3.8

3.8 下面的语言都是字母表 $\{0, 1\}$ 上的语言，以实现水平的描述给出判定这些语言的图灵机：

- A a. $\{w \mid w \text{ 包含相同个数的 } 0 \text{ 和 } 1\}$ 。
- b. $\{w \mid w \text{ 所包含的 } 0 \text{ 的个数是 } 1 \text{ 的个数的两倍}\}$ 。
- c. $\{w \mid w \text{ 所包含的 } 0 \text{ 的个数不是 } 1 \text{ 的个数的两倍}\}$ 。

图 52

设带字母表中另外加入标记符号 X ，表示“这个位置已经配对过”。

(a) 构造判定

$$A = \{w \mid w \text{ 中 } 0 \text{ 和 } 1 \text{ 的个数相同}\}$$

的图灵机 M_A 如下。

输入 w 后：

1. 从带子左端开始向右扫描，找第一个尚未标记的符号。
2. 如果找不到这样的符号，说明所有 0,1 都已成对删去，接受。
3. 若找到的是 0，就把它改写成 X ，然后继续向右寻找第一个尚未标记的 1。若找不到，则拒绝；若找到，则把该 1 也改写成 X 。
4. 若找到的是 1，则对称地处理：把它改写成 X ，再向右寻找第一个尚未标记的 0；若找不到则拒绝，若找到则也改写成 X 。
5. 回到带子左端，重复上述过程。

每一轮恰好删去一个 0 和一个 1，因此机器必停机，并且当且仅当二者个数相等时接受。

(b) 构造判定

$$B = \{w \mid w \text{ 中 } 0 \text{ 的个数是 } 1 \text{ 的个数的两倍}\}$$

的图灵机 M_B 如下。

1. 从左端开始寻找第一个尚未标记的 1。若不存在这样的 1，转第 4 步。
2. 把该 1 改写成 X ，然后从当前位置向右寻找两个尚未标记的 0。每找到一个就改写成 X 。
3. 若在找满两个 0 之前已无未标记的 0，则拒绝；否则回到左端继续第 1 步。
4. 此时所有 1 都已处理完。再从左到右扫描一遍：若还存在未标记的 0，则拒绝；否则接受。

每轮删去一个 1 和两个 0，所以机器必停机，并且当且仅当

$$\#0 = 2\#1$$

时接受。

(c) 设

$$C = \{w \mid w \text{ 中 } 0 \text{ 的个数不是 } 1 \text{ 的个数的两倍}\}.$$

因为 (b) 中的 M_B 是 B 的判定器，所以只需构造机器 M_C ：

在输入 w 上运行 M_B ；若 M_B 接受，则拒绝；若 M_B 拒绝，则接受。

于是 M_C 判定 C 。■

解释. 这三小题的核心都是“反复做有限次配对并标记”。(a) 是每次配走一个 0 和一个 1；(b) 是每次配走两个 0 和一个 1；(c) 直接利用 (b) 的补语言即可，因为 (b) 已经是判定器。

3.10

3.10 双无限带图灵机(Turing machine with doubly infinite tape)与普通图灵机相似,所不同的是它的带子向左和向右都是无限的。此带在开始时候,除了包括输入区域外,其他都填以空白符,计算也像通常一样定义,只是在它向左移动时不会遇到带的端点。证明这种类型的图灵机能够识别图灵可识别语言类。

图 53

证明。设双无限带图灵机所识别的语言类为 $\mathcal{C}$, 普通图灵机所识别的语言类为 $\mathcal{R}$ 。

先证

$$\mathcal{R} \subseteq \mathcal{C}.$$

这是显然的: 普通图灵机只是双无限带图灵机的一种特殊情形, 只要从不使用输入左侧那些额外的格子即可。

再证

$$\mathcal{C} \subseteq \mathcal{R}.$$

设 D 是一台双无限带图灵机。构造一台普通图灵机 M 来模拟它。

把 D 的带内容编码成

$$u\#v,$$

其中:

1. v 记录当前读写头所在格及其右边各格的内容, 当前格放在 v 的最左端;
2. u 记录当前读写头左边各格的内容, 但按“离头越近越靠左”的逆序存放。

也就是说, 若 D 的带在当前格局下形如

$$\cdots a_{-2}a_{-1}\underline{a_0}a_1a_2\cdots,$$

则 M 在自己的带上存放

$$a_{-1}a_{-2}\cdots\#a_0a_1a_2\cdots$$

并把 D 的当前状态记在有限控制中。

现在说明如何模拟 D 的一步转移。

1. 若 D 在当前格写入符号 b , 则 M 只需把 v 的第一个符号改写为 b 。
2. 若 D 向右移动, 则 M 把 v 的第一个符号删去并接到 u 的最左端。若删去后 v 为空, 则补上一个空白符, 表示新当前格原本为空白。
3. 若 D 向左移动, 则 M 把 u 的第一个符号取出并放到 v 的最左端; 若 u 为空, 则取出的符号视为空白符。

因此, M 用有限次普通图灵机操作就能模拟 D 的一步。于是 M 能逐步模拟 D 的整个计算, 并且仅在 D 接受时接受。

故

$$L(M) = L(D).$$

因此

$$\mathcal{C} = \mathcal{R},$$

即双无限带图灵机识别的正是图灵可识别语言类。■

解释. 双无限带并没有增加本质能力, 因为“头左边的内容”和“头右边的内容”都可以用一条普通带上的有限编码保存。真正重要的不是带子朝哪边无限, 而是机器是否能进行任意长的有限控制和重写。

3.12

3.12 左复位图灵机(Turing machine with left reset)和普通图灵机类似, 只是它的转移函数具有下列形式:

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, \text{RESET}\}$$

如果 $\delta(q, a) = (r, b, \text{RESET})$, 则当机器处于状态 q 且读 a 时, 在带上写下 b 并进入状态 r 后, 读写头就跳到带的左端点。注意, 这样的机器没有将它的读写头向左移动一个符号的普通能力。证明左复位图灵机识别图灵可识别语言类。

图 54

证明. 设左复位图灵机识别的语言类为 $\mathcal{C}$, 普通图灵机识别的语言类为 $\mathcal{R}$ 。

先证

$$\mathcal{C} \subseteq \mathcal{R}.$$

给定一台左复位图灵机, 只需用普通图灵机来模拟其转移即可。对右移转移照常模拟; 若遇到

RESET,

就不断向左移动, 直到到达带的左端点。这样普通图灵机即可逐步模拟左复位图灵机。

再证

$$\mathcal{R} \subseteq \mathcal{C}.$$

设 M 是一台普通图灵机。构造左复位图灵机 P 来模拟它。

P 在自己的带上保存 M 当前格局的编码

$$\#uqa v\#,$$

其中 $u, v \in \Gamma^*$, q 是 M 的当前状态, a 是当前读写头所读到的符号。也就是说, u 是头左边的带内容, a 是当前格, v 是头右边的带内容。

现在说明如何模拟 M 的一步。

1. P 先执行一次 RESET, 从左到右扫描当前编码, 找到唯一的局部片段

$$qa.$$

于是就知道

$$\delta_M(q, a) = (r, b, D).$$

2. 接着 P 再次 RESET, 并通过若干次“从左到右扫描旧编码, 再把新编码写到右侧空白区域”的过程, 构造下一格局的编码。

若 $D = R$, 则把

$$\#uqacv\#$$

改成

$$\#ubrcv\#.$$

若当前格右边没有已写出的符号，则把 c 视为空白符。

若 $D = L$ ，则把

$$\#u'cqav\#$$

改成

$$\#u'rcbv\#.$$

若当前格左边没有已写出的符号，则把 c 视为空白符。

3. 新编码完成后， P 忽略旧编码，继续在新编码上模拟下一步。

因为每次模拟一步只需要有限次

RESET

和有限次从左到右的扫描，所以 P 能逐步模拟 M 的全部计算。因此

$$L(P) = L(M).$$

故

$$\mathcal{C} = \mathcal{R},$$

即左复位图灵机识别的也是图灵可识别语言类。■

解释. 左复位看起来比“向左移动一格”弱很多，但它仍然足以模拟普通图灵机。原因是：虽然它不能原地左移，但它可以反复回到左端，再把整个当前格局重新扫描并重写成“下一格局”。代价只是时间变慢，而不是能力变弱。

3.15

3.15 证明可判定语言类在下列运算下封闭：

- ^A a. 并
- b. 连接
- c. 星号
- d. 补
- e. 交

图 55

设 A, B 都是可判定语言，分别由判定器 M_A, M_B 判定。

(a) 并 构造判定器 M ：

输入 w 后，先运行 M_A 于 w 上。若 M_A 接受，则接受；否则再运行 M_B 于 w 上。若 M_B 接受，则接受；否则拒绝。

因为 M_A, M_B 都停机，所以 M 停机，并且判定

$$A \cup B.$$

(b) 连接 构造判定器 M :

输入 w 后, 枚举所有分解

$$w = xy$$

的方式。这样的分解只有 $|w| + 1$ 种。对每一种分解, 分别运行 M_A 于 x 上、 M_B 于 y 上; 若存在某种分解使得二者都接受, 则接受; 若所有分解都失败, 则拒绝。

因为分解数有限且每次调用都停机, 所以 M 判定

$$AB.$$

(c) 星号 构造判定器 M :

输入 w 后, 枚举把 w 分割成有限多个子串

$$w = w_1 w_2 \cdots w_k$$

的所有可能方式。若 $w = \varepsilon$, 则直接接受。对每一种分割, 逐个运行 M_A 于各段 w_i 上; 若存在一种分割使得每一段都在 A 中, 则接受; 若所有分割都失败, 则拒绝。

因为 w 的分割方式只有有限多种, 所以该机器停机, 并判定

$$A^*.$$

(d) 补 构造判定器 M :

输入 w 后运行 M_A 。若 M_A 接受, 则拒绝; 若 M_A 拒绝, 则接受。

于是 M 判定

$$\overline{A}.$$

(e) 交 构造判定器 M :

输入 w 后, 先运行 M_A , 再运行 M_B 。当且仅当二者都接受时接受, 否则拒绝。

因为 M_A, M_B 都停机, 所以 M 判定

$$A \cap B.$$

因此可判定语言类在并、连接、星号、补、交这五种运算下都封闭。■

解释. 证明“可判定”最关键的一点是: 所有子过程都必须停机。并、交、补只是在已有判定器上做有限次组合; 连接和星号虽然要“猜切分”, 但输入长度固定时, 切分方式只有有限种, 所以也仍然能穷尽。

3.18

***3.18 证明:** 一个语言是可判定的, 当且仅当有枚举器以字典顺序枚举这个语言。

图 56

证明。

先证“若 A 可判定, 则存在按字典顺序枚举 A 的枚举器”。

设 M 是 A 的判定器。构造枚举器 E :

忽略输入，按固定的字典顺序依次产生

$$s_1, s_2, s_3, \dots$$

并对每个 s_i 运行 M 。若 M 接受，则打印 s_i ；若 M 拒绝，则不打印，继续处理下一个字符串。

因为 M 对每个输入都停机，所以 E 会依次检查所有字符串，并且恰好按字典顺序打印 A 的所有成员。

再证“若存在按字典顺序枚举 A 的枚举器，则 A 可判定”。

设 E 按字典顺序枚举 A ；若 A 有限，则约定 E 在输出最后一个字符串后停机。构造判定器 D ：

输入字符串 w 后，模拟 E 的运行，并观察其打印输出。若 E 打印出 w ，则接受；若 E 打印出某个严格大于 w 的字符串，则拒绝；若 E 在打印完所有不大于 w 的字符串后停机，也拒绝。

这个过程一定停机。因为在字典顺序中，不超过 w 的字符串只有有限多个；而 E 的输出又是递增的，所以最终只可能发生以下三种情况之一：

1. 打印出 w ，此时 $w \in A$ ；
2. 先打印出一个严格大于 w 的字符串，此时 $w \notin A$ ；
3. 输出完所有不大于 w 的成员后停机，此时 $w \notin A$ 。

因此 D 判定 A 。

综上，一个语言是可判定的，当且仅当存在枚举器按字典顺序枚举它。■

解释. 普通枚举器只能说明“可识别”，因为你不知道某个串只是“还没轮到”，还是“永远不会出现”。一旦输出顺序被限制成字典序，这种不确定性就消失了：只要已经越过了 w ，就能立刻断定 $w \notin A$ 。

3.20

* 3.20 证明：不能在带子的输入区域写的单带图灵机只能识别正则语言。

图 57

证明。设 M 是一台单带图灵机，并且它从不在初始输入所在的那一段区域内写符号。记它识别的语言为 $L(M)$ 。下面证明 $L(M)$ 必为正则语言。

对每个非空串 $w \in \Sigma^+$ ，记 B_w 为初始输入所在的那段连续带格。由于 M 从不在 B_w 上写符号，所以每次读头进入 B_w 时，它在这段只读区域内的行为只取决于：

1. 当前状态；
2. 输入串 w 本身。

设

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}}).$$

定义

$$D = \{\text{init}\} \cup Q, \quad R = Q \cup \{\text{acc}, \text{rej}, \text{loop}\}.$$

对每个 $w \in \Sigma^+$ ，定义它的摘要

$$F_w : D \rightarrow R$$

如下：

- $F_w(\text{init})$ 表示 M 在输入 w 上开始运行后, 第一次离开 B_w 的右端时所处的状态; 若它在离开前已接受、拒绝, 或永远困在 B_w 内, 则分别记为 $\text{acc}, \text{rej}, \text{loop}$ 。
- 对每个 $q \in Q$, $F_w(q)$ 表示: 若机器从“读头位于 w 的最右端符号、当前状态为 q ”开始, 只观察它在 B_w 内的这段运行, 则第一次离开 B_w 的右端时所处的状态; 若在离开前已接受、拒绝, 或永远困在 B_w 内, 则分别记为 $\text{acc}, \text{rej}, \text{loop}$ 。

由于 D, R 都是有限集, 可能出现的摘要 F_w 只有有限多个。

对每个 $a \in \Sigma$, 存在一个确定的更新函数

$$U_a$$

作用在这些摘要上, 使得

$$U_a(F_w) = F_{wa}.$$

理由是: 在串 wa 上, 机器在最后一个符号 a 上的第一步动作是固定的; 若它向左进入前缀 w , 则随后在 w 内的全部行为完全由摘要 F_w 决定; 若它又回到符号 a , 状态也由 F_w 决定, 于是后续过程继续被同一个有限摘要控制。因此 F_{wa} 由 F_w 和 a 唯一决定。

现在构造 DFA

$$A = (Q_A, \Sigma, \delta_A, s, F_A),$$

其中

$$Q_A = \{s\} \cup \{F_w \mid w \in \Sigma^+\},$$

其中 s 是新初态, 转移定义为

$$\delta_A(s, a) = F_a, \quad \delta_A(F_w, a) = U_a(F_w),$$

接受态定义为

$$F_A = \{F_w \mid F_w(\text{init}) = \text{acc}\} \cup (\{s\} \text{ 若且唯若 } \varepsilon \in L(M)).$$

对任意非空串 w , 由定义立刻归纳得

$$\delta_A(s, w) = F_w.$$

因此

$$w \in L(A) \iff F_w(\text{init}) = \text{acc} \iff M \text{ 在输入 } w \text{ 上接受}.$$

再加上空串的情形也已经由初态 s 处理, 所以

$$L(A) = L(M).$$

由于 A 是 DFA, $L(M)$ 为正则语言。■

解释. 核心就是把每个输入串 w 看成一个有限摘要 F_w : 它记录“机器在这段只读输入上能发生什么”。由于摘要只有有限多个, 并且读入一个新符号 a 时摘要按确定规则更新, 于是这些摘要本身就组成了一个 DFA 的状态集, 因此原机器识别的语言必为正则语言。

Chapter 4

4.3

4.3 设 $ALL_{DFA} = \{\langle A \rangle \mid A \text{ 是一个识别 } \Sigma^* \text{ 的 DFA}\}$ 。证明 ALL_{DFA} 是可判定的。

图 58

证明。构造判定器 M 来判定

$$ALL_{DFA} = \{\langle A \rangle \mid A \text{ 是 DFA 且 } L(A) = \Sigma^*\}.$$

M = “对于输入 $\langle A \rangle$ ，其中

$$A = (Q, \Sigma, \delta, q_0, F),$$

做如下工作：

1. 从开始状态 q_0 出发，在状态图中做一次可达性搜索，求出所有从 q_0 可达的状态。
2. 若在这些可达状态中发现某个状态

$$q \notin F,$$

则拒绝；否则接受。”

证明该算法正确。若 M 接受，则每个从 q_0 可达的状态都是接受态。对任意字符串 w ，DFA A 读入 w 后必停在某个从 q_0 可达的状态上，因此该状态接受，从而

$$w \in L(A).$$

故

$$L(A) = \Sigma^*.$$

反之，若 $L(A) = \Sigma^*$ ，则任何从 q_0 可达的状态都必须是接受态；否则若某个可达状态 q 不是接受态，取一条从 q_0 到 q 的路径所对应的字符串 w ，则 A 在输入 w 上拒绝，与

$$L(A) = \Sigma^*$$

矛盾。

因此 M 当且仅当

$$\langle A \rangle \in ALL_{DFA}$$

时接受。由于状态图有限，搜索必定停机，所以 ALL_{DFA} 是可判定的。■

解释. 对 DFA 而言，“接受所有串”只需看一件事：从开始状态真正能到达的状态里，有没有非接受态。若有，就有某个串把机器带到那里；若没有，每个输入都会落在接受态。

4.4

4.4 设 $A_{\epsilon\text{CFG}} = \{\langle G \rangle \mid G \text{ 是一个派生 } \epsilon \text{ 的 CFG}\}$ 。证明 $A_{\epsilon\text{CFG}}$ 是可判定的。

图 59

证明。构造判定器 M 来判定

$$A_{\epsilon\text{CFG}} = \{\langle G \rangle \mid G \text{ 是 CFG 且 } \epsilon \in L(G)\}.$$

$M =$ “对于输入 $\langle G \rangle$ ，其中

$$G = (V, \Sigma, R, S),$$

做如下工作：

1. 令集合 $N \subseteq V$ 初始为空。
2. 重复执行下述操作，直到 N 不再变化：若某条产生式

$$A \rightarrow X_1 X_2 \cdots X_k$$

满足右边为空串，或者每个 X_i 都已在 N 中，则把 A 加入 N 。

3. 若

$$S \in N,$$

则接受；否则拒绝。”

下面证明正确性。先证：若某个变量 A 被标记，则

$$A \Rightarrow^* \epsilon.$$

这是按标记顺序直接归纳得到的：若由规则

$$A \rightarrow \epsilon$$

被标记，则显然可推出 ϵ ；若由

$$A \rightarrow X_1 \cdots X_k$$

被标记，且各 X_i 都已能推出 ϵ ，则

$$A \Rightarrow X_1 \cdots X_k \Rightarrow^* \epsilon.$$

再证反向：若

$$A \Rightarrow^* \epsilon,$$

则最终 A 会被标记。对这棵推出 ϵ 的语法树做自底向上的归纳即可：叶子对应 ϵ -产生式，因此其父变量会被标记；逐层向上，整个树根 A 也会被标记。

因此

$$S \in N \iff S \Rightarrow^* \epsilon \iff \epsilon \in L(G).$$

故 $A_{\epsilon\text{CFG}}$ 是可判定的。■

解释. 这题的本质是求“可空变量”集合。因为一个 CFG 生成 ϵ ，当且仅当开始变量是可空的；而可空性是一个有限的闭包计算问题。

4.6

4.6 设 B 是 $\{0, 1\}$ 上所有无限序列的集合。用对角化方法证明 B 是不可数的。

图 60

证明。设

$$B = \{(b_1, b_2, b_3, \dots) \mid b_i \in \{0, 1\} \text{ 对每个 } i \geq 1\}.$$

我们用反证法和对角化证明 B 不可数。

假设 B 是可数的，则可以把 B 中所有无限 0-1 序列排成一个表：

$$s_1, s_2, s_3, \dots$$

其中

$$s_i = (a_{i1}, a_{i2}, a_{i3}, \dots), \quad a_{ij} \in \{0, 1\}.$$

也就是说，这张表的第 i 行是第 i 个序列，第 j 列是该序列的第 j 项。

现在构造一个新序列

$$c = (c_1, c_2, c_3, \dots),$$

其中对每个 $i \geq 1$ ，定义

$$c_i = \begin{cases} 1, & a_{ii} = 0, \\ 0, & a_{ii} = 1. \end{cases}$$

换言之， c_i 总是取成与第 i 个序列的第 i 项相反的值。

于是对任意 $i \geq 1$ ，序列 c 与 s_i 在第 i 个位置不同，因为

$$c_i \neq a_{ii}.$$

所以 c 不可能等于 s_i 。这说明 c 不在我们列出的序列

$$s_1, s_2, s_3, \dots$$

之中。

但另一方面， c 的每一项也是 0 或 1，因此

$$c \in B.$$

这与“上面的表已经列出了 B 的所有元素”矛盾。

故假设错误， B 不是可数集，也就是说 B 是不可数的。■

解释。 对角化的关键是：如果你声称已经把所有无限二进制序列都列出来了，我就沿着第 1 行第 1 列、第 2 行第 2 列、第 3 行第 3 列这样走下去，并把这些位置上的值逐个取反，造出一个新序列。这个新序列会和第 i 个序列至少在第 i 位不同，所以它不可能出现在你的列表里。

4.10

4.10 设 $INFINITE_{PDA} = \{\langle M \rangle \mid M \text{ 是一个 PDA, 且 } L(M) \text{ 是一个无限语言}\}$ 。证明 $INFINITE_{PDA}$ 是可判定的。

图 61

证明。构造判定器 M 来判定

$$INFINITE_{PDA} = \{\langle P \rangle \mid P \text{ 是 PDA 且 } L(P) \text{ 是无限语言}\}.$$

$M =$ “对于输入 $\langle P \rangle$:

1. 把 PDA P 有效地转换成一个等价的 CFG G 。
2. 删除 G 中所有无用变量, 并把所得文法转换成 Chomsky 范式, 记结果为

$$G' = (V, \Sigma, R, S).$$

3. 构造有向图 H : 顶点集为 V ; 若某条产生式

$$A \rightarrow BC$$

属于 R , 则在 H 中加入边

$$A \rightarrow B \text{ 和 } A \rightarrow C.$$

4. 检查从开始变量 S 可达的变量中, 是否有某个变量位于 H 的一个有向环上。若有, 则接受; 否则拒绝。”

下面证明正确性。若 M 接受, 则存在某个从 S 可达且位于有向环上的变量 A 。由于 G' 已删除无用变量, A 必能推出某个终结字符串; 又由于 A 在依赖图中处于环上, 存在推导把 A 再次带回自身。因为 G' 处于 Chomsky 范式, 没有单位产生式, 所以绕环一次必然在 A 的两侧添入某些最终能推出终结字符串的部分。于是可得到

$$A \Rightarrow^* uAv$$

其中 $u, v \in \Sigma^*$, 且至少有一个非空。反复迭代这段推导, 便得到无穷多个不同长度的字符串, 因此

$$L(G') = L(P)$$

是无限的。

反之, 若 M 拒绝, 则从 S 可达的变量部分在图 H 中无环。于是任何语法树的一条根到叶路径上都不会重复出现变量, 所以语法树高度至多为

$$|V|.$$

而在 Chomsky 范式下, 高度至多为 $|V|$ 的语法树, 其叶子数至多为

$$2^{|V|},$$

因此它生成的终结字符串长度也至多为 $2^{|V|}$ 。可见

$$L(G')$$

只包含有限多个字符串，从而

$$L(P)$$

也是有限的。

因此 M 当且仅当

$$\langle P \rangle \in \text{INFINITE}_{\text{PDA}}$$

时接受，所以 $\text{INFINITE}_{\text{PDA}}$ 是可判定的。■

解释. PDA 是否接受无限多个串，可以先转成 CFG 来看。对 CFG 而言，真正决定“能不能无限生成”的，是变量依赖图里是否存在一个从开始变量可达的环：有环就能反复绕圈、不断增长；无环则语法树高度有统一上界，语言只能是有限的。

4.12

4.12 设 $A = \{ \langle R, S \rangle \mid R \text{ 和 } S \text{ 是正则表达式, 且 } L(R) \subseteq L(S) \}$ ，证明 A 是可判定的。

图 62

证明。构造判定器 M 来判定

$$A = \{ \langle R, S \rangle \mid R \text{ 和 } S \text{ 是正则表达式, 且 } L(R) \subseteq L(S) \}.$$

M = “对于输入 $\langle R, S \rangle$ ：

1. 把正则表达式 R 和 S 分别转换成等价的 DFA，记为 A_R 和 A_S 。
2. 把 A_S 补全后取补，得到一个 DFA B ，满足

$$L(B) = \overline{L(S)}.$$

3. 构造 DFA C 来识别

$$L(A_R) \cap L(B) = L(R) \cap \overline{L(S)}.$$

4. 运行 DFA 空性判定过程，检查

$$L(C) = \emptyset$$

是否成立。若成立，则接受；否则拒绝。”

该机器正确，因为

$$L(R) \subseteq L(S) \iff L(R) \cap \overline{L(S)} = \emptyset.$$

而正则表达式到 DFA 的转换、DFA 的补、正则语言的交以及 DFA 空性检测都是有效且可停机的过程。因此 M 判定语言 A ，从而 A 是可判定的。■

解释. “ $L(R) \subseteq L(S)$ ” 最标准的改写就是：不存在某个串被 R 接受但不被 S 接受。于是问题变成检查

$$L(R) \cap \overline{L(S)}$$

是否为空，而这完全落在正则语言的可判定范围里。

4.15

4.15 设 $A = \{\langle R \rangle \mid R \text{ 是一个正则表达式, 其所描述的语言中至少有一个串 } w \text{ 以 } 111 \text{ 为子串 (即有 } x \text{ 和 } y \text{ 使得 } w = x111y)\}$ 。证明 A 是可判定的。

图 63

证明. 构造判定器 M 来判定

$$A = \{\langle R \rangle \mid R \text{ 是正则表达式, 且 } L(R) \text{ 中至少有一个串含有子串 } 111\}.$$

$M =$ “对于输入 $\langle R \rangle$:

1. 把正则表达式 R 转换成等价的 DFA A_R 。
2. 构造一个固定的 DFA B , 使其识别语言

$$T = \{w \in \{0,1\}^* \mid w \text{ 含有子串 } 111\}.$$

3. 构造 DFA C 来识别

$$L(A_R) \cap T.$$

4. 运行 DFA 空性判定过程, 检查

$$L(C) = \emptyset$$

是否成立。若为空, 则拒绝; 否则接受。”

该机器正确, 因为

$$\langle R \rangle \in A \iff L(R) \cap T \neq \emptyset.$$

而

$$T = (0 \cup 1)^* 111 (0 \cup 1)^*$$

是正则语言, 所以第 2 步可有效完成; 其余各步也都是正则语言上的标准可判定操作。因此 A 是可判定的。■

解释. 题目问的是“正则表达式描述的语言里, 有没有某种特殊形状的串”。这类问题的标准套路就是: 先把“特殊形状”本身写成一个固定正则语言, 再与输入语言求交, 最后检查交集是否为空。

4.17

4.17 设 C 是一个语言。证明 C 是图灵可识别的, 当且仅当存在一个可判定语言 D , 使得 $C = \{x \mid \exists y (\langle x, y \rangle \in D)\}$ 。

图 64

证明。我们证明如下双向命题：

$$C \text{ 是图灵可识别的} \iff \exists D \text{ 可判定, 使得 } C = \{x \mid \exists y (\langle x, y \rangle \in D)\}.$$

正向。 设 C 是图灵可识别的, 令 N 为 C 的一个识别器。定义

$$D = \{\langle x, c \rangle \mid c \text{ 是 } N \text{ 在输入 } x \text{ 上的一个接受计算历史}\}.$$

我们先说明 D 是可判定的：给定输入 $\langle x, c \rangle$, 只需检查

1. c 的第一项是否是 N 在输入 x 上的初始格局；
2. c 的最后一项是否为接受格局；
3. c 中每一对相邻格局是否满足 N 的转移规则。

这些检查都是有限且机械可做的, 因此 D 可判定。

接着证明

$$x \in C \iff \exists c (\langle x, c \rangle \in D).$$

若 $x \in C$, 则识别器 N 在输入 x 上存在某条接受计算, 于是这条接受计算的历史 c 满足

$$\langle x, c \rangle \in D.$$

反之, 若存在某个 c 使得

$$\langle x, c \rangle \in D,$$

那么 c 正是 N 在输入 x 上的一条接受计算历史, 因此 N 接受 x , 即

$$x \in C.$$

反向。 现在设存在某个可判定语言 D , 使得

$$C = \{x \mid \exists y (\langle x, y \rangle \in D)\}.$$

令 T 为 D 的判定器。构造图灵机 N 来识别 C :

N = “对于输入 x :

1. 按字典顺序枚举所有字符串

$$y_1, y_2, y_3, \dots$$

2. 依次运行判定器 T 于输入

$$\langle x, y_i \rangle$$

上。

3. 若某次运行接受, 则接受。”

因为 T 是判定器, 所以第 2 步的每次调用都停机。若

$$x \in C,$$

则存在某个见证字符串 y 使

$$\langle x, y \rangle \in D,$$

于是机器 N 最终会枚举到这个 y 并接受。若

$$x \notin C,$$

则不存在任何这样的 y ，于是 N 永远不会接受。这正说明 N 识别 C ，故 C 是图灵可识别的。■

解释. 这一定理说明：图灵可识别语言恰好是“某个可判定二元关系对第二个参数做存在量词投影”得到的语言。正向里，见证 y 可以取成“接受计算历史”；反向里，只要把所有可能的见证逐个试过去，就得到了一个识别器。

4.23

4.23 设 $BAL_{DFA} = \{ \langle M \rangle \mid M \text{ 是一个 DFA, 它接受某些有相同多个 0 和 1 的串} \}$ ，证明 BAL_{DFA} 是可判定的。（提示：一些关于 CFL 的定理对此会有帮助。）

图 65

证明。令

$$C_{=} = \{ w \in \{0,1\}^* \mid \#_0(w) = \#_1(w) \}.$$

该语言是上下文无关的，例如由文法

$$S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$$

生成。

构造判定器 D ：

$D =$ “对于输入 $\langle M \rangle$ ：

1. 利用正则语言与上下文无关语言求交的有效构造，构造一个 CFG G_M ，满足

$$L(G_M) = L(M) \cap C_{=}.$$

2. 运行上下文无关语言空性问题的判定器，判断

$$L(G_M) = \emptyset$$

是否成立。

3. 若

$$L(G_M) = \emptyset,$$

则拒绝；否则接受。”

若 D 接受输入 $\langle M \rangle$ ，则存在某个被 M 接受且满足

$$\#_0 = \#_1$$

的字符串；若 D 拒绝输入 $\langle M \rangle$ ，则不存在这样的字符串。于是

$$\text{BAL}_{\text{DFA}} = \{\langle M \rangle \mid M \text{ 是 DFA 且接受某个 } 0, 1 \text{ 个数相同的串}\}$$

是可判定的。■

解释. 这题的思路是把“0 和 1 个数相同”先写成一个 CFL，再和输入 DFA 的语言求交。这样原问题就化成了一个上下文无关语言的空性判定。

4.27

4.27 设 $C_{\text{CFG}} = \{\langle G, k \rangle \mid L(G) \text{ 包含 } k \text{ 个字符串, 其中 } k \geq 0 \text{ 或 } k = \infty\}$ ，证明 C_{CFG} 是可判定的。

图 66

证明。构造判定器 M 来判定

$$C_{\text{CFG}} = \{\langle G, k \rangle \mid L(G) \text{ 恰好包含 } k \text{ 个字符串, 其中 } k \geq 0 \text{ 或 } k = \infty\}.$$

$M =$ “对于输入 $\langle G, k \rangle$ ：

1. 检查 k 的编码是否表示一个非负整数，或特殊符号 ∞ 。若不是，则拒绝。
2. 把 CFG G 转换成等价文法 G' ，删除其中的无用变量，并把结果化为 Chomsky 范式。记

$$G' = (V, \Sigma, R, S).$$

3. 构造变量依赖图 H ：顶点集为 V ；若某条产生式

$$A \rightarrow BC$$

属于 R ，则加入边

$$A \rightarrow B \text{ 和 } A \rightarrow C.$$

4. 检查从 S 可达的变量中是否存在落在有向环上的变量。
5. 若存在这样的变量，则 $L(G)$ 是无限的。此时若 $k = \infty$ ，则接受；否则拒绝。
6. 若不存在这样的变量，则 $L(G)$ 是有限的。令

$$N = 2^{|V|}.$$

枚举字母表 Σ 上所有长度不超过 N 的字符串。

7. 对每个被枚举出的字符串 w ，运行 CFG 成员性判定过程，检查

$$w \in L(G)$$

是否成立，并统计满足条件的字符串个数，记为 t 。

8. 若 k 是一个非负整数且

$$t = k,$$

则接受；否则拒绝。”

下面说明正确性。第 4 步与 4.10 中同理：若从开始变量可达的部分存在环，则可以反复绕环，从而推出无穷多个不同字符串，所以

$$|L(G)| = \infty.$$

反之，若无环，则任何语法树的一条根到叶路径上都不会重复出现变量，因此语法树高度至多为

$$|V|.$$

Chomsky 范式下高度至多为 $|V|$ 的语法树，其叶子数至多为

$$2^{|V|},$$

故所生成字符串的长度也至多为 $2^{|V|}$ 。于是只要把所有长度不超过

$$N = 2^{|V|}$$

的字符串全部枚举并做成员性测试，就能穷尽

$$L(G)$$

中的所有字符串，因而第 7 步得到的计数 t 正是

$$|L(G)|.$$

因此 M 当且仅当

$$\langle G, k \rangle \in C_{\text{CFG}}$$

时接受，所以 C_{CFG} 是可判定的。■

解释. 这题的本质是先区分“有限”还是“无限”。若文法存在可回到自身的变量环，就能无限地产生新串；若不存在这种环，语法树高度就有统一上界，于是所有生成串的长度也有统一上界，接下来只需暴力枚举并计数即可。

Chapter 5

5.2

5.2 证明 EQ_{CFG} 是补图灵可识别的。

图 67

证明。要证明

EQ_{CFG}

是 co-Turing-recognizable, 只需证明它的补语言

$$\overline{EQ_{CFG}} = \{\langle G, H \rangle \mid L(G) \neq L(H)\}$$

是 Turing-recognizable。

构造识别器 R 如下。在输入

$$\langle G, H \rangle$$

上, 枚举所有字符串

$$w_1, w_2, w_3, \dots$$

并对每个 w_i 测试:

1. 是否

$$w_i \in L(G);$$

2. 是否

$$w_i \in L(H).$$

因为 CFG 的成员资格问题是可判定的, 所以这两个测试都能停机。若发现某个 w_i 满足

$$w_i \in L(G) \setminus L(H) \quad \text{或} \quad w_i \in L(H) \setminus L(G),$$

就接受。

若

$$L(G) \neq L(H),$$

则它们的对称差非空, 故必存在某个字符串最终会被枚举到并使 R 接受。若

$$L(G) = L(H),$$

则 R 永不接受。

因此

$$\overline{EQ_{CFG}}$$

是 Turing-recognizable, 从而

$$EQ_{CFG}$$

是 co-Turing-recognizable。■

解释. “两个 CFG 不相等” 只要找到一个见证串即可。因为 CFG 成员资格可判定，我们可以把所有字符串逐个检验；一旦发现落在对称差中的字符串，就识别出了补语言。

5.5

^ 5.5 证明 A_{TM} 无法映射归约到 E_{TM} 。换句话说，也就是证明没有可计算函数可以将 A_{TM} 归约为 E_{TM} 。（提示：用 A_{TM} 和 E_{TM} 的一些已知的矛盾和事实来证明。）

图 68

证明。反设存在一个可计算函数 f ，把

A_{TM}

映射归约到

E_{TM} 。

也就是说，对任意输入 x ，

$$x \in A_{TM} \iff f(x) \in E_{TM}.$$

记

$$\overline{E_{TM}} = \{\langle M \rangle \mid L(M) \neq \emptyset\}.$$

这个语言是 Turing-recognizable。设图灵机 R 识别

$\overline{E_{TM}}$ 。

我们构造图灵机 S 来识别

$\overline{A_{TM}}$

如下：

Algorithm 1 构造的图灵机 S

Require: 输入 x

Ensure: 若 $x \in \overline{A_{TM}}$ 则接受；否则不接受

- 1: 计算 $f(x)$
 - 2: 在输入 $f(x)$ 上运行 R
 - 3: **if** R 接受 **then**
 - 4: **接受**
 - 5: **end if**
-

下面验证 S 的正确性。对任意输入 x ，

$$x \in \overline{A_{TM}} \iff x \notin A_{TM} \iff f(x) \notin E_{TM} \iff f(x) \in \overline{E_{TM}}.$$

因此，若

$$x \in \overline{A_{TM}},$$

则 R 接受 $f(x)$ ，故 S 接受；若

$$x \notin \overline{A_{TM}},$$

则 R 不会接受 $f(x)$, 故 S 也不接受。于是 S 识别了

$$\overline{A_{\text{TM}}}.$$

这与已知结论矛盾, 因为

$$\overline{A_{\text{TM}}}$$

不是 Turing-recognizable。

故不存在这样的可计算函数 f 。因此

$$A_{\text{TM}} \not\leq_m E_{\text{TM}}.$$

■

解释. 这题不是说两个语言都不可判定, 就一定不能归约; 关键在于它们的“可识别方向”不同。 E_{TM} 是 co-Turing-recognizable, 而 A_{TM} 不是; 如果真能把 A_{TM} 归约到 E_{TM} , 补语言的可识别性就会被错误地传回去。

5.9

5.9 设 $T = \{\langle M \rangle \mid M \text{ 是一个 TM, 且每当 } M \text{ 接受 } w \text{ 时, } M \text{ 也接受 } w^R\}$, 证明 T 是不可判定的。

图 69

证明

设

$$T = \{\langle M \rangle \mid M \text{ 是一台 TM, 且当 } M \text{ 接受 } w \text{ 时也接受 } w^R\}.$$

反设 T 可判定, 设判定器为 R 。我们构造图灵机 S 来判定

$$A_{\text{TM}}.$$

给定输入 $\langle M, w \rangle$, 构造图灵机 N :

$N =$ “对于输入 x :

1. 若 $x = 01$, 则模拟 M 在 w 上的运行。
2. 若模拟接受, 则接受; 否则拒绝或继续模拟。
3. 若 $x \neq 01$, 则拒绝。”

于是:

$$\langle M, w \rangle \in A_{\text{TM}} \implies L(N) = \{01\},$$

而

$$(01)^R = 10 \notin L(N),$$

所以

$$\langle N \rangle \notin T.$$

若

$$\langle M, w \rangle \notin A_{\text{TM}},$$

则 M 在 w 上不接受, 于是 N 不接受任何字符串, 从而

$$L(N) = \emptyset.$$

空语言显然满足题目中的条件, 因此

$$\langle N \rangle \in T.$$

所以

$$\langle M, w \rangle \in A_{\text{TM}} \iff \langle N \rangle \notin T.$$

于是 S 在输入 $\langle M, w \rangle$ 上先构造 $\langle N \rangle$, 再运行 R ; 若 R 接受则拒绝, 若 R 拒绝则接受。这样 S 就判定了 A_{TM} , 矛盾。

故 T 不可判定。■

5.13

5.13 图灵机的一个无用状态, 是对任何输入它都不会进入的状态。考虑检查一个图灵机是否有无用状态的问题。将这个问题形式化为一个语言, 并证明它是不可判定的。

图 70

形式化

把题目形式化为语言

$$\text{HASUSELESS}_{\text{TM}} = \{\langle M \rangle \mid M \text{ 是一台 TM, 且 } M \text{ 有一个无用状态}\}.$$

证明

我们从

$$E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$$

归约到 $\text{HASUSELESS}_{\text{TM}}$ 。已知 E_{TM} 不可判定。

给定输入 $\langle M \rangle$, 构造图灵机 N 。 N 含有一个特别指定的状态 q_* , 并使用一套固定的通用模拟器状态来模拟 M ; 这些模拟器状态与 M 本身的状态无关。

N 的工作方式如下:

N = “对于输入 z :

1. 若 z 以 1 开头, 则沿一条预先写好的有限路径运行, 依次访问 N

的每个状态 (除了 q_*) 一次, 然后停机。

2. 若 z 以 0 开头, 设 $z = 0x$, 则利用固定的通用模拟器模拟 M 在 x 上的运行。

3. 若模拟发现 M 接受 x , 则进入 q_* , 随后接受。

4. 否则按模拟结果拒绝或继续模拟。”

由第 1 步可知, N 的所有状态除了 q_* 以外, 都在某个以 1 开头的输入上被访问, 因此它们都不是无用状态。于是 N 是否有无用状态, 完全取决于 q_* 是否无用。

现在考察 q_* :

$$q_* \text{ 可达} \iff \exists x (M \text{ 接受 } x) \iff L(M) \neq \emptyset.$$

因此

$$L(M) = \emptyset \iff q_* \text{ 是无用状态} \iff \langle N \rangle \in \text{HASUSELESS}_{\text{TM}}.$$

若 $\text{HASUSELESS}_{\text{TM}}$ 可判定, 就可据此判定 E_{TM} , 矛盾。故

$$\text{HASUSELESS}_{\text{TM}}$$

不可判定。■

5.17

5.17 证明 PCP 在一元字母表上, 即在字母表 $\Sigma = \{1\}$ 上, 是可判定的。

图 71

证明

设 PCP 实例的字母表是一元字母表

$$\Sigma = \{1\}.$$

于是每张牌

$$\begin{bmatrix} t_i \\ \overline{b_i} \end{bmatrix}$$

只由两个正整数

$$m_i = |t_i|, \quad n_i = |b_i|$$

决定, 因为

$$t_i = 1^{m_i}, \quad b_i = 1^{n_i}.$$

对任意非空牌序列

$$i_1, i_2, \dots, i_k,$$

其顶部与底部拼接后相等, 当且仅当它们的长度相等, 即

$$m_{i_1} + \dots + m_{i_k} = n_{i_1} + \dots + n_{i_k}.$$

等价地,

$$(m_{i_1} - n_{i_1}) + \dots + (m_{i_k} - n_{i_k}) = 0.$$

记

$$d_i = m_i - n_i.$$

问题就变成: 能否取一个非空序列, 使若干个 d_i 的和为 0。

这可以完全按符号分类:

- (1) 若某个 $d_i = 0$, 则单独选这一张牌就是一个解。
- (2) 若存在 $d_i > 0$ 且存在 $d_j < 0$, 则也一定有解。因为可以取 $|d_j|$ 张第 i 张牌和 d_i 张第 j 张牌, 于是总差为

$$|d_j|d_i + d_id_j = 0.$$

由于字母表是一元的, 只要总长度相等, 顶部串和底部串就必然相同。

- (3) 若所有 d_i 都严格为正, 或都严格为负, 则任意非空序列的总差也严格为正或严格为负, 不可能得到 0, 因此无解。

所以, 一元字母表上的 PCP 实例有解, 当且仅当满足下面两种情形之一:

$$\exists i (d_i = 0), \quad \text{或} \quad \exists i, j (d_i > 0 \text{ 且 } d_j < 0).$$

这只是对有限个整数做检查, 因此是可判定的。故 PCP 在一元字母表上可判定。■

5.21

5.21 设 $AMBIG_{CFG} = \{\langle G \rangle \mid G \text{ 是歧义的 CFG}\}$ 。证明 $AMBIG_{CFG}$ 是不可判定的。(提示: 使用来自 PCP 的归约。给一个 PCP 的实例

$$P = \left\{ \left[\frac{t_1}{b_1} \right], \left[\frac{t_2}{b_2} \right], \dots, \left[\frac{t_k}{b_k} \right] \right\},$$

用下列规则构造一个 CFG G :

$$S \rightarrow T \mid B$$

$$T \rightarrow t_1 T a_1 \mid \dots \mid t_k T a_k \mid t_1 a_1 \mid \dots \mid t_k a_k$$

$$B \rightarrow b_1 B a_1 \mid \dots \mid b_k B a_k \mid b_1 a_1 \mid \dots \mid b_k a_k$$

其中 $a_1, \dots, a_k$ 是新的终结符号。证明这个归约可行。)

图 72

证明

反设存在图灵机 R 判定

$$AMBIG_{CFG} = \{\langle G \rangle \mid G \text{ 是歧义的 CFG}\}.$$

我们构造图灵机 S 来判定 PCP。

给定 PCP 实例

$$P = \left\{ \left[\frac{t_1}{b_1} \right], \left[\frac{t_2}{b_2} \right], \dots, \left[\frac{t_k}{b_k} \right] \right\},$$

引入新的终结符

$$a_1, a_2, \dots, a_k,$$

并按题目提示构造文法

$$S \rightarrow T \mid B,$$

$$T \rightarrow t_1 T a_1 \mid \dots \mid t_k T a_k \mid t_1 a_1 \mid \dots \mid t_k a_k,$$

$$B \rightarrow b_1 B a_1 \mid \dots \mid b_k B a_k \mid b_1 a_1 \mid \dots \mid b_k a_k.$$

我们证明:

$$P \text{ 有解} \iff G \text{ 是歧义文法}.$$

若 P 有解, 则 G 歧义 设

$$i_1 i_2 \cdots i_m$$

是一个 PCP 解, 即

$$t_{i_1} t_{i_2} \cdots t_{i_m} = b_{i_1} b_{i_2} \cdots b_{i_m}.$$

于是字符串

$$x = t_{i_1} t_{i_2} \cdots t_{i_m} a_{i_m} \cdots a_{i_2} a_{i_1}$$

既可由 T 推导, 也可由 B 推导, 因此

$$S \Rightarrow T \Rightarrow^* x \quad \text{且} \quad S \Rightarrow B \Rightarrow^* x.$$

所以 G 歧义。

若 G 歧义, 则 P 有解 注意, 由 T 生成的任意字符串末尾的

$$a_{i_m} \cdots a_{i_1}$$

唯一确定所使用的牌序列

$$i_1, \dots, i_m,$$

因此 T 本身不歧义; 同理 B 也不歧义。故 G 的歧义只能来自某个字符串同时属于 $L(T)$ 和 $L(B)$ 。

于是存在某个字符串 x , 既可写成

$$t_{i_1} \cdots t_{i_m} a_{i_m} \cdots a_{i_1},$$

也可写成

$$b_{i_1} \cdots b_{i_m} a_{i_m} \cdots a_{i_1}.$$

比较其前缀部分, 就得到

$$t_{i_1} \cdots t_{i_m} = b_{i_1} \cdots b_{i_m},$$

故牌序列 $i_1, \dots, i_m$ 是 PCP 的一个解。

因此

$$\langle P \rangle \in \text{PCP} \iff \langle G \rangle \in \text{AMBIG}_{\text{CFG}}.$$

若 R 存在, 就能判定 PCP, 矛盾。故

$$\text{AMBIG}_{\text{CFG}}$$

不可判定。■

5.22

5.22 证明当且仅当 $A \leq_m A_{\text{TM}}$ 时, A 是图灵可识别的。

图 73

证明

我们证明下面两个方向。

若 $A \leq_m A_{\text{TM}}$, 则 A 是图灵可识别的 设 f 是一个可计算函数, 满足对任意字符串 w ,

$$w \in A \iff f(w) \in A_{\text{TM}}.$$

又因为 A_{TM} 是图灵可识别的, 设 U 是其识别器。构造识别器 R :

$R =$ “对于输入 w :

1. 计算 $f(w)$ 。
2. 在输入 $f(w)$ 上运行 U 。
3. 若 U 接受, 则接受。”

若 $w \in A$, 则 $f(w) \in A_{\text{TM}}$, 于是 U 接受, R 接受。若 $w \notin A$, 则 $f(w) \notin A_{\text{TM}}$, 于是 U 不会接受, R 也不接受。因此 R 识别 A , 故 A 图灵可识别。

若 A 是图灵可识别的, 则 $A \leq_m A_{\text{TM}}$ 设 M 是 A 的识别器。定义可计算函数

$$f(w) = \langle M, w \rangle.$$

显然 f 可计算, 且对任意 w ,

$$w \in A \iff M \text{ 接受 } w \iff \langle M, w \rangle \in A_{\text{TM}} \iff f(w) \in A_{\text{TM}}.$$

因此

$$A \leq_m A_{\text{TM}}.$$

综上,

$$A \text{ 是图灵可识别的} \iff A \leq_m A_{\text{TM}}.$$

■

5.24

5.24 设 $J = \{w \mid w = 0x \text{ 对于某个 } x \in A_{\text{TM}}, \text{ 或 } w = 1y \text{ 对于某个 } y \in \overline{A_{\text{TM}}}\}$ 。证明 J 和 $\bar{J}$ 都不是图灵可识别的。

图 74

证明

记

$$J = \{w \mid w = 0x \text{ 对于某个 } x \in A_{\text{TM}}, \text{ 或 } w = 1y \text{ 对于某个 } y \in \overline{A_{\text{TM}}}\}.$$

J 不是图灵可识别的 反设 J 是图灵可识别的, 设 R 是其识别器。构造图灵机 S 来识别

$$\overline{A_{\text{TM}}}$$

如下:

$S =$ “对于输入 y :

1. 构造字符串 $1y$ 。
2. 在输入 $1y$ 上运行 R 。
3. 若 R 接受, 则接受。”

因为

$$y \in \overline{A_{\text{TM}}} \iff 1y \in J,$$

所以 S 识别了 $\overline{A_{\text{TM}}}$ 。这与 $\overline{A_{\text{TM}}}$ 不是图灵可识别的矛盾。故 J 不是图灵可识别的。

$\overline{J}$ 也不是图灵可识别的 注意

$$\overline{J} = \{0x \mid x \in \overline{A_{\text{TM}}}\} \cup \{1y \mid y \in A_{\text{TM}}\}.$$

反设 $\overline{J}$ 是图灵可识别的, 设其识别器为 R' 。构造图灵机 S' 来识别 $\overline{A_{\text{TM}}}$:

S' = “对于输入 x :

1. 构造字符串 $0x$ 。
2. 在输入 $0x$ 上运行 R' 。
3. 若 R' 接受, 则接受。”

因为

$$x \in \overline{A_{\text{TM}}} \iff 0x \in \overline{J},$$

所以 S' 识别了 $\overline{A_{\text{TM}}}$, 再次矛盾。

因此

J 和 $\overline{J}$

都不是图灵可识别的。■

5.28

5.28 赖斯定理。设 P 是任何图灵机的语言的非平凡属性。证明确定某一图灵机的语言是否具有属性 P 这一问题是不可判定的。更正式地说, 设 P 是一个语言, 它由图灵机的描述组成, 且 P 满足下列两个条件: 首先, P 是非平凡的——它包含某些 TM 的描述, 但不是所有的; 其次, P 是 TM 的语言的属性——无论何时 $L(M_1) = L(M_2)$, 当且仅当 $\langle M_2 \rangle \in P$ 时, 有 $\langle M_1 \rangle \in P$, 此处的 M_1 和 M_2 是任意 TM。证明 P 是一个不可判定语言。

图 75

证明

设 $\mathcal{P}$ 是 TM 语言的一个非平凡属性。定义描述语言

$$L_{\mathcal{P}} = \{\langle M \rangle \mid L(M) \text{ 具有属性 } \mathcal{P}\}.$$

我们证明 $L_{\mathcal{P}}$ 不可判定。

因为 $\mathcal{P}$ 非平凡, 所以存在两台图灵机 M_{yes} 和 M_{no} , 满足

$$L(M_{\text{yes}}) \text{ 具有 } \mathcal{P}, \quad L(M_{\text{no}}) \text{ 不具有 } \mathcal{P}.$$

分两种情形讨论。

情形 1: $\emptyset$ 不具有属性 $\mathcal{P}$ 反设 $L_{\mathcal{P}}$ 可判定, 设判定器为 R 。我们构造图灵机 S 来判定 A_{TM} 。
给定输入 $\langle M, w \rangle$, 构造图灵机 N :

$N =$ “对于输入 x :

1. 模拟 M 在 w 上的运行。
2. 若 M 接受 w , 则模拟 M_{yes} 在 x 上的运行。
3. 若 M 不接受 w , 则永不接受 x 。”

若 $\langle M, w \rangle \in A_{\text{TM}}$, 则

$$L(N) = L(M_{\text{yes}}),$$

因而 $L(N)$ 具有属性 $\mathcal{P}$ 。若 $\langle M, w \rangle \notin A_{\text{TM}}$, 则

$$L(N) = \emptyset,$$

而按本情形假设, $\emptyset$ 不具有属性 $\mathcal{P}$ 。所以

$$\langle M, w \rangle \in A_{\text{TM}} \iff \langle N \rangle \in L_{\mathcal{P}}.$$

运行 R 即可判定 A_{TM} , 矛盾。

情形 2: $\emptyset$ 具有属性 $\mathcal{P}$ 此时仍反设 $L_{\mathcal{P}}$ 可判定。给定 $\langle M, w \rangle$, 构造图灵机 N :

$N =$ “对于输入 x :

1. 模拟 M 在 w 上的运行。
2. 若 M 接受 w , 则模拟 M_{no} 在 x 上的运行。
3. 若 M 不接受 w , 则永不接受 x 。”

若 $\langle M, w \rangle \in A_{\text{TM}}$, 则

$$L(N) = L(M_{\text{no}}),$$

不具有属性 $\mathcal{P}$ 。若 $\langle M, w \rangle \notin A_{\text{TM}}$, 则

$$L(N) = \emptyset,$$

而按本情形假设, $\emptyset$ 具有属性 $\mathcal{P}$ 。因此

$$\langle M, w \rangle \in A_{\text{TM}} \iff \langle N \rangle \notin L_{\mathcal{P}}.$$

再次得到 A_{TM} 可判定, 矛盾。

两种情形都导出矛盾, 所以

$$L_{\mathcal{P}}$$

不可判定。这就是 Rice 定理。■

术语说明

这里“平凡”与“非平凡”说的是一个性质，不是说某个具体语言本身简单不简单。

更一般地，若我们在某一类对象 C 上讨论一个性质 $\mathcal{P}$ ，则：

$\mathcal{P}$ 是平凡的

是指它要么对 C 中每个对象都成立，要么对 C 中每个对象都不成立；也就是说，这个性质不能把这类对象区分开。

相反，

$\mathcal{P}$ 是非平凡的

是指它对某些对象成立、对另一些对象不成立。等价地，存在

$A, B \in C$

使得 A 有性质 $\mathcal{P}$ ，而 B 没有性质 $\mathcal{P}$ 。

在本题里， C 不是“所有图灵机描述”，而是“所有图灵机可识别的语言”。因此，“ $\mathcal{P}$ 是 TM 语言的非平凡属性”的意思是：

有些图灵机可识别语言具有 $\mathcal{P}$ ， 而另一些图灵机可识别语言不具有 $\mathcal{P}$ 。

这也正是后面引入

$M_{\text{yes}}, M_{\text{no}}$

的原因：它们分别提供“有这个性质”和“没有这个性质”的两个例子。

例如，“语言是否包含字符串 101”是非平凡的，因为有些可识别语言包含 101，有些不包含；而“这个语言是否是某台图灵机可识别的”若把讨论范围本身就限定在 TM 可识别语言上，那它就是平凡的，因为答案永远是“是”。

证明思路

Rice 定理的核心不是分析图灵机的程序细节，而是利用“这个问题只取决于语言”这一点，把任意一个接受问题嵌进去。

具体做法是：因为属性 $\mathcal{P}$ 非平凡，所以总能先找出两个样板语言，一个有 $\mathcal{P}$ ，一个没有 $\mathcal{P}$ 。然后对任意输入 $\langle M, w \rangle$ ，构造一台新机器 N ，让它的语言在两种情况之间切换：

$M \text{ 接受 } w \Rightarrow L(N) \text{ 变成某个预先选好的 yes-语言；}$

$M \text{ 不接受 } w \Rightarrow L(N) \text{ 变成某个预先选好的 no-语言。}$

这样一来，只要能判定

$L(N)$ 是否具有 $\mathcal{P}$,

就等于能判定

$\langle M, w \rangle \in A_{\text{TM}},$

这就与 A_{TM} 不可判定矛盾。

证明里之所以分成“ $\emptyset$ 是否具有 $\mathcal{P}$ ”两种情形，只是为了让“不接受时的备用语言”选得最方便。整体结构始终一样：把原问题编码成“新机器的语言到底落在哪一边”。

5.33

5.33 设 $S = \{\langle M \rangle \mid M \text{ 是一个 TM, 且 } L(M) = \{\langle M \rangle\}\}$ 。证明 S 和 $\bar{S}$ 都不是图灵可识别的。

图 76

证明 S 不是图灵可识别的

我们从

$$E_{\text{TM}} = \{\langle M \rangle \mid L(M) = \emptyset\}$$

归约到 S 。已知 E_{TM} 不是图灵可识别的。

给定输入 $\langle M \rangle$ ，利用递归定理构造一台图灵机 N ，使它知道自己的描述

$$d = \langle N \rangle.$$

N 的行为定义为：

N = “对于输入 x ：

1. 枚举所有长度不超过 $|x|$ 的字符串，并把 M 在这些字符串上的计算并行模拟 $|x|$ 步。
2. 若在上述有限搜索中发现 M 接受了某个字符串，则接受 x 。
3. 若没有发现接受且 $x = d$ ，则接受；否则拒绝。”

若

$$L(M) = \emptyset,$$

则第 1 步永远找不到接受，故 N 只接受 d 。于是

$$L(N) = \{d\} = \{\langle N \rangle\},$$

所以

$$\langle N \rangle \in S.$$

若

$$L(M) \neq \emptyset,$$

设 M 在某个字符串 y 上经过 t 步接受。取任意字符串 x ，满足

$$|x| \geq \max\{|y|, t\} \quad \text{且} \quad x \neq d.$$

则第 1 步会发现这一接受计算，从而 N 接受这个 x 。因此 $L(N)$ 至少包含一个不同于 d 的字符串，故

$$L(N) \neq \{d\}.$$

于是

$$\langle N \rangle \notin S.$$

所以

$$\langle M \rangle \in E_{\text{TM}} \iff \langle N \rangle \in S.$$

这说明

$$E_{\text{TM}} \leq_m S.$$

若 S 图灵可识别, 则 E_{TM} 也图灵可识别, 矛盾。故 S 不是图灵可识别的。

证明 $\bar{S}$ 也不是图灵可识别的

仍从 E_{TM} 归约。给定 $\langle M \rangle$, 再次利用递归定理构造一台知道自己描述

$$d = \langle N \rangle$$

的图灵机 N , 令

$N =$ “对于输入 x :

1. 若 $x \neq d$, 则立即拒绝。
2. 若 $x = d$, 则并行模拟 M 在所有字符串上的计算。
3. 一旦发现 M 接受了某个字符串, 就接受。”

若

$$L(M) = \emptyset,$$

则第 3 步永远不会发生, 因此 N 不接受任何字符串, 故

$$L(N) = \emptyset \neq \{d\}.$$

于是

$$\langle N \rangle \in \bar{S}.$$

若

$$L(M) \neq \emptyset,$$

则第 3 步最终会在输入 d 上接受, 而对任何 $x \neq d$, 第 1 步立即拒绝。因此

$$L(N) = \{d\} = \{\langle N \rangle\},$$

于是

$$\langle N \rangle \in S, \quad \langle N \rangle \notin \bar{S}.$$

所以

$$\langle M \rangle \in E_{\text{TM}} \iff \langle N \rangle \in \bar{S}.$$

这说明

$$E_{\text{TM}} \leq_m \bar{S}.$$

若 $\bar{S}$ 图灵可识别, 则 E_{TM} 也图灵可识别, 矛盾。故 $\bar{S}$ 不是图灵可识别的。■

本章总结

可判定的证明方法

证明一个语言可判定, 核心是真的构造出一台一定停机的判定器。常见做法有:

- (1) 直接构造判定器: 把题目要求翻译成一套明确的有限步骤, 并逐步说明为什么每一步都停机。

- (2) 归约到已知可判定的问题：把原问题变形成若干个已经知道可判定的子问题，例如成员性、空性、交、补等，再把这些子程序串起来。
- (3) 利用有限性或有界搜索：若问题可化成对有限个对象的枚举与检查，或者虽然看似无限但可以压缩成有限判据，就往往可以得到判定器。
- (4) 利用闭包性质：若某类语言在某些运算下封闭，而这些运算之后的问题可判定，那么原问题也可判定。

不可判定的证明方法

证明不可判定，最常见的方法是归约。其标准结构是：

已知不可判定的问题 $\leq_m$ 当前要证明不可判定的问题.

也就是说，不是把当前问题归约到旧问题，而是把旧难题编码进当前问题中。这样一来，若当前问题可判定，旧难题也就可判定，产生矛盾。

本章里常见的来源问题包括：

A_{TM} , E_{TM} , PCP.

此外，Rice 定理提供了一种更高层的不可判定证明法：只要一个问题问的是“图灵机识别的语言是否具有某个非平凡属性”，那它通常不需要再为每一题重新设计细节归约，直接可由 Rice 定理推出不可判定。

不可识别的证明方法

证明一个语言不是图灵可识别的，通常比证明不可判定更强，因此也往往需要更强的矛盾来源。常见套路有：

- (1) 从一个已知不可识别的语言归约过来，例如

$\overline{A_{TM}}$ 或 E_{TM} .

- (2) 反设该语言可识别，然后构造识别器去识别一个已知不可识别的语言。
- (3) 若还要证明补语言也不可识别，就分别对语言和其补语言各做一次归约。
- (4) 对涉及“机器描述自身”的语言，常常需要配合递归定理或自引用构造。

从逻辑强弱上看：

可判定 $\implies$ 图灵可识别,

但反过来不成立；而

不是图灵可识别 $\implies$ 不可判定.

因此，一旦能证明“不可识别”，就自动完成了“不可判定”的证明。

证明思路的共性

这一章大多数证明虽然表面形式不同，但背后的共性很强：我们不是在直接“求解”某个具体输入，而是在分析一个问题本身承载了多少计算内容。

可判定性的证明，说明这个问题最终能被压缩成有限、机械、总会停机的步骤；不可判定性的证明，说明一旦你能解决它，就等于偷偷解决了某个已知无解的核心难题；不可识别性的证明，则更进一步表明，连“只在答案为 yes 时最终停下来报喜”都做不到。

为什么要这样做

从哲学上看，这一章关心的不是“我们暂时会不会做”，而是“原则上能不能做”。这和一般算法设计的视角不同：算法设计假设问题能解，然后追问怎样更快；可计算性理论先追问这个问题是否根本可被算法化。

之所以大量使用归约，是因为我们很难直接观察一个问题的全部无限行为，但可以比较两个问题之间的信息含量。归约本质上是在说：如果你能回答后者，你其实也就能回答前者。于是，不可解性并不是来自某个证明技巧的偶然，而是来自问题内部确实包含了足够复杂的计算结构。

Rice 定理背后的思想也很重要：一旦问题关心的是程序识别出的语言，而不是程序源码的表面形式，那么程序语义中的不确定性和无限性就会系统性地出现。换句话说，真正困难的不是语法，而是语义。

因此，本章的意义不只是学会若干定理和套路，更是建立一种边界意识：并非每个表达清楚的问题都能被算法解决；并非每个 yes/no 问题都能被完全判定；甚至并非每个问题都能在答案为 yes 时被可靠地“发现”。这些结论划出了计算的能力边界，也解释了为什么后续复杂性理论要在“已经可解”的前提下继续讨论时间与空间代价。

本章总结

可判定的证明方法

证明一个语言可判定，核心是真的构造出一台一定停机的判定器。常见做法有：

- (1) 直接构造判定器：把题目要求翻译成一套明确的有限步骤，并逐步说明为什么每一步都停机。
- (2) 归约到已知可判定的问题：把原问题变形成若干个已经知道可判定的子问题，例如成员性、空性、交、补等，再把这些子程序串起来。
- (3) 利用有限性或有界搜索：若问题可化成对有限个对象的枚举与检查，或者虽然看似无限但可以压缩成有限判据，就往往可以得到判定器。
- (4) 利用闭包性质：若某类语言在某些运算下封闭，而这些运算之后的问题可判定，那么原问题也可判定。

不可判定的证明方法

证明不可判定，最常见的方法是归约。其标准结构是：

已知不可判定的问题 $\leq_m$ 当前要证明不可判定的问题.

也就是说，不是把当前问题归约到旧问题，而是把旧难题编码进当前问题中。这样一来，若当前问题可判定，旧难题也就可判定，产生矛盾。

本章里常见的来源问题包括：

A_{TM} , E_{TM} , PCP.

此外，Rice 定理提供了一种更高层的不可判定证明法：只要一个问题问的是“图灵机识别的语言是否具有某个非平凡属性”，那它通常不需要再为每一题重新设计细节归约，直接可由 Rice 定理推出不可判定。

不可识别的证明方法

证明一个语言不是图灵可识别的，通常比证明不可判定更强，因此也往往需要更强的矛盾来源。常见套路有：

- (1) 从一个已知不可识别的语言归约过来，例如

$\overline{A_{TM}}$ 或 E_{TM} .

- (2) 反设该语言可识别，然后构造识别器去识别一个已知不可识别的语言。
- (3) 若还要证明补语言也不可识别，就分别对语言和其补语言各做一次归约。
- (4) 对涉及“机器描述自身”的语言，常常需要配合递归定理或自引用构造。

从逻辑强弱上看：

可判定 $\implies$ 图灵可识别，

但反过来不成立；而

不是图灵可识别 $\implies$ 不可判定。

因此，一旦能证明“不可识别”，就自动完成了“不可判定”的证明。

证明思路的共性

这一章大多数证明虽然表面形式不同，但背后的共性很强：我们不是在直接“求解”某个具体输入，而是在分析一个问题本身承载了多少计算内容。

可判定性的证明，说明这个问题最终能被压缩成有限、机械、总会停机的步骤；不可判定性的证明，说明一旦你能解决它，就等于偷偷解决了某个已知无解的核心难题；不可识别性的证明，则更进一步表明，连“只在答案为 yes 时最终停下来报喜”都做不到。

为什么要这样做

从哲学上看，这一章关心的不是“我们暂时会不会做”，而是“原则上能不能做”。这和一般算法设计的视角不同：算法设计假设问题能解，然后追问怎样更快；可计算性理论先追问这个问题是否根本可被算法化。

之所以大量使用归约，是因为我们很难直接观察一个问题的全部无限行为，但可以比较两个问题之间的信息含量。归约本质上是在说：如果你能回答后者，你其实也就能回答前者。于是，不可解性并不是来自某个证明技巧的偶然，而是来自问题内部确实包含了足够复杂的计算结构。

Rice 定理背后的思想也很重要：一旦问题关心的是程序识别出的语言，而不是程序源码的表面形式，那么程序语义中的不确定性和无限性就会系统性地出现。换句话说，真正困难的不是语法，而是语义。

因此，本章的意义不只是学会若干定理和套路，更是建立一种边界意识：并非每个表达清楚的问题都能被算法解决；并非每个 yes/no 问题都能被完全判定；甚至并非每个问题都能在答案为 yes 时被可靠地“发现”。这些结论划出了计算的能力边界，也解释了为什么后续复杂性理论要在“已经可解”的前提下继续讨论时间与空间代价。

本章总结

可判定的证明方法

证明一个语言可判定，核心是真的构造出一台一定停机的判定器。常见做法有：

- (1) 直接构造判定器：把题目要求翻译成一套明确的有限步骤，并逐步说明为什么每一步都停机。
- (2) 归约到已知可判定的问题：把原问题变形成若干个已经知道可判定的子问题，例如成员性、空性、交、补等，再把这些子程序串起来。
- (3) 利用有限性或有界搜索：若问题可化成对有限个对象的枚举与检查，或者虽然看似无限但可以压缩成有限判据，就往往可以得到判定器。
- (4) 利用闭包性质：若某类语言在某些运算下封闭，而这些运算之后的问题可判定，那么原问题也可判定。

不可判定的证明方法

证明不可判定，最常见的方法是归约。其标准结构是：

已知不可判定的问题 $\leq_m$ 当前要证明不可判定的问题.

也就是说，不是把当前问题归约到旧问题，而是把旧难题编码进当前问题中。这样一来，若当前问题可判定，旧难题也就可判定，产生矛盾。

本章里常见的来源问题包括：

A_{TM} , E_{TM} , PCP.

此外，Rice 定理提供了一种更高层的不可判定证明法：只要一个问题问的是“图灵机识别的语言是否具有某个非平凡属性”，那它通常不需要再为每一题重新设计细节归约，直接可由 Rice 定理推出不可判定。

不可识别的证明方法

证明一个语言不是图灵可识别的，通常比证明不可判定更强，因此也往往需要更强的矛盾来源。常见套路有：

- (1) 从一个已知不可识别的语言归约过来，例如

$\overline{A_{TM}}$ 或 E_{TM} .

- (2) 反设该语言可识别，然后构造识别器去识别一个已知不可识别的语言。
- (3) 若还要证明补语言也不可识别，就分别对语言和其补语言各做一次归约。
- (4) 对涉及“机器描述自身”的语言，常常需要配合递归定理或自引用构造。

从逻辑强弱上看：

可判定 $\implies$ 图灵可识别，

但反过来不成立；而

不是图灵可识别 $\implies$ 不可判定.

因此，一旦能证明“不可识别”，就自动完成了“不可判定”的证明。

证明思路的共性

这一章大多数证明虽然表面形式不同，但背后的共性很强：我们不是在直接“求解”某个具体输入，而是在分析一个问题本身承载了多少计算内容。

可判定性的证明，说明这个问题最终能被压缩成有限、机械、总会停机的步骤；不可判定性的证明，说明一旦你能解决它，就等于偷偷解决了某个已知无解的核心难题；不可识别性的证明，则更进一步表明，连“只在答案为 yes 时最终停下来报喜”都做不到。

为什么要这样做

从哲学上看，这一章关心的不是“我们暂时会不会做”，而是“原则上能不能做”。这和一般算法设计的视角不同：算法设计假设问题能解，然后追问怎样更快；可计算性理论先追问这个问题是否根本可被算法化。

之所以大量使用归约，是因为我们很难直接观察一个问题的全部无限行为，但可以比较两个问题之间的信息含量。归约本质上是在说：如果你能回答后者，你其实也就能回答前者。于是，不可解性并不是来自某个证明技巧的偶然，而是来自问题内部确实包含了足够复杂的计算结构。

Rice 定理背后的思想也很重要：一旦问题关心的是程序识别出的语言，而不是程序源码的表面形式，那么程序语义中的不确定性和无限性就会系统性地出现。换句话说，真正困难的不是语法，而是语义。

因此，本章的意义不只是学会若干定理和套路，更是建立一种边界意识：并非每个表达清楚的问题都能被算法解决；并非每个 yes/no 问题都能被完全判定；甚至并非每个问题都能在答案为 yes 时被可靠地“发现”。这些结论划出了计算的能力边界，也解释了为什么后续复杂性理论要在“已经可解”的前提下继续讨论时间与空间代价。

Chapter 6

6.3

A 6.3 证明：如果 $A \leq_T B$ 且 $B \leq_T C$ ，则 $A \leq_T C$ 。

图 77

证明。假设

$$A \leq_T B \quad \text{且} \quad B \leq_T C.$$

由定义，存在一台带预言机 B 的图灵机 M^B 判定 A ，并且存在一台带预言机 C 的图灵机 N^C 判定 B 。也就是说，对任意字符串 x ，

$$N^C \text{ 接受 } x \iff x \in B.$$

构造下面这台带预言机 C 的图灵机 T^C 。

$T^C =$ “在输入 w 上：

1. 模拟 M^B 在输入 w 上的运行。
2. 每当被模拟的 M^B 向它的 B -预言机询问某个字符串 x 是否属于 B 时，执行下面的子过程：
 1. 在输入 x 上运行 N^C 。
 2. 如果 N^C 接受，则把这次 B -预言机询问回答为 YES；如果 N^C 拒绝，则回答为 NO。
3. 按照上一步得到的 oracle 答案继续模拟 M^B 。
4. 如果被模拟的 M^B 接受，则接受；如果它拒绝，则拒绝。”

现在证明 T^C 判定 A 。对每一次 M^B 发出的 B -预言机询问

$$x \in B?$$

机器 T^C 都用 $N^C(x)$ 来回答。因为 N^C 判定 B ，所以这个回答与真正的 B -预言机给出的回答完全相同。因此，在任意输入 w 上， T^C 对 $M^B(w)$ 的模拟与真正拥有 B -预言机时的运行一致。

于是对任意 w ，

$$T^C \text{ 接受 } w \iff M^B \text{ 接受 } w \iff w \in A.$$

此外， M^B 是判定器，所以它在每个输入上只进行有限步计算并最终停机；每次被调用的 N^C 也是判定器，所以每个 oracle 询问的替代计算也会停机。因此 T^C 对每个输入都停机。

故 T^C 是一台带预言机 C 的判定器，并且它判定 A 。所以

$$A \leq_T C.$$

这就证明了图灵可归约性的传递性。■

解释。 这是预言机版本的“函数复合”。 $A \leq_T B$ 的意思不是“由 A 判定 B ”，而是“如果有 B 的答案，就能判定 A ”。所以在证明传递性时，先用 M^B 把 A 的问题转化成若干个 B 的问题，再用 N^C 把每个 B 的问题转化成 C 的问题，最终得到一个 T^C 来判定 A 。

6.4

6.4 设 $A'_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ 是一个谕示 TM, } M^{A_{\text{TM}}} \text{ 接受串 } w\}$ 。证明： A'_{TM} 相对 A_{TM} 是不可判定的。

图 78

设

$$A_{\text{TM}}^{A_{\text{TM}}} = \{\langle M, w \rangle \mid M \text{ 是预言机图灵机且 } M^{A_{\text{TM}}} \text{ 接受 } w\}.$$

我们证明：

$$A_{\text{TM}}^{A_{\text{TM}}}$$

相对于预言机

$$A_{\text{TM}}$$

仍然不可判定。

反设存在一台带预言机 A_{TM} 的图灵机 $H^{A_{\text{TM}}}$ 判定

$$A_{\text{TM}}^{A_{\text{TM}}}.$$

也就是说，对任意预言机图灵机 M 和输入 w ，

$$H^{A_{\text{TM}}}(\langle M, w \rangle)$$

接受当且仅当

$$M^{A_{\text{TM}}} \text{ 接受 } w.$$

Algorithm 2 对角化构造的预言机图灵机 $D^{A_{\text{TM}}}$

Require: 输入 $\langle M \rangle$ ，其中 M 是预言机图灵机

Ensure: 与 $M^{A_{\text{TM}}}$ 在输入 $\langle M \rangle$ 上的行为相反

1: 在输入 $\langle M, \langle M \rangle \rangle$ 上运行 $H^{A_{\text{TM}}}$

2: **if** $H^{A_{\text{TM}}}$ 接受 **then**

3: **拒绝**

4: **else**

5: **接受**

6: **end if**

现在考察 $D^{A_{\text{TM}}}$ 在自身描述上的行为。若

$$D^{A_{\text{TM}}} \text{ 接受 } \langle D \rangle,$$

则按照 D 的定义，

$$H^{A_{\text{TM}}}(\langle D, \langle D \rangle \rangle)$$

必须拒绝。但 H 判定

$$A_{\text{TM}}^{A_{\text{TM}}},$$

因此它拒绝恰意味着

$$D^{A_{\text{TM}}}$$

不接受

$\langle D \rangle$,

矛盾。

反之, 若

$D^{A_{TM}}$ 不接受 $\langle D \rangle$,

则 $H^{A_{TM}}(\langle D, \langle D \rangle \rangle)$ 必须接受, 于是按 D 的定义,

$D^{A_{TM}}$ 接受 $\langle D \rangle$,

仍然矛盾。

故这样的 $H^{A_{TM}}$ 不存在。于是

$A_{TM}^{A_{TM}}$

相对于

A_{TM}

不可判定。■

解释. 这就是普通

A_{TM}

不可判定证明的相对化版本。即便机器本身已经拥有

A_{TM}

预言机, 我们仍然可以对“带这个预言机的机器是否接受”做对角化, 所以再强一层的接受问题依然超出这一层预言机的能力。

6.7

6.7 在递归定理的不动点形式(定理 6.6)中, 令变换 t 是互换图灵机描述中的状态 q_{accept} 和 q_{reject} 得到的函数。给出 t 的不动点的一个例子。

图 79

解答. 令 t 是这样一个可计算变换: 对任意图灵机描述

$\langle M \rangle$,

$t(\langle M \rangle)$ 是把 M 的

q_{accept} 与 q_{reject}

互换后得到的机器描述。

在递归定理的不动点形式里, t 的一个不动点是任何满足

$L(M) = L(t(\langle M \rangle))$

的机器 M 。一个最简单的例子是:

在任意输入上都一直循环、从不进入接受态或拒绝态的图灵机。

设这台机器为 M 。那么对任意输入 w ,

M 在 w 上一直循环,

因此

$$w \notin L(M).$$

把

q_{accept} 和 q_{reject}

互换之后, 机器的实际运行并没有任何变化, 因为这两个状态本来就永远不会被访问到。所以对任意输入 w ,

$$t(\langle M \rangle)$$

也仍然一直循环, 从而

$$w \notin L(t(\langle M \rangle)).$$

于是

$$L(M) = \emptyset = L(t(\langle M \rangle)).$$

故这台“对所有输入都循环”的图灵机就是 t 的一个不动点例子。■

解释. 这里的不动点不是说机器描述字面上一模一样, 而是说变换前后识别的语言相同。若一台机器从来不入接受态或拒绝态, 那么交换这两个状态根本不会改变它的行为, 因此自然就是一个不动点。

6.10

^ 6.10 给出下列句子的一个模型:

$$\begin{aligned} \phi_{\text{eq}} = & \forall x[R_1(x, x)] \\ & \wedge \forall x, y[R_1(x, y) \leftrightarrow R_1(y, x)] \\ & \wedge \forall x, y, z[(R_1(x, y) \wedge R_1(y, z)) \rightarrow R_1(x, z)]. \end{aligned}$$

图 80

解答. 一个最简单的模型是

$$\mathcal{M} = (\mathbb{N}, R_1^{\mathcal{M}}),$$

其中论域取为自然数集

$$\mathbb{N} = \{0, 1, 2, \dots\},$$

并把二元关系 R_1 解释为“相等关系”:

$$R_1^{\mathcal{M}}(x, y) \iff x = y.$$

下面验证它满足题目给出的三个条件。

(1) **自反性** 题中的第一部分是

$$\forall x R_1(x, x).$$

在模型 $\mathcal{M}$ 中, 这变成:

$$\forall x \in \mathbb{N}, x = x,$$

显然成立。

(2) **对称性** 题中的第二部分是

$$\forall x, y [R_1(x, y) \leftrightarrow R_1(y, x)].$$

在模型 $\mathcal{M}$ 中,

$$R_1(x, y) \iff x = y \iff y = x \iff R_1(y, x),$$

故该式成立。

(3) **传递性** 题中的第三部分是

$$\forall x, y, z [(R_1(x, y) \wedge R_1(y, z)) \rightarrow R_1(x, z)].$$

在模型 $\mathcal{M}$ 中, 若

$$R_1(x, y) \wedge R_1(y, z),$$

则

$$x = y \quad \text{且} \quad y = z,$$

从而

$$x = z,$$

也就是

$$R_1(x, z).$$

所以该式也成立。

因此

$$\mathcal{M} = (\mathbb{N}, =)$$

就是题目所求的一个模型。■

解释. 这个句子正是在刻画“等价关系”: 自反、对称、传递。最直接的例子就是把 R_1 解释成真正的“等于”。当然, 像“同奇偶”这样的关系也同样给出一个模型。

6.12

6.12 设 $(\mathcal{N}, <)$ 是有论域 $\mathcal{N}$ 和“小于”关系的模型, 证明 $\text{Th}(\mathcal{N}, <)$ 是可判定的。

图 81

解答. 我们把语言

$$\{<\}$$

中的每个公式有效翻译成语言

$$\{+\}$$

中的公式。关键观察是：在自然数上，

$$x < y \iff \exists z (z \neq 0 \wedge x + z = y).$$

因此，对任意一阶句子 φ （其唯一非逻辑符号是 $<$ ），都可以机械地构造一个新句子 $\varphi^\sharp$ （其唯一非逻辑符号是 $+$ ），方法是把 φ 中每个原子式

$$x < y$$

都替换成

$$\exists z (z \neq 0 \wedge x + z = y),$$

其余逻辑联结词和量词保持不变。

这样构造满足：

$$(\mathbb{N}, <) \models \varphi \iff (\mathbb{N}, +) \models \varphi^\sharp.$$

原因很直接：在自然数结构上，

$$<$$

与

$$\exists z (z \neq 0 \wedge x + z = y)$$

定义的是同一个关系，所以替换前后真值不变。

现在利用已知结果：

$$\text{Th}(\mathbb{N}, +)$$

是可判定的。于是判定任意句子

$$\varphi \in \text{Th}(\mathbb{N}, <),$$

只需：

(1) 有效地把 φ 翻译成 $\varphi^\sharp$ ；

(2) 用

$$\text{Th}(\mathbb{N}, +)$$

的判定算法判定 $\varphi^\sharp$ 是否为真。

若 $\varphi^\sharp$ 在

$$(\mathbb{N}, +)$$

中为真，则 φ 在

$$(\mathbb{N}, <)$$

中为真；反之亦然。故

$$\text{Th}(\mathbb{N}, <)$$

可判定。■

解释. 这里的符号 $\models$ 表示“满足”关系： $\mathcal{M} \models \psi$ 的意思是句子 ψ 在结构 $\mathcal{M}$ 中为真。因此

$$(\mathbb{N}, <) \models \varphi \iff (\mathbb{N}, +) \models \varphi^\sharp$$

就是说，原句子 φ 在自然数的小于结构中为真，当且仅当翻译后的句子 $\varphi^\sharp$ 在自然数的加法结构中为真。

需要注意的是，这里的存在量词不是说判定器要盲目枚举

$$z = 1, 2, 3, \dots$$

直到找到为止。若 x, y 已经被赋为具体自然数，那么

$$x + z = y$$

一旦成立就必有 $z \leq y$ ，所以只需要检查有限多个候选 $z = 1, \dots, y$ 。特别地，当 $x > y$ 时，对任意 $z \in \mathbb{N}$ 都有

$$x + z \geq x > y,$$

因此根本不存在满足 $x + z = y$ 的 z 。

对一般的一阶句子，证明也不是靠对量词取值作无界搜索，而是先把句子有效翻译到只含加法的语言中，再调用已知的

$$\text{Th}(\mathbb{N}, +)$$

的判定算法。也就是说，有限步停机性来自 Presburger 算术的可判定性，而不是来自对存在量词的朴素枚举。

这里的核心不是重新发明一个关于 $<$ 的判定器，而是把“小于”翻译成“加法存在一个正增量”。一旦这个翻译是有效的，而

$$(\mathbb{N}, +)$$

的理论已经可判定，那么

$$(\mathbb{N}, <)$$

的理论也就跟着可判定。

6.14

6.14 证明：对于任意两个语言 A 和 B ，存在语言 J 使得 $A \leq_T J$ 且 $B \leq_T J$ 。

图 82

解答。 定义语言

$$J = \{0x \mid x \in A\} \cup \{1x \mid x \in B\}.$$

我们证明

$$A \leq_T J \quad \text{且} \quad B \leq_T J.$$

证明 $A \leq_T J$. 构造带预言机 J 的图灵机 M_A^J ：在输入 x 上，仅向预言机询问一次

$$0x \in J?$$

若预言机回答“是”，则接受；否则拒绝。

由 J 的定义，

$$0x \in J \iff x \in A.$$

因此 M_A^J 判定 A ，从而

$$A \leq_T J.$$

证明 $B \leq_T J$. 同理, 构造带预言机 J 的图灵机 M_B^J : 在输入 x 上询问

$$1x \in J?$$

并按照答案接受或拒绝。因为

$$1x \in J \iff x \in B,$$

所以 M_B^J 判定 B , 从而

$$B \leq_T J.$$

于是我们找到了一个语言 J , 使得

$$A \leq_T J \quad \text{且} \quad B \leq_T J.$$

命题得证。■

解释. 这个构造就是把两个语言“并排打包”进一个新语言里。前缀 0 表示“这是关于 A 的问题”, 前缀 1 表示“这是关于 B 的问题”。于是同一个预言机 J 同时包含了两边的信息。

6.17

* **6.17** 设 A 和 B 是两个不交的语言。如果 $A \subseteq C$ 且 $B \subseteq \overline{C}$, 则称语言 C 分离 A 和 B 。描述两个不交的图灵可识别语言, 使得它们不可由任何的可判定语言分离。

图 83

解答. 取

$$A = \{\langle M \rangle \mid M \text{ 接受 } \langle M \rangle\},$$

$$B = \{\langle M \rangle \mid M \text{ 拒绝 } \langle M \rangle\}.$$

1. A 与 B 不相交且都是图灵可识别的。显然

$$A \cap B = \emptyset,$$

因为一台图灵机在同一个输入上不可能既接受又拒绝。

对 A , 给定输入 $\langle M \rangle$, 直接模拟 M 在输入 $\langle M \rangle$ 上的运行; 若 M 接受, 则接受。故 A 是图灵可识别的。

对 B , 给定输入 $\langle M \rangle$, 同样模拟 M 在输入 $\langle M \rangle$ 上的运行; 若 M 拒绝, 则接受。故 B 也是图灵可识别的。

2. 证明它们不能被任何可判定语言分离。反设存在一个可判定语言 C , 满足

$$A \subseteq C \quad \text{且} \quad B \subseteq \overline{C}.$$

设图灵机 R 判定 C 。构造图灵机 D :

Algorithm 3 由分离器 R 构造的图灵机 D

Require: 输入 x

Ensure: 在 R 的答案上对角化


```

1: 在输入  $x$  上运行  $R$ 
2: if  $R$  接受 then
3:   拒绝
4: else
5:   接受
6: end if

```

现在考虑 $\langle D \rangle$ 。

若

$$\langle D \rangle \in C,$$

则 R 在输入 $\langle D \rangle$ 上接受, 因此 D 在输入 $\langle D \rangle$ 上拒绝。于是

$$\langle D \rangle \in B.$$

但

$$B \subseteq \overline{C},$$

故

$$\langle D \rangle \notin C,$$

矛盾。

若

$$\langle D \rangle \notin C,$$

则 R 在输入 $\langle D \rangle$ 上拒绝, 因此 D 在输入 $\langle D \rangle$ 上接受。于是

$$\langle D \rangle \in A.$$

但

$$A \subseteq C,$$

故

$$\langle D \rangle \in C,$$

仍然矛盾。

因此不存在任何可判定语言 C 能把 A 与 B 分离开。故题目所求的两个语言就是上述 A 与 B 。■

解释. 这两个语言分别记录“机器在自己描述上接受”和“机器在自己描述上拒绝”。如果某个可判定语言真能把这两种情况完全分开, 就能像标准对角化那样造出一台在自己描述上永远与这个分离器作对的机器, 于是立刻矛盾。

6.22

6.22 证明函数 $K(x)$ 不是可计算函数。

图 84

解答. 反设函数

$$K(x)$$

是可计算的。我们将导出矛盾。

在这个假设下，构造如下图灵机 M ：给定输入 n （用二进制编码）， M 依次枚举所有字符串

$$x$$

并计算

$$K(x),$$

直到找到字典序最小的满足

$$K(x) > n$$

的字符串，然后输出该字符串。

Algorithm 4 假设 K 可计算时构造的机器 M

Require: 输入正整数 n

Ensure: 输出字典序最小的满足 $K(x) > n$ 的字符串

```

1: for all 字符串  $x$ , 按字典序枚举 do
2:   计算  $K(x)$ 
3:   if  $K(x) > n$  then
4:     输出  $x$  并停机
5:   end if
6: end for
```

先说明这样的字符串一定存在。因为长度至多为 n 的描述只有有限多个，所以满足

$$K(x) \leq n$$

的字符串也只有有限多个；而字符串总数是无限的，因此必然存在某个 x 使得

$$K(x) > n.$$

故机器 M 对每个输入 n 都会停机。

设机器 M 在输入 n 时输出的字符串为 x_n 。由构造，

$$K(x_n) > n.$$

另一方面， x_n 可以由“机器 M 的固定描述”加上“输入 n 的编码”唯一确定。因此存在常数 c ，使得对所有 n ，

$$K(x_n) \leq |\langle n \rangle| + c.$$

由于 n 采用二进制编码，

$$|\langle n \rangle| \leq \lceil \log_2 n \rceil + O(1),$$

所以存在常数 c' ，使得

$$K(x_n) \leq \log_2 n + c'.$$

综上，对充分大的 n ，

$$n < K(x_n) \leq \log_2 n + c',$$

这是不可能的，因为

$$\log_2 n + c' < n$$

当 n 足够大时成立。

矛盾说明最初假设错误。因此函数

$$K(x)$$

不是可计算函数。■

解释. 这里的 $K(x)$ 是字符串 x 的描述复杂度, 也就是 Kolmogorov 复杂度。教材在第 6.4 节的 Definition 6.23 中先定义 $d(x)$ 为 x 的最短描述: 它是最短的串 $\langle M, w \rangle$, 其中图灵机 M 在输入 w 上停机并输出 x ; 若有多个同样短的描述, 就取字典序最小的那个。然后定义

$$K(x) = |d(x)|.$$

所以 $K(x)$ 衡量的是生成 x 所需的最短有效描述长度。

证明中

$$K(x_n) \leq |\langle n \rangle| + c$$

的原因是: x_n 是固定机器 M 在输入 n 上输出的字符串。因此, 只要给出“机器 M 的固定描述”以及“输入 n 的编码”, 就能重新运行 $M(n)$ 并唯一得到 x_n 。所以

$$\langle M, n \rangle$$

本身就是 x_n 的一个合法描述。由于 $K(x_n)$ 取的是最短描述长度, 它不会比这个具体描述更长。

其中 M 已经固定, $\langle M \rangle$ 的长度以及配对编码的额外开销都可以并入一个常数 c , 于是只剩下输入 n 的编码长度:

$$K(x_n) \leq |\langle n \rangle| + c.$$

又因为 n 采用二进制编码, 所以

$$|\langle n \rangle| \leq \lceil \log_2 n \rceil + O(1),$$

从而可把常数项合并, 得到

$$K(x_n) \leq \log_2 n + c'.$$

这正是教材 Theorem 6.24 后面的思想: 为了给 $K(x)$ 找上界, 只需要展示一个能生成 x 的描述; 最短描述只会更短。

这是 Kolmogorov 复杂度版本的 Berry 悖论。若 K 真能计算, 我们就能有效地找到“第一个复杂度大于 n 的串”。但这样一个串又被“这台寻找它的机器 + 参数 n ”简短地描述了, 于是它反而不该那么复杂。

本章总结

这一章的题目集中在可计算性理论的几个高级工具上: 预言机与图灵归约、递归定理式的自引用、逻辑理论的可判定性、以及 Kolmogorov 复杂度。它们表面上分散, 但共同主题是: 当普通判定器不够用时, 我们怎样比较问题的计算能力, 或者证明某些能力边界仍然无法突破。

预言机与图灵归约

图灵归约

$$A \leq_T B$$

表示“若可以把 B 当作预言机使用, 就能判定 A ”。6.3 的传递性说明: 如果 A 的判定过程只需要询问 B , 而每个 B 的询问又能借助 C 回答, 那么这些询问可以逐层替换, 最终得到一个只使用 C 的判定器。

这就是预言机计算中的组合原则。

6.14 的构造

$$J = \{0x \mid x \in A\} \cup \{1x \mid x \in B\}$$

则说明：两个语言可以被打包进同一个预言机语言中。前缀 0 和 1 分别标记问题来自 A 还是 B ，因此 J 同时携带了两边的信息。

相对化与对角化

6.4 的重点是：即使机器已经拥有

$$A_{\text{TM}}$$

作为预言机，仍然不能判定“带这个预言机的机器是否接受输入”的更高一层问题

$$A_{\text{TM}}^{A_{\text{TM}}}.$$

证明仍然使用对角化。

这里的“对角化”不是线性代数中的对角化，而是计算理论中的自指取反技巧。可以把所有机器 M_i 和所有输入 $\langle M_j \rangle$ 排成一张表，对角线位置是

$$M_i(\langle M_i \rangle),$$

也就是“机器在自己的描述上运行”。对角化构造一台新机器 D ，专门在这些对角线位置上与假想判定器的预测相反：若判定器说接受， D 就拒绝；若判定器说不接受， D 就接受。最后把 D 自己放到自己的输入上，就会得到

$$D \text{ 接受 } \langle D \rangle \iff D \text{ 不接受 } \langle D \rangle$$

这种矛盾。

6.17 中的不可分离证明也是同一个思想。若存在可判定语言 C 能把

$$A = \{\langle M \rangle \mid M \text{ 接受 } \langle M \rangle\}$$

和

$$B = \{\langle M \rangle \mid M \text{ 拒绝 } \langle M \rangle\}$$

完全分开，就可以构造一台机器 D 在自己的描述上永远与这个分离器的答案相反，从而推出矛盾。因此，对角化的核心不是某个特殊语言，而是“假设有一个全局正确的判定器，再构造一个专门反着它来的自指对象”。

递归定理与自引用

6.7 展示了递归定理背后的不动点思想：对一个可计算变换 t ，可以寻找某种意义上不被 t 改变行为的机器。这里的例子是把接受态和拒绝态交换。如果机器在所有输入上都一直循环，那么接受态和拒绝态根本不会被访问，交换它们不改变识别的语言。因此它给出了一个直观的不动点。

这和对角化一样都涉及“机器谈论自己”，但方向不同：对角化用自引用制造矛盾，递归定理则说明图灵机可以在严格意义上获得并利用自己的描述。

逻辑理论与可判定性

6.10 和 6.12 属于逻辑理论部分。6.10 是模型构造题：把一个关系符号解释成等号，就得到满足自反、对称、传递的结构，也就是一个等价关系模型。

6.12 的关键是把自然数上的小于关系定义进加法结构：

$$x < y \iff \exists z (z \neq 0 \wedge x + z = y).$$

因此，任何只含 $<$ 的句子都可以有效翻译成只含 $+$ 的句子。由于

$$\text{Th}(\mathbb{N}, +)$$

即 Presburger 算术是可判定的，翻译之后就能调用它的判定算法，从而得到

$$\text{Th}(\mathbb{N}, <)$$

也是可判定的。这里要注意，存在量词本身不是要求无界枚举；有限步停机性来自加法理论的判定算法。

Kolmogorov 复杂度

6.22 讨论描述复杂度

$$K(x),$$

也就是生成字符串 x 所需的最短有效描述长度。证明 K 不可计算的思路是 Berry 悖论式的：如果 K 可计算，就能构造机器找出“第一个复杂度大于 n 的字符串”。但这个字符串又可以被“这台机器加参数 n ”很短地描述出来，于是它不可能真的有那么高的复杂度，矛盾。

这一部分说明，计算不可判定性不仅出现在“机器是否接受输入”这类问题中，也会出现在“一个对象到底含有多少可压缩信息”这种信息论式问题中。

整体脉络

本章可以看成三条主线的交汇：

- (1) 用预言机和图灵归约比较问题之间的相对计算能力；
- (2) 用对角化、递归定理和不可分离性刻画自引用带来的不可判定边界；
- (3) 用逻辑理论和 Kolmogorov 复杂度展示可计算性思想在数理逻辑与信息描述中的应用。

这些题共同强调一点：给计算模型增加工具并不等于消除所有不可判定性。预言机能提升能力，但仍会出现更高层的问题；逻辑语言可以互相翻译，但可判定性依赖于目标理论本身；字符串可以有描述复杂度，但这个复杂度函数本身不可计算。

Chapter 7

7.1

7.1 下面各项是真是假?

- | | |
|---------------------------------------|-------------------------------------|
| a. $2n = O(n)$ | ^A d. $n \log n = O(n^2)$ |
| b. $n^2 = O(n)$ | e. $3^n = 2^{O(n)}$ |
| ^A c. $n^2 = O(n \log^2 n)$ | f. $2^{2^n} = O(2^{2^n})$ |

图 85

解答

(a) $2n = O(n)$ ——真. 取常数 $c = 2, n_0 = 1$. 对所有 $n \geq n_0$, 有

$$2n \leq 2 \cdot n = c \cdot n,$$

因此 $2n = O(n)$ 。

(b) $n^2 = O(n)$ ——假. 假设存在常数 $c > 0$ 和 n_0 使得对所有 $n \geq n_0$ 有 $n^2 \leq cn$ 。两边同除以 n (此处 $n > 0$), 得

$$n \leq c,$$

但 n 可以任意大, 这与 c 为固定常数矛盾。故 $n^2 \neq O(n)$ 。

(c) $n^2 = O(n \log^2 n)$ ——假. 考察极限

$$\lim_{n \rightarrow \infty} \frac{n^2}{n \log^2 n} = \lim_{n \rightarrow \infty} \frac{n}{\log^2 n} = +\infty.$$

比值趋于无穷, 说明不存在常数 c 使得 $n^2 \leq c \cdot n \log^2 n$ 对足够大的 n 恒成立。故 $n^2 \neq O(n \log^2 n)$ 。

(d) $n \log n = O(n^2)$ ——真. 对所有 $n \geq 1$, 有 $\log n \leq n$, 因此

$$n \log n \leq n \cdot n = n^2.$$

取 $c = 1, n_0 = 1$ 即可。故 $n \log n = O(n^2)$ 。

(e) $3^n = 2^{O(n)}$ ——真. 利用换底公式

$$3^n = (2^{\log_2 3})^n = 2^{n \log_2 3}.$$

由于 $\log_2 3 \approx 1.585$ 是常数, 故 $n \log_2 3 = O(n)$, 即存在常数 $c = \log_2 3$ 使得 $n \log_2 3 \leq cn$ 。因此 $3^n = 2^{O(n)}$ 。

(f) $2^{2^n} = O(2^{2^n})$ ——真. 任何函数 $f(n)$ 都满足 $f(n) = O(f(n))$: 取 $c = 1, n_0 = 1$, 显然

$$2^{2^n} \leq 1 \cdot 2^{2^n}.$$

故 $2^{2^n} = O(2^{2^n})$ 。

案：结合 Big-O 和 Small-O 记号的定义，可无惑。

7.2

7.2 下面各项是真是假？

a. $n = o(2n)$

b. $2n = o(n^2)$

^A c. $2n = o(3^n)$

^A d. $1 = o(n)$

e. $n = o(\log n)$

f. $1 = o(1/n)$

图 86

小 o 的定义回顾。 $f(n) = o(g(n))$ 当且仅当

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

也就是说, f 相对于 g 是“真正可忽略”的; 仅仅相差常数倍并不满足小 o 关系。

解答

(a) $n = o(2n)$ ——假.

$$\lim_{n \rightarrow \infty} \frac{n}{2n} = \frac{1}{2} \neq 0.$$

n 与 $2n$ 只相差常数因子, 属于同阶, 不满足小 o 关系。

(b) $2n = o(n^2)$ ——真.

$$\lim_{n \rightarrow \infty} \frac{2n}{n^2} = \lim_{n \rightarrow \infty} \frac{2}{n} = 0.$$

故 $2n = o(n^2)$ 。

(c) $2^n = o(3^n)$ ——真.

$$\lim_{n \rightarrow \infty} \frac{2^n}{3^n} = \lim_{n \rightarrow \infty} \left(\frac{2}{3}\right)^n = 0,$$

因为底数 $2/3 < 1$ 。故 $2^n = o(3^n)$ 。

(d) $1 = o(n)$ ——真.

$$\lim_{n \rightarrow \infty} \frac{1}{n} = 0.$$

故 $1 = o(n)$ 。

(e) $n = o(\log n)$ ——假. 由洛必达法则 (或基本增长速度比较)

$$\lim_{n \rightarrow \infty} \frac{n}{\log n} = +\infty.$$

比值发散而非趋于 0, 故 $n \neq o(\log n)$ 。事实上关系恰好相反: $\log n = o(n)$ 。

(f) $1 = o(1/n)$ ——假.

$$\lim_{n \rightarrow \infty} \frac{1}{1/n} = \lim_{n \rightarrow \infty} n = +\infty.$$

比值发散, 故 $1 \neq o(1/n)$ 。正确的关系是 $1/n = o(1)$ 。

案：结合 Big-O 和 Small-O 记号的定义，可无惑。

7.5

7.5 下面的公式是可满足的吗？

$$(x \vee y) \wedge (x \vee \bar{y}) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y})$$

图 87

解答

枚举验证。只有两个变量 x, y ，共有 $2^2 = 4$ 种真值赋值。下表枚举全部可能：

x	y	$x \vee y$	$x \vee \bar{y}$	$\bar{x} \vee y$	$\bar{x} \vee \bar{y}$	φ
0	0	0	1	1	1	0
0	1	1	0	1	1	0
1	0	1	1	0	1	0
1	1	1	1	1	0	0

对于每一组赋值，总能找到至少一个子句的取值为 0，于是整个合取式为 0。

结构化解释。两变量的二元析取子句一共恰好有 4 个，而每个子句 $(\ell_1 \vee \ell_2)$ 只在令 ℓ_1, ℓ_2 同时为假时取 0，即恰好排除 $\{0, 1\}^2$ 中的一个赋值：

$$\begin{aligned} (x \vee y) : \text{排除}(x, y) &= (0, 0), \\ (x \vee \bar{y}) : \text{排除}(x, y) &= (0, 1), \\ (\bar{x} \vee y) : \text{排除}(x, y) &= (1, 0), \\ (\bar{x} \vee \bar{y}) : \text{排除}(x, y) &= (1, 1). \end{aligned}$$

这四个子句合取起来，恰把 $\{0, 1\}^2$ 的全部四种赋值全部排除，因此没有任何赋值能令公式为真。

结论。

$$\varphi \equiv 0,$$

即 φ 是一个矛盾式，不可满足 (unsatisfiable)。

7.7

7.7 证明 NP 在并和连接运算下封闭。

图 88

即：若 $A, B \in \mathbf{NP}$ ，则 $A \cup B \in \mathbf{NP}$ 且 $A \circ B \in \mathbf{NP}$ 。

总体思路。由 $A, B \in \mathbf{NP}$ ，存在非确定型图灵机 N_A 和 N_B ，分别在多项式时间内判定 A 和 B 。设 N_A 的运行时间为 $O(n^{k_1})$ ， N_B 的运行时间为 $O(n^{k_2})$ ，其中 k_1, k_2 为常数。我们分别构造两个非确定型图灵机 $N_{\cup}$ 与 $N_{\circ}$ 。判定 $A \cup B$ 与 $A \circ B$ 。

第一部分: 并封闭

构造 $N_{\cup}$. 对输入 w :

1. 非确定地在以下两个分支中选择其一:

- **分支 1:** 在 w 上模拟 N_A ; 若 N_A 接受则接受, 否则拒绝。
- **分支 2:** 在 w 上模拟 N_B ; 若 N_B 接受则接受, 否则拒绝。

正确性.

$$w \in A \cup B \iff w \in A \text{ 或 } w \in B \iff N_A \text{ 接受 } w \text{ 或 } N_B \text{ 接受 } w.$$

而 $N_{\cup}$ 接受 w 当且仅当其某个非确定分支接受 w , 因此 $N_{\cup}$ 恰好判定 $A \cup B$ 。

时间分析. 每个分支的运行时间为 $O(n^{k_1})$ 或 $O(n^{k_2})$, 因此 $N_{\cup}$ 的运行时间为 $O(n^{\max(k_1, k_2)})$, 仍为多项式。故 $A \cup B \in \mathbf{NP}$ 。

第二部分: 连接封闭

回忆连接的定义

$$A \circ B = \{xy \mid x \in A \text{ 且 } y \in B\}.$$

构造 $N_{\circ}$. 对输入 $w = w_1w_2 \cdots w_n$:

1. **非确定地猜切分点** $i \in \{0, 1, \dots, n\}$, 令 $x = w_1w_2 \cdots w_i, y = w_{i+1}w_{i+2} \cdots w_n$ (当 $i = 0$ 时 $x = \varepsilon$; 当 $i = n$ 时 $y = \varepsilon$)。
2. 在 x 上模拟 N_A 。
3. 在 y 上模拟 N_B 。
4. 若 N_A 接受 x 且 N_B 接受 y , 则接受; 否则在此分支拒绝。

正确性. $w \in A \circ B$ 当且仅当存在切分 $w = xy$ 使 $x \in A$ 且 $y \in B$ 。由非确定性, $N_{\circ}$ 的某条分支恰能猜到正确的 i ; 而在该分支里 N_A 有接受路径判定 $x \in A$, N_B 有接受路径判定 $y \in B$ 。综合三重非确定性 (切分点 + N_A 的分支 + N_B 的分支), $N_{\circ}$ 接受 w 当且仅当 $w \in A \circ B$ 。

时间分析. 猜切分点的代价为 $O(\log n)$ (记录一个 $\{0, \dots, n\}$ 之间的整数), 可放宽记为 $O(n)$ 。模拟 N_A 于 x 的代价为 $|x|^{k_1} \leq n^{k_1}$, 模拟 N_B 于 y 的代价为 $|y|^{k_2} \leq n^{k_2}$ 。因此 $N_{\circ}$ 总运行时间为

$$O(n + n^{k_1} + n^{k_2}) = O(n^{\max(k_1, k_2)}),$$

仍为多项式。故 $A \circ B \in \mathbf{NP}$ 。 ■

7.9

无向图中的三角形是一个 3 团。证明 $\text{TRIANGLE} \in P$, 其中 $\text{TRIANGLE} = \{\langle G \rangle \mid G \text{ 包含一个三角形}\}$ 。

证明

要证明 $\text{TRIANGLE} \in P$, 只需构造一个确定性多项式时间算法, 能够正确判定任意无向图 G 是否包含三角形。

算法构造

设输入无向图 $G = (V, E)$, 其中 $|V| = n$, $|E| = m$ 。以邻接矩阵表示 G , 则边的存在性查询为 $O(1)$ 。

Algorithm 5 判定 G 是否包含三角形的算法 M

Require: 无向图 $G = (V, E)$ 的邻接矩阵表示

Ensure: 若 G 包含三角形则接受, 否则拒绝

```

1: for all 互不相同的顶点三元组  $(u, v, w) \in V^3$ ,  $u < v < w$  do
2:   if  $(u, v) \in E$  且  $(v, w) \in E$  且  $(u, w) \in E$  then
3:     接受
4:   end if
5: end for
6: 拒绝

```

正确性证明

($\Rightarrow$) **充分性:** 若 G 包含三角形, 设该三角形的三个顶点为 $u, v, w \in V$, 则由三角形定义知:

$$(u, v) \in E, \quad (v, w) \in E, \quad (u, w) \in E.$$

算法在枚举三元组时必然枚举到 (u, v, w) , 此时三个条件全部成立, 算法**接受**。

($\Leftarrow$) **必要性:** 若算法接受, 则存在某三元组 (u, v, w) 使得三条边:

$$(u, v) \in E, \quad (v, w) \in E, \quad (u, w) \in E$$

均成立。由三角形的定义, $\{u, v, w\}$ 构成 G 中的一个三角形。

综合两方面, 算法正确判定 TRIANGLE。

时间复杂度分析

- **枚举三元组:** 从 n 个顶点中选取 3 个互不相同的顶点, 共有

$$\binom{n}{3} = \frac{n(n-1)(n-2)}{6} = O(n^3)$$

个三元组。

- **单次检查:** 对每个三元组, 使用邻接矩阵检查三条边是否存在, 每次查询为 $O(1)$, 三条边共 $O(1)$ 。
- **总复杂度:** 算法总时间复杂度为

$$O(n^3) \cdot O(1) = O(n^3).$$

$O(n^3)$ 是关于输入规模 n 的多项式, 故算法 M 是一个**确定性多项式时间算法**。

案: 证明问题属于 P, 即能够给出一个多项式时间的算法, 并分析时间负责度。

7.10

证明 ALL_{DFA} 属于 P。

证明

记

$$\text{ALL}_{DFA} = \{\langle A \rangle \mid A \text{ 是一个 DFA, 且 } L(A) = \Sigma^*\}.$$

要证明 $\text{ALL}_{DFA} \in P$, 只需构造一个确定性多项式时间判定算法。

关键思路：将问题转化为图的可达性问题。 设

$$A = (Q, \Sigma, \delta, q_0, F)$$

是一个 DFA。注意到

$$L(A) = \Sigma^* \iff A \text{ 接受每一个字符串} \iff A \text{ 没有任何可达的拒绝状态.}$$

等价地,

$$L(A) \neq \Sigma^* \iff \text{存在某个状态 } q \in Q \setminus F, \text{ 使得 } q \text{ 从 } q_0 \text{ 可达.}$$

将状态转移函数 δ 视为一张有向图 G_A : 顶点集为 Q , 对每个 $q \in Q$ 和每个 $a \in \Sigma$, 从 q 向 $\delta(q, a)$ 连一条边。于是, 判定 $\langle A \rangle \in \text{ALL}_{DFA}$ 就等价于判断: 是否存在一个从 q_0 可达且不属于 F 的顶点。

算法构造

Algorithm 6 判定 $\langle A \rangle \in \text{ALL}_{DFA}$ 的算法 M

Require: DFA $A = (Q, \Sigma, \delta, q_0, F)$

Ensure: 若 $L(A) = \Sigma^*$ 则接受, 否则拒绝

```

1: 标记初始状态  $q_0$ 
2: repeat
3:   对每个已标记状态  $q$  和每个字符  $a \in \Sigma$ , 标记状态  $\delta(q, a)$ 
4: until 没有新状态被标记
5: if 存在某个已标记状态  $q \notin F$  then
6:   拒绝 ▷ 存在可达的拒绝状态
7: else
8:   接受 ▷ 所有可达状态均为接受状态
9: end if

```

正确性证明

($\Rightarrow$) **充分性:** 步骤 1-2 计算出从 q_0 出发在图 G_A 中可达的全部状态集合, 记为 $R \subseteq Q$ 。若 $L(A) = \Sigma^*$, 则对任意 $w \in \Sigma^*$, 都有

$$\delta^*(q_0, w) \in F.$$

因此从 q_0 可达的每个状态都属于 F , 即 $R \subseteq F$ 。于是步骤 3 中不存在已标记的拒绝状态, 算法接受。

($\Leftarrow$) **必要性:** 若 $L(A) \neq \Sigma^*$, 则存在某个字符串 $w \in \Sigma^*$ 被 A 拒绝。令

$$q = \delta^*(q_0, w),$$

则 $q \notin F$, 且 q 由字符串 w 从 q_0 可达, 所以 $q \in R$ 。因此步骤 3 一定会发现某个已标记的拒绝状态并拒绝。故算法接受当且仅当 $L(A) = \Sigma^*$, 算法正确。

时间复杂度分析

设 $n = |Q|$, $k = |\Sigma|$ 。

- **步骤 1:** 标记初始状态 q_0 , 耗时 $O(1)$ 。
- **步骤 2:** 每轮至多检查 n 个已标记状态与 k 个字符的组合, 共 $O(nk)$ 次转移查询; 而轮数至多为 n , 因为每轮若继续执行, 至少会新增一个已标记状态。因此这一步总耗时为

$$O(n^2k).$$

- **步骤 3:** 扫描全部已标记状态并判断是否属于 F , 耗时 $O(n)$ 。
- **总复杂度:**

$$O(1) + O(n^2k) + O(n) = O(n^2k),$$

这是关于输入规模的多项式时间。

因此, 算法 M 是一个确定性多项式时间算法, 能够正确判定 ALL_{DFA} 。故

$$\text{ALL}_{DFA} \in \mathbf{P}.$$

案: 先把问题翻译成“是否存在可达的拒绝状态”, 再在状态图上做可达性搜索。由于状态数和字母表大小都是有限的, 整个过程在多项式时间内完成。

7.14

7.14 证明 $\mathbf{P}$ 在星号运算下封闭。(提示: 使用动态规划。对于任意的 $A \in \mathbf{P}$, 对输入 $y = y_1 \cdots y_n$, 其中 $y_i \in \Sigma$, 建一个表, 表示对于每个 $i \leq j$, 是否有子串 $y_i \cdots y_j \in A^*$ 。)

图 89

证明

我们证明: 若 $A \in \mathbf{P}$, 则 $A^* \in \mathbf{P}$ 。

设 $A \in \mathbf{P}$, 则存在一个确定性多项式时间判定机 M_A , 对任意长度为 m 的输入, 都能在时间 $O(m^c)$ 内判定其是否属于 A , 其中 c 是某个固定常数。下面构造一个确定性多项式时间算法判定 A^* 。

关键思路: 动态规划。 设输入字符串为

$$y = y_1 y_2 \cdots y_n,$$

其中每个 $y_i \in \Sigma$ 。当 $n = 0$ 时, $y = \varepsilon$, 而 $\varepsilon \in A^*$ 恒成立, 因此可直接接受。

对非空输入, 我们建立一个布尔表 T 。对任意 $1 \leq i \leq j \leq n$, 定义

$$T[i][j] = \begin{cases} \text{true}, & \text{若子串 } y_i y_{i+1} \cdots y_j \in A^*, \\ \text{false}, & \text{否则。} \end{cases}$$

最后只需检查 $T[1][n]$ 是否为真。

递推关系. 子串 $y_i \cdots y_j$ 属于 A^* , 当且仅当它可以写成

$$(y_i \cdots y_{k-1})(y_k \cdots y_j),$$

其中前半段属于 A^* , 后半段属于 A 。这里允许前半段为空串; 当 $k = i$ 时, 前半段就是 ε , 显然属于 A^* 。
因此, 对每个 $i \leq j$ 有

$$T[i][j] = \bigvee_{k=i}^j \left((k = i \text{ 或 } T[i][k-1]) \wedge (y_k \cdots y_j \in A) \right).$$

算法构造

Algorithm 7 判定 A^* 的算法 M^*

Require: 输入字符串 $y = y_1 y_2 \cdots y_n$

Ensure: 若 $y \in A^*$ 则接受, 否则拒绝

```

1: if  $n = 0$  then
2:   接受
3: end if
4: 建立一个  $n \times n$  的布尔表  $T$ , 初值全为 false
5: for  $\ell = 1$  to  $n$  do
6:   for  $i = 1$  to  $n - \ell + 1$  do
7:      $j \leftarrow i + \ell - 1$ 
8:     for  $k = i$  to  $j$  do
9:       if  $k = i$  then
10:         $left \leftarrow \text{true}$ 
11:       else
12:         $left \leftarrow T[i][k-1]$ 
13:       end if
14:       if  $left = \text{true}$  and  $M_A$  接受子串  $y_k \cdots y_j$  then
15:         $T[i][j] \leftarrow \text{true}$ 
16:        break
17:       end if
18:     end for
19:   end for
20: end for
21: if  $T[1][n] = \text{true}$  then
22:   接受
23: else
24:   拒绝
25: end if

```

正确性证明

对长度 $\ell = j - i + 1$ 做归纳, 证明算法算出的每个 $T[i][j]$ 都正确刻画了子串 $y_i \cdots y_j$ 是否属于 A^* 。

基础情形: $\ell = 1$. 此时子串只有一个字符 y_i 。算法会检查唯一的分割点 $k = i$, 此时前半段为空串, 自动视为属于 A^* ; 因此

$$T[i][i] = \text{true} \iff M_A \text{ 接受 } y_i \iff y_i \in A.$$

而长度为 1 的字符串属于 A^* ，只能是它本身就属于 A ，故此时结论成立。

归纳步骤： $\ell \geq 2$ 。假设所有长度小于 ℓ 的子串对应的表项都被正确计算。考虑任意长度为 ℓ 的子串 $y_i \cdots y_j$ 。

根据算法， $T[i][j]$ 被置为 **true** 当且仅当存在某个 $k \in [i, j]$ ，使得

$$(k = i \text{ 或 } T[i][k-1] = \text{true}) \quad \text{且} \quad M_A \text{ 接受 } y_k \cdots y_j.$$

由归纳假设可知，当 $k > i$ 时， $T[i][k-1] = \text{true}$ 等价于 $y_i \cdots y_{k-1} \in A^*$ ；而当 $k = i$ 时，前半段为空串 $\varepsilon \in A^*$ 。同时， M_A 的正确性保证

$$M_A \text{ 接受 } y_k \cdots y_j \iff y_k \cdots y_j \in A.$$

于是，

$$T[i][j] = \text{true} \iff \text{存在某个分割点 } k, y_i \cdots y_{k-1} \in A^* \text{ 且 } y_k \cdots y_j \in A.$$

这正是 $y_i \cdots y_j \in A^*$ 的定义。因此长度为 ℓ 的情形也成立。

由归纳法，所有表项都正确，特别地，

$$T[1][n] = \text{true} \iff y \in A^*.$$

因此算法 M^* 正确判定 A^* 。

时间复杂度分析

- 表 T 一共有 $O(n^2)$ 个表项。
- 对每个表项 $T[i][j]$ ，算法至多枚举 n 个分割点 k 。
- 对每个 k ，至多调用一次 M_A 判定子串 $y_k \cdots y_j$ ；该子串长度不超过 n ，因此耗时为 $O(n^c)$ 。

故总时间复杂度为

$$O(n^2) \cdot O(n) \cdot O(n^c) = O(n^{c+3}),$$

这是关于输入长度 n 的多项式时间。

因此， M^* 是一个确定性多项式时间判定算法，从而

$$A^* \in \mathbf{P}.$$

由于 $A \in \mathbf{P}$ 是任意的，得到

$$A \in \mathbf{P} \implies A^* \in \mathbf{P}.$$

故 $\mathbf{P}$ 在星号运算下封闭。

案：用动态规划记录每个子串是否已经能被分解成若干个属于 A 的片段；这样避免了对所有切分方式做指数级回溯，从而保持了多项式时间。

7.17

7.17 证明若 $\mathbf{P} = \mathbf{NP}$ ，则除了语言 $A = \emptyset$ 和 $A = \Sigma^*$ 以外，所有语言 $A \in \mathbf{P}$ 都是 $\mathbf{NP}$ 完全的。

证明. 采用多项式时间 many-one 归约作为 NP 完全性的定义。假设

$$P = NP.$$

任取语言 $A \in P$, 且

$$A \neq \emptyset, \quad A \neq \Sigma^*.$$

由于 $A \in P \subseteq NP$, 故有

$$A \in NP.$$

下面证明 A 是 NP-hard 的。任取 $L \in NP$ 。由假设 $P = NP$, 可得

$$L \in P.$$

因此存在一个多项式时间算法可以判定 L 。

又因为 $A \neq \emptyset$, 所以存在字符串 $y_1 \in A$; 因为 $A \neq \Sigma^*$, 所以存在字符串 $y_0 \notin A$ 。定义函数 $f: \Sigma^* \rightarrow \Sigma^*$ 为

$$f(x) = \begin{cases} y_1, & x \in L, \\ y_0, & x \notin L. \end{cases}$$

由于 $L \in P$, 可以在多项式时间内判断 $x \in L$ 是否成立; 而 y_0, y_1 是固定字符串, 因此 f 是多项式时间可计算函数。

对任意 $x \in \Sigma^*$, 若 $x \in L$, 则

$$f(x) = y_1 \in A;$$

若 $x \notin L$, 则

$$f(x) = y_0 \notin A.$$

因此

$$x \in L \iff f(x) \in A.$$

所以

$$L \leq_p A.$$

由于 $L \in NP$ 是任意的, 故

$$\forall L \in NP, \quad L \leq_p A.$$

因此 A 是 NP-hard 的。

综上, $A \in NP$ 且 A 是 NP-hard, 所以 A 是 NP 完全的。 $\square$

案: 这道题最值得玩味的地方在于, 规约并不只是“把一个难题机械地变成另一个难题”, 而是在不同问题的表述之间传递同一个判定结构。平时我们把多项式时间规约理解为“把未知问题压到一个已知的标准问题上”; 而在假设 $P = NP$ 之后, 原本“难”与“易”的边界被彻底抹平了。于是, 任何一个既不是空集、也不是全集的语言 $A \in P$, 都可以作为这种传递的落点。证明里只需固定一个属于 A 的字符串 y_1 和一个不属于 A 的字符串 y_0 , 就能把任意 $L \in NP$ 的真假判定压缩成对 A 的真假判定。换句话说, 从一个问题到另一个问题的规约, 可以理解成是在不断切换看问题的视角; 而当这些视角被压缩到极致时, 最后剩下的其实只是最基本的“是或否”的数学判定。

7.23

7.23 令 $CNF_k = \{\langle \phi \rangle \mid \phi \text{ 是一个可满足的 cnf 公式, 其中每个变量最多出现在 } k \text{ 个位置}\}$ 。

- a. 证明 $CNF_2 \in P$ 。
- b. 证明 CNF_3 是 NP 完全的。

图 91

$CNF_k = \{\langle \phi \rangle \mid \phi \text{ 是可满足的 CNF 公式, 且每个变量至多出现 } k \text{ 次}\}.$

(a) 证明 $CNF_2 \in P$. 设输入是一个 CNF 公式 ϕ , 并且其中每个变量在整个公式中出现的总次数至多为 2。记输入长度为 N , 这里 N 可以取为 ϕ 中文字出现的总数。

我们构造如下确定性算法, 用来判定在每个变量至多出现两次的前提下, CNF 公式 ϕ 是否可满足。

Algorithm 8 判定 2-出现 CNF 公式 ϕ 是否可满足的算法

Require: 一个 CNF 公式 ϕ , 其中每个变量至多出现 2 次

Ensure: 若 ϕ 可满足则接受, 否则拒绝

```

1:  $\psi \leftarrow \phi$ 
2: while  $\psi$  中仍含有变量 do
3:   if  $\psi$  中存在空子句 then
4:     拒绝
5:   end if
6:   任取一个仍在  $\psi$  中出现的变量  $x$ 
7:   if  $x$  在  $\psi$  中只正出现, 或只负出现 then
8:     令  $x$  取使这些子句满足的真值
9:     从  $\psi$  中删除所有包含  $x$  或  $\neg x$  的子句
10:  else
11:     $x$  在  $\psi$  中恰好一次正出现、一次负出现
12:    设这两个子句分别为  $C_1 = (x \vee A)$  和  $C_2 = (\neg x \vee B)$ 
13:    从  $\psi$  中删除  $C_1$  和  $C_2$ 
14:    构造归结子句  $D = A \vee B$ 
15:    在  $D$  中删去重复文字
16:    if  $D$  同时含有某个变量  $y$  与  $\neg y$  then
17:      丢弃  $D$  ▷  $D$  为重言式
18:    else if  $D$  为空子句 then
19:      拒绝
20:    else
21:      将  $D$  加入  $\psi$ 
22:    end if
23:  end if
24: end while
25: if  $\psi$  中存在空子句 then
26:   拒绝
27: else
28:   接受
29: end if

```

算法保持的性质. 在算法执行过程中, 始终保持“每个变量至多出现两次”这一性质。

- 在第一种操作中, 我们只是删除一些子句, 因此每个变量的出现次数只会减少。
- 在第二种操作中, 被消去的变量 x 被完全删除。对于其他变量, 它们在新子句 D 中的出现, 仅仅来自原来 A 或 B 中的出现, 因此它们的总出现次数不会增加。

因此每次循环中, 对被选中的变量 x , 确实只会落入算法中的两种情形之一。

正确性证明. 我们证明: 每次循环对公式做的变换都保持可满足性。

第一种情形: x 只以同一种极性出现. 不妨设 x 只正出现。令

$$\psi = C_1 \wedge C_2 \wedge \cdots \wedge C_t \wedge R,$$

其中 $C_1, \dots, C_t$ 是所有含有 x 的子句, R 中不含 x 。

设 ψ' 为从 ψ 中删除 $C_1, \dots, C_t$ 后得到的公式, 也就是算法更新后的公式。

- 若 ψ 可满足, 则取一个满足赋值。把其中 x 的取值改成 1 后, 所有 C_i 仍然满足, 而 R 不受影响。因此该赋值在删去 $C_1, \dots, C_t$ 后仍满足 ψ' 。
- 若 ψ' 可满足, 则把满足 ψ' 的赋值扩张为 $x = 1$, 即可满足所有 C_i , 从而满足 ψ 。

故 ψ 可满足当且仅当 ψ' 可满足。若 x 只负出现, 证明完全类似。

第二种情形: x 一正一负各出现一次. 设

$$\psi = (x \vee A) \wedge (\neg x \vee B) \wedge R,$$

其中 A, B, R 都不含变量 x 。算法将其替换为

$$\psi' = (A \vee B) \wedge R,$$

若 $A \vee B$ 是重言式则直接删除该子句; 若 $A \vee B$ 为空子句则立即拒绝。

下面证明 ψ 与 ψ' 等可满足。

- 若 ψ 可满足, 取其在除 x 之外各变量上的限制赋值。由于

$$(x \vee A) \wedge (\neg x \vee B)$$

被满足, 因此若 $x = 0$ 则必须有 $A = 1$, 若 $x = 1$ 则必须有 $B = 1$ 。无论哪种情况, 都有 $A \vee B = 1$, 故该限制赋值满足 ψ' 。

- 若 ψ' 可满足, 则在满足 ψ' 的赋值下, $A \vee B = 1$ 。若 $A = 1$, 令 $x = 0$, 则 $(\neg x \vee B)$ 自动为真, 且 $(x \vee A)$ 也为真; 若 $A = 0$, 则必有 $B = 1$, 令 $x = 1$ 即可。于是该赋值可扩张为 ψ 的满足赋值。

故 ψ 可满足当且仅当 ψ' 可满足。

由以上两种情形可知, 算法每一步都保持可满足性不变。最终若算法拒绝, 则一定是产生了空子句, 而空子句不可满足; 若算法接受, 则所有变量都已被消去且未出现空子句, 此时剩余公式为空合取, 必可满足。因此算法正确。

时间复杂度分析. 令 N 为输入公式中出现的文字总数。

- 每次循环至少消去一个变量。
- 更重要的是, 每次循环都会使当前公式中的文字总数严格下降。
- 在第一种情形中, 至少删除一个包含 x 的文字。
- 在第二种情形中, 原来的两个子句一共含有

$$|A| + |B| + 2$$

个文字, 而加入的新子句最多只含有

$$|A| + |B|$$

个文字, 因此文字总数至少减少 2。

所以循环次数至多为 N 次。若采用朴素实现, 每次循环中扫描当前公式、定位相关子句并构造归结子句, 时间均为 $O(N)$ 。因此总时间复杂度为

$$O(N^2),$$

是多项式时间。

故

$$CNF_2 \in P.$$

(b) 证明 CNF_3 是 NP 完全的。

第一步: 证明 $CNF_3 \in NP$. 给定一个真值赋值, 我们可以在多项式时间内逐个检查所有子句是否被满足, 并统计每个变量在公式中的出现次数是否至多为 3。因此

$$CNF_3 \in NP.$$

第二步: 证明 CNF_3 是 NP-hard. 我们从标准的

$$CNF-SAT = \{\langle \phi \rangle \mid \phi \text{ 是可满足的 CNF 公式}\}$$

做多项式时间 many-one 归约。

设 ϕ 是任意一个 CNF 公式。对 ϕ 中每个变量 x , 记它在 ϕ 中总共出现 d_x 次。

- 若 $d_x \leq 3$, 则保持变量 x 不变。
- 若 $d_x > 3$, 则把这 d_x 次出现分别替换为新的变量

$$x_1, x_2, \dots, x_{d_x}.$$

若原来第 i 次出现是正文字 x , 则替换为 x_i ; 若原来第 i 次出现是否定文字 $\neg x$, 则替换为 $\neg x_i$ 。

为了强制这些新变量取相同真值, 对每个满足 $d_x > 3$ 的原变量 x 再加入如下 d_x 个子句:

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge \cdots \wedge (\neg x_{d_x-1} \vee x_{d_x}) \wedge (\neg x_{d_x} \vee x_1).$$

把经过上述替换后的原子句, 与所有新加入的环形子句合起来, 得到新公式 ψ 。这就是归约输出。

归约的大小与出现次数控制. 对每个被拆开的变量 x :

- 每个新变量 x_i 在替换后的原公式部分中恰好出现一次。
- 在环形子句中, x_i 恰好正出现一次、负出现一次。

因此每个 x_i 在 ψ 中总共恰好出现 3 次; 而未被拆开的变量本来就至多出现 3 次。因此 ψ 中每个变量至多出现 3 次, 即

ψ 的确满足 CNF_3 定义中的出现次数限制。

同时, 若 ϕ 的总文字数为 N , 则新引入的变量数和子句数都只是 $O(N)$, 故整个构造可在多项式时间内完成。

正确性: 若 ϕ 可满足, 则 ψ 可满足. 设 α 是 ϕ 的一个满足赋值。

- 对于未拆开的变量 x , 令其在 ψ 中保持同样取值。
- 对于被拆开的变量 x , 令

$$x_1 = x_2 = \cdots = x_{d_x} = \alpha(x).$$

这样, 替换后的每个原子句都仍然满足, 因为我们只是把原来某一次出现的 x 或 $\neg x$ 换成了与 $\alpha(x)$ 同值的新变量。另一方面, 每个环形子句

$$(\neg x_i \vee x_{i+1})$$

在上述赋值下都变成

$$(\neg \alpha(x) \vee \alpha(x)),$$

恒为真。因此 ψ 可满足。

正确性: 若 ψ 可满足, 则 ϕ 可满足. 设 β 是 ψ 的一个满足赋值。我们证明: 对每个被拆开的原变量 x , 一定有

$$\beta(x_1) = \beta(x_2) = \cdots = \beta(x_{d_x}).$$

若不然, 则在循环序列

$$x_1, x_2, \dots, x_{d_x}, x_1$$

中, 必存在某个相邻位置满足

$$\beta(x_i) = 1, \quad \beta(x_{i+1}) = 0,$$

其中下标按模 d_x 计算。于是子句

$$(\neg x_i \vee x_{i+1})$$

在 β 下取值为假, 这与 β 满足 ψ 矛盾。故所有 x_i 必同值。

现在定义原公式 ϕ 的赋值 α :

- 对未拆开的变量 x , 令 $\alpha(x) = \beta(x)$ 。
- 对被拆开的变量 x , 令 $\alpha(x) = \beta(x_1)$ 。

由于同一个原变量 x 的所有副本 x_i 在 β 下同值, 故 ϕ 中每个原子句在 α 下的真假, 与 ψ 中对应替换后子句在 β 下的真假完全一致。因为后者都被满足, 所以前者也都被满足。因此 ϕ 可满足。

归约结论. 我们已经证明

$$\phi \in \text{CNF-SAT} \iff \psi \in \text{CNF}_3.$$

因此

$$\text{CNF-SAT} \leq_p \text{CNF}_3.$$

由于 CNF-SAT 是 NP 完全的, 故 CNF_3 是 NP-hard 的。

综合两步, 得到

$$\text{CNF}_3 \in \text{NP} \text{ 且 } \text{CNF}_3 \text{ 是 NP-hard.}$$

因此

$$\text{CNF}_3 \text{ 是 NP 完全的.}$$

7.27

7.27 图的一个着色是给结点指定颜色, 使得相邻的结点不会被指定为同一种颜色。令

$$3\text{COLOR} = \{ \langle G \rangle \mid G \text{ 的结点被三种颜色着色, 使得任何有边相连的两个结点都有不同的颜色} \}$$

证明 3COLOR 是 NP 完全的。(提示: 利用下面三个子图。)

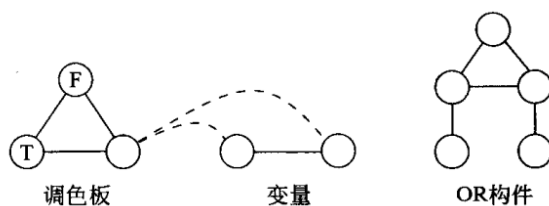


图 92

解答. 先证

$$3\text{COLOR} \in \text{NP}.$$

给定一个 3-着色方案作为证书, 只需在多项式时间内检查每个结点是否被赋予三种颜色之一, 并验证每条边两端颜色不同即可。

下面证明它是 NP-hard。我们从

$$3\text{SAT}$$

归约到

$$3\text{COLOR}.$$

设

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

是一个 3CNF 公式, 变量为

$$x_1, \dots, x_n.$$

按照图 92 中给出的三个小图来构造图 G_ϕ 。

1. 调色板. 先放入一个三角形, 其三个顶点分别记为

$$T, F, B.$$

在任何合法 3-着色中，这三个顶点必然被染成三种互不相同的颜色。以后我们直接把这三种颜色也分别称为 T, F, B 。

2. 变量构件。 对每个变量 x_i ，加入两个顶点

$$x_i, \quad \overline{x_i},$$

并令它们彼此相连；再分别把这两个顶点都与调色板中的 B 相连。这正是图中的“变量”构件。

由于 $x_i, \overline{x_i}$ 都与 B 相邻，所以它们都不能取颜色 B ，只能在 T, F 两色中取值；又因为它们彼此相邻，所以二者颜色必定相反。于是每个变量都自然地对应一个真假赋值：

$$x_i = T \iff x_i \text{ 取真}, \quad x_i = F \iff x_i \text{ 取假}.$$

同时

$$\overline{x_i}$$

自动取相反真值。

3. OR 构件。 图中的“OR 构件”有两个输入端和一个输出端。设其两个下端顶点为输入 u, v ，上端顶点为输出 w 。再把这三个端点都与调色板中的 B 相连，于是它们都只能在 T, F 两色中着色。

我们 claim：在这种限制下，OR 构件存在合法 3-着色，且输出端被强制为 T ，当且仅当两个输入中至少有一个为 T 。

原因如下。设中间两个内部顶点为 p, q 。因为

$$w, p, q$$

构成三角形，它们三者必须取三种不同颜色。

- 若要求 $w = T$ ，则 $\{p, q\} = \{F, B\}$ 。
- 若又有 $u = F$ ，由于 u 与 p 相邻， p 不能取 F ，故只能取 B 。
- 同理，若 $v = F$ ，则 q 只能取 B 。

因此若

$$u = v = F,$$

则会同时推出

$$p = q = B,$$

与 p, q 相邻矛盾。所以当输出被强制为 T 时，不可能两个输入都为 F 。

反过来，若两个输入中至少有一个为 T ，则总可以给 p, q 适当赋予 F, B 两色，使整个 OR 构件合法着色。例如：

$$(u, v) = (T, F)$$

时可取

$$p = F, \quad q = B, \quad w = T;$$

$$(u, v) = (F, T)$$

时可取

$$p = B, \quad q = F, \quad w = T;$$

$$(u, v) = (T, T)$$

时可取

$$p = F, q = B, w = T.$$

所以这个构件确实实现了“输出为真，当且仅当两个输入至少一个为真”的 OR 逻辑。

4. 子句构件. 对每个子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}),$$

使用两个 OR 构件串接起来:

- 第一个 OR 构件以

$$\ell_{j1}, \ell_{j2}$$

为输入, 输出新顶点 y_j ;

- 第二个 OR 构件以

$$y_j, \ell_{j3}$$

为输入, 输出新顶点 z_j 。

再把

$$z_j$$

同时与调色板中的

$$F \text{ 和 } B$$

相连, 于是

$$z_j$$

只能取颜色 T 。因此这个子句构件可合法着色, 当且仅当

$$\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}$$

为真。

5. 正确性.

若 ϕ 可满足, 则 G_ϕ 可 3-着色. 取 ϕ 的一个满足赋值。令调色板取三种不同颜色, 仍记作 T, F, B 。

- 对每个变量, 若 x_i 取真, 就把顶点 x_i 染成 T 、 $\overline{x_i}$ 染成 F ; 若 x_i 取假, 则反之。
- 对每个子句, 由于至少有一个文字为真, 按上面对 OR 构件的分析, 可以逐个给其中的内部顶点着色, 使最后输出 z_j 取到颜色 T 。

于是整个图得到一个合法 3-着色。

若 G_ϕ 可 3-着色, 则 ϕ 可满足. 任取 G_ϕ 的一个合法 3-着色。

- 调色板三角形确定了三种不同颜色 T, F, B 。
- 每个变量构件保证

$$x_i, \overline{x_i}$$

分别取 T, F 中的相反颜色, 于是得到一个一致的真假赋值。

- 每个子句构件的输出顶点 z_j 因与 F, B 相邻而被强制为 T 。由 OR 构件的性质可知, 该子句中至少有一个文字取真。

故所有子句都被满足, 从而 ϕ 可满足。

综上,

$$\phi \in 3SAT \iff G_\phi \in 3COLOR.$$

最后说明构造规模。若公式 ϕ 有 n 个变量和 m 个子句, 则调色板只引入常数个顶点; 每个变量只引入常数个顶点和边; 每个子句只用两个固定大小的 OR 部件以及常数条连接边。因此

$$|V(G_\phi)| = O(n + m), \quad |E(G_\phi)| = O(n + m).$$

按变量和子句顺序扫描一遍公式即可写出所有顶点和边, 所以构造可在多项式时间内完成。因此

$$3SAT \leq_p 3COLOR.$$

因此

$$3COLOR$$

是 NP-hard 的。再结合

$$3COLOR \in NP,$$

得到

$$3COLOR$$

是 NP 完全的。■

7.28

7.28 令 $SET-SPLITTING = \{\langle S, C \rangle \mid S \text{ 是一个有穷集}, C = \{C_1, \dots, C_k\} \text{ 是由 } S \text{ 的某些子集组成的集合}, k > 0, \text{ 使得 } S \text{ 的元素可以被染为红色或蓝色, 而且对所有 } C_i, C_i \text{ 中的元素不会被染成同一种颜色}\}$ 。证明 $SET-SPLITTING$ 是 NP 完全的。

图 93

解答. 先证

$$SET-SPLITTING \in NP.$$

证书就是对集合

$$S$$

中每个元素的红蓝染色。验证时逐个检查每个

$$C_i \in C$$

是否同时含有红色元素与蓝色元素即可, 故验证在多项式时间内完成。

下面从

$$3SAT$$

归约到

$$SET-SPLITTING.$$

设

$$\phi = C_1 \wedge \cdots \wedge C_m$$

是一个 3CNF 公式, 变量为

$$x_1, \dots, x_n.$$

构造有穷集

$$S = \{x_1, \overline{x_1}, \dots, x_n, \overline{x_n}, t, f\}.$$

其中 t, f 是两个新元素。再构造子集族

$$C$$

如下:

- 加入集合

$$\{t, f\};$$

- 对每个变量 x_i , 加入集合

$$\{x_i, \overline{x_i}\};$$

- 对每个子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}),$$

加入集合

$$\{\ell_{j1}, \ell_{j2}, \ell_{j3}, f\}.$$

设构造得到的实例为

$$\langle S, C \rangle.$$

若 ϕ 有 n 个变量和 m 个子句, 则

$$|S| = 2n + 2, \quad |C| = n + m + 1.$$

其中每个集合的大小至多为 4。因此输出实例的总大小为 $O(n + m)$, 并且只需扫描变量和子句即可列出 S 和 C , 所以该构造可在多项式时间内完成。

若 ϕ 可满足, 则 $\langle S, C \rangle \in \text{SET-SPLITTING}$. 取 ϕ 的一个满足赋值。把

$$t$$

染成红色,

$$f$$

染成蓝色。对每个变量 x_i :

- 若 x_i 在赋值下为真, 则把

$$x_i$$

染成红色, 把

$$\overline{x_i}$$

染成蓝色;

- 若 x_i 为假, 则反过来染色。

这样,

$$\{t, f\}$$

不是单色集; 每个

$$\{x_i, \overline{x_i}\}$$

也不是单色集。

再看任一子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}).$$

由于它被满足, 三个文字中至少有一个取真, 因此在上述染色下至少有一个文字与 t 同色, 即为红色。而集合

$$\{\ell_{j1}, \ell_{j2}, \ell_{j3}, f\}$$

中又含有蓝色元素 f , 故该集合也不是单色集。

因此这是一组合法的集合分裂染色。

若 $\langle S, C \rangle \in SET\text{-}SPLITTING$, 则 ϕ 可满足。 取一组使所有集合都非单色的红蓝染色。交换“红”“蓝”标签不会影响性质, 所以不妨设

$$t \text{ 为红色, } f \text{ 为蓝色.}$$

由于

$$\{x_i, \overline{x_i}\}$$

不是单色集, 而其中只有两个元素, 因此它们必定一红一蓝。于是可定义赋值:

$$x_i = \text{真} \iff x_i \text{ 与 } t \text{ 同色.}$$

这样

$$\overline{x_i}$$

自动取相反真假值。

现在看任一子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}).$$

对应集合

$$\{\ell_{j1}, \ell_{j2}, \ell_{j3}, f\}$$

不是单色集, 而

$$f$$

是蓝色, 所以这四个元素中至少有一个不是蓝色, 即至少有一个是红色。也就是说,

$$\ell_{j1}, \ell_{j2}, \ell_{j3}$$

中至少有一个与 t 同色, 于是至少有一个文字为真。故每个子句都被满足。

因此

$$\phi$$

可满足。

综上,

$$\phi \in 3SAT \iff \langle S, C \rangle \in SET\text{-}SPLITTING.$$

故

$$3SAT \leq_p SET-SPLITTING.$$

结合

$$SET-SPLITTING \in NP,$$

可得

$$SET-SPLITTING$$

是 NP 完全的。■

7.29

7.29 思考下述计划安排问题。有一份期末考试清单 $F_1, \dots, F_k$ 需要安排时间, 以及学生清单 $S_1, \dots, S_l$ 。每个学生都选择了这些考试的某个子集。必须将这些考试安排到各时段中, 使得同一时段中不会有某个学生同时需要参加两门考试。试问, 如果只用 h 个时段, 是否存在合乎要求的计划。将这个问题形式化为一个语言, 并证明它是 NP 完全的。

图 94

解答.

问题形式化. 定义语言

$$EXAM-SCHEDULE = \{\langle F_1, \dots, F_m, S_1, \dots, S_\ell, h \rangle \mid \text{存在合法的 } h\text{-时段安排}\}.$$

其中合法安排要求: 存在函数

$$\tau: \{F_1, \dots, F_m\} \rightarrow \{1, \dots, h\},$$

使得对每个学生 S_i 和其选择的任意两门考试 F_a, F_b , 都有

$$\tau(F_a) \neq \tau(F_b).$$

1. 证明 $EXAM-SCHEDULE \in NP$. 给定一个证书, 即分配函数

$$\tau: \{F_1, \dots, F_m\} \rightarrow \{1, \dots, h\},$$

验证器检查: 对所有学生 S_i , 其选择的每对考试 (F_a, F_b) 是否满足

$$\tau(F_a) \neq \tau(F_b).$$

验证时间至多为多项式时间, 例如 $O(\ell \cdot m^2)$ 。因此

$$EXAM-SCHEDULE \in NP.$$

2. 证明 NP-hard. 从

$$k\text{-COLORING}$$

规约。这里把颜色数 k 作为输入, 即

$$k\text{-COLORING} = \{\langle G, k \rangle \mid G \text{ 可用 } k \text{ 种颜色合法着色}\}.$$

给定一个图着色实例

$$\langle G, k \rangle, \quad G = (V, E),$$

构造一个考试安排实例。对每个顶点

$$v \in V$$

建立一门考试

$$F_v.$$

对每条边

$$\{u, v\} \in E,$$

建立一个学生

$$S_{uv},$$

并令他恰好参加两门考试

$$F_u \text{ 和 } F_v.$$

最后令

$$h = k.$$

也就是说，图中每个顶点对应一门考试，每条边对应一个同时选择这两门考试的学生，颜色数 k 对应时段数 h 。

该构造只需扫描顶点集和边集一次，输出考试数为 $|V|$ ，学生数为 $|E|$ ，所以构造时间为 $O(|V| + |E|)$ ，是多项式时间。

正确性.

若 G 有合法 k -着色，则考试实例有合法 k -时段安排。 设

$$c: V \rightarrow \{1, \dots, k\}$$

是一个合法 k -着色。定义考试安排

$$\tau(F_v) = c(v).$$

对任意学生

$$S_{uv},$$

他对应图中的边

$$\{u, v\} \in E.$$

由着色合法性可知

$$c(u) \neq c(v),$$

所以

$$\tau(F_u) \neq \tau(F_v).$$

故考试安排合法。

若考试实例有合法 k -时段安排，则 G 有合法 k -着色。 设

$$\tau$$

是一组合法安排。定义图着色

$$c(v) = \tau(F_v).$$

对任意边

$$\{u, v\} \in E,$$

对应学生 S_{uv} 同时选择考试

$$F_u \text{ 和 } F_v.$$

由于安排合法, 有

$$\tau(F_u) \neq \tau(F_v),$$

因此

$$c(u) \neq c(v).$$

故 c 是 G 的合法 k -着色。

综上,

$$\langle G, k \rangle \in k\text{-COLORING} \iff \langle F_1, \dots, F_m, S_1, \dots, S_\ell, k \rangle \in \text{EXAM-SCHEDULE}.$$

所以

$$k\text{-COLORING} \leq_p \text{EXAM-SCHEDULE}.$$

由于 $k\text{-COLORING}$ 是 NP 完全的, 且

$$\text{EXAM-SCHEDULE} \in \text{NP},$$

故

$$\text{EXAM-SCHEDULE}$$

是 NP 完全的。■

7.30

7.30 这个问题来源于单人游戏“扫雷”, 并归结到图。令 G 是一个无向图, 它的每个结点要么有一个隐藏的雷, 要么是空的。游戏者一个一个地选择结点。如果游戏者选择了有雷的结点, 则失败。如果选择了一个空结点, 则可以知道该结点的相邻结点中共有多少雷。(相邻结点即同该结点有边相连接的结点。)如果所有的空结点都被选中了, 则游戏者获胜。水雷相连问题是一个图, 图中的有些结点上标记了数字。标记了数字 m 的结点 v 表示它的相邻结点中有 m 个有雷, 我们可以据此判定其他结点中是否有雷。将此问题形式化为一个语言, 并证明它是 NP 完全的。

图 95

解答. 把题目形式化为语言

$$\text{MINE} = \{ \langle G \rangle \mid G \text{ 是一个部分带数字标记的无向图, 存在一种给未标记结点放置地雷的方式,}$$

使得每个标记为 m 的结点恰好有 m 个相邻结点带雷}.

1. 证明 $MINE \in NP$. 证书就是一组“哪些未标记结点放雷”的选择。验证时，对每个带标号的结点统计其相邻雷的数目，并检查是否等于该标号即可。每条边至多在统计相邻关系时被查看常数次，所以验证时间为图规模的多项式。因此

$$MINE \in NP.$$

2. 证明 NP-hard. 从

$$3SAT$$

归约。

给定一个 3CNF 公式

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

其中变量为

$$x_1, \dots, x_n,$$

每个子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$$

含有三个文字。构造一个带数字标记的图 G_ϕ 。

变量构件. 对每个变量 x_i ，建立两个未标记顶点

$$x_i, \quad \neg x_i,$$

分别表示文字 x_i 与 $\neg x_i$ 。再建立一个标号为 1 的顶点

$$a_i,$$

并把 a_i 与

$$x_i, \quad \neg x_i$$

相连。由于 a_i 的标号为 1，合法布雷时 x_i 与 $\neg x_i$ 中恰有一个带雷，这正好表示变量只能在真、假两种取值中选择一个。

子句构件. 对每个子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}),$$

建立一个标号为 3 的顶点

$$c_j,$$

以及两个未标记的辅助顶点

$$s_{j1}, \quad s_{j2}.$$

把 c_j 与该子句的三个文字顶点以及这两个辅助顶点相连。具体地，如果

$$\ell_{jr} = x_i,$$

则连接 c_j 与顶点 x_i ；如果

$$\ell_{jr} = \neg x_i,$$

则连接 c_j 与顶点 $\neg x_i$ 。此外连接

$$c_j$$

与

$$s_{j1}, s_{j2}.$$

该构造中共有 $2n$ 个文字顶点、 n 个变量约束顶点、 m 个子句约束顶点和 $2m$ 个辅助顶点，边数也是 $O(n+m)$ 。因此 G_ϕ 可在多项式时间内构造。

若 ϕ 可满足，则 $G_\phi \in MINE$ 。设

$$\alpha$$

是 ϕ 的一个满足赋值。我们按照赋值给文字顶点放雷：

- 若

$$\alpha(x_i) = \text{真},$$

则在顶点 x_i 上放雷；

- 若

$$\alpha(x_i) = \text{假},$$

则在顶点 $\neg x_i$ 上放雷。

于是每个变量约束顶点 a_i 的两个相邻文字顶点中恰有一个带雷，标号 1 被满足。

再看任意子句 C_j 。设它的三个文字中有

$$t_j$$

个在赋值 α 下为真。因为 α 满足 ϕ ，所以

$$t_j \in \{1, 2, 3\}.$$

在两个辅助顶点

$$s_{j1}, s_{j2}$$

中任意选择

$$3 - t_j$$

个放雷。由于 $3 - t_j \leq 2$ ，这总是可以做到。这样，子句顶点 c_j 的相邻顶点中共有

$$t_j + (3 - t_j) = 3$$

个带雷，正好满足标号 3。故得到一个合法布雷方案。

若 $G_\phi \in MINE$ ，则 ϕ 可满足。设 G_ϕ 存在一个合法布雷方案。对每个变量 x_i ，由于 a_i 标号为 1，且只与

$$x_i, \neg x_i$$

相邻，所以二者中恰有一个带雷。定义赋值

$$\alpha(x_i) = \begin{cases} \text{真}, & x_i \text{ 带雷}, \\ \text{假}, & \neg x_i \text{ 带雷}. \end{cases}$$

考虑任意子句

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}).$$

顶点 c_j 标号为 3, 且它只与三个文字顶点以及两个辅助顶点 s_{j1}, s_{j2} 相邻。如果三个文字顶点都不带雷, 那么即使两个辅助顶点都带雷, c_j 的相邻带雷数也至多为 2, 不可能满足标号 3。因此三个文字顶点中至少有一个带雷。

按赋值 α 的定义, 某个文字顶点带雷就表示该文字为真。因此每个子句至少有一个文字为真, 所以 ϕ 可满足。

综上,

$$\phi \in 3SAT \iff G_\phi \in MINE.$$

故

$$3SAT \leq_p MINE.$$

再结合

$$MINE \in NP,$$

得到

$$MINE$$

是 NP 完全的。■

7.32

7.32 令 $U = \{\langle M, x, \#^t \rangle \mid \text{非确定型图灵机 } M \text{ 在至少一个分支上在 } t \text{ 步内接受输入 } x\}$ 。证明 U 是 NP 完全的。

图 96

解答. 这里把输入中的

$$\#^t$$

理解为由 t 个符号 $\#$ 组成的一元串。

1. 证明 $U \in NP$. 一个自然证书是一条长度至多为 t 的接受分支的完整计算历史:

$$C_0, C_1, \dots, C_m, \quad m \leq t,$$

其中

$$C_0$$

是 M 在输入 x 上的初始格局,

$$C_m$$

是接受格局, 且每个

$$C_{i+1}$$

都是

$$C_i$$

在某个非确定分支上的合法后继。

因为时间界是 t , 每个格局长度至多与

$$|x| + t$$

成线性关系，所以整个证书长度是多项式的；又由于 t 用一元编码，输入长度本身也至少为 t 。因此我们可以在多项式时间内检查这份计算历史是否真实且以接受格局结束。故

$$U \in NP.$$

2. 证明 U 是 NP-hard. 任取语言

$$A \in NP.$$

则存在某个非确定型图灵机 N 和多项式 p ，使得 N 在时间

$$p(n)$$

内判定 A 。

定义归约

$$f(x) = \langle N, x, \#^{p(|x|)} \rangle.$$

这个函数可在多项式时间内计算，因为我们只需把固定机器 N 的描述、输入 x ，以及一元串

$$\#^{p(|x|)}$$

写出来。

对任意输入 x ，

$$x \in A \iff N \text{ 在某个分支上于 } p(|x|) \text{ 步内接受 } x \iff \langle N, x, \#^{p(|x|)} \rangle \in U.$$

因此

$$A \leq_p U.$$

因为 $A \in NP$ 是任意的，所以 U 是 NP-hard。结合

$$U \in NP,$$

得到

$$U$$

是 NP 完全的。■

7.34

7.34 如果图 G 中有某个结点子集，其他结点都至少与该子集中的某一结点相邻，则该子集称为**支配集**。令

$$\text{DOMINATING-SET} = \{ \langle G, k \rangle \mid G \text{ 有一个包含 } k \text{ 个结点的支配集} \}$$

通过从 VERTEX-COVER 问题归约到此问题来证明它是 NP 完全的。

图 97

解答. 先证

$$\text{DOMINATING-SET} \in NP.$$

给定一个大小为 k 的结点集合 D 作为证书, 只需检查: 对每个不在 D 中的结点 v , 是否存在

$$u \in D$$

使得

$$\{u, v\} \in E.$$

只需枚举每个顶点及证书中的顶点, 或先标记 D 的闭邻域再检查所有顶点, 时间均为图规模的多项式。

下面按题意, 从

VERTEX-COVER

归约。

给定实例

$$\langle G, k \rangle, \quad G = (V, E).$$

若

$$k = 0,$$

则该实例是否属于

VERTEX-COVER

是平凡可判定的, 因此以后只考虑

$$k \geq 1.$$

构造图 $G' = (V', E')$ 如下:

- 保留原图中的每个顶点

$$v \in V;$$

- 对每条边

$$e = \{u, v\} \in E,$$

加入一个新顶点

$$w_e;$$

- 把原顶点集合

$$V$$

变成一个团, 也就是说, 对任意不同的

$$u, v \in V$$

都加入边

$$\{u, v\};$$

- 对每条边

$$e = \{u, v\} \in E,$$

令新顶点

$$w_e$$

仅与 u, v 相连。

输出实例

$$\langle G', k \rangle.$$

该构造的输出规模为多项式。它包含原图的 $|V|$ 个顶点，并为每条边加入一个新顶点，所以

$$|V'| = |V| + |E|.$$

边包括原图补图中的边，以及每个新顶点到对应边两个端点的边，因此至多有

$$\binom{|V|}{2} + 2|E|$$

条边。枚举原图的顶点对即可写出补图边，再扫描 E 写出新顶点及其两条连接边，所以构造可在多项式时间内完成。

若 G 有大小为 k 的顶点覆盖，则 G' 有大小为 k 的支配集。设

$$C \subseteq V, \quad |C| = k$$

是 G 的一个顶点覆盖。我们 claim C 也是 G' 的支配集。

- 原顶点集合 V 在 G' 中构成团，而

$$C \neq \emptyset$$

(因为 $k \geq 1$)，所以每个原顶点要么属于 C ，要么与 C 中任意一个顶点相邻。

- 对任意新顶点 w_e ，其中

$$e = \{u, v\} \in E.$$

因为 C 是顶点覆盖，所以 u, v 中至少有一个在 C 里。故 w_e 与 C 中某个顶点相邻。

因此 C 支配了 G' 的所有顶点。

若 G' 有大小为 k 的支配集，则 G 有大小为 k 的顶点覆盖。设

$$D$$

是 G' 的一个大小为 k 的支配集。若 D 中含有某个新顶点

$$w_e,$$

其中

$$e = \{u, v\},$$

我们把它替换成端点 u 。这样做不会增大集合大小，并且仍然保持支配性质：

- 原顶点集合 V 是团，所以用 $u \in V$ 代替 w_e 后，所有原顶点仍被支配；
- 顶点 w_e 自己由 u 支配；
- 其他新顶点本来就不依赖 w_e 来被支配，因为新顶点之间没有边，而若它们与 u 相邻，替换后反而更好。

反复进行这一步，可得到一个同样大小的支配集

$$D' \subseteq V.$$

现在任取原图的一条边

$$e = \{u, v\} \in E.$$

对应的新顶点

$$w_e$$

在 G' 中只能由 u 或 v 来支配 (因为

$$D' \subseteq V$$

且 w_e 只与 u, v 相连)。所以

$$u \in D' \text{ 或 } v \in D'.$$

这说明 D' 覆盖了原图的每一条边, 因此 D' 是 G 的一个大小为 k 的顶点覆盖。

综上,

$$\langle G, k \rangle \in \text{VERTEX-COVER} \iff \langle G', k \rangle \in \text{DOMINATING-SET}.$$

故

$$\text{VERTEX-COVER} \leq_p \text{DOMINATING-SET}.$$

结合

$$\text{DOMINATING-SET} \in \text{NP},$$

得到它是 NP 完全的。■

7.36

7.36 证明: 若 $P=NP$, 则任给布尔公式 ϕ , 若 ϕ 是可满足的, 则存在多项式时间算法给出一个 ϕ 的满足赋值。(注意: 要求提供的算法计算一个函数而非若干函数, 除非是 NP 包含的语言。 $P=NP$ 的假定说明 SAT 在 P 中, 所以其可满足性的验证是多项式时间可解的。但是这个假设没有说明这项验证如何进行, 而且验证不能给出满足赋值。必须证明它们一定能够被找到。提示: 重复地使用可满足性验证器一位一位地找出满足赋值。)

图 98

解答. 假设

$$P = NP.$$

于是

$$\text{SAT} \in P.$$

设

$$T$$

是一台多项式时间算法, 用来判定一个布尔公式是否可满足。

现在给定一个可满足的布尔公式

$$\phi(x_1, \dots, x_n),$$

我们构造一个多项式时间算法输出它的一个满足赋值。

Algorithm 9 在假设 $P = NP$ 下求可满足公式的满足赋值

Require: 一个可满足的布尔公式 $\phi(x_1, \dots, x_n)$

Ensure: 输出 ϕ 的一个满足赋值

```

1:  $\psi \leftarrow \phi$ 
2: for  $i = 1$  to  $n$  do
3:   if  $T(\psi \wedge x_i)$  接受 then
4:     令  $x_i = 1$ 
5:      $\psi \leftarrow \psi \wedge x_i$ 
6:   else
7:     令  $x_i = 0$ 
8:      $\psi \leftarrow \psi \wedge \neg x_i$ 
9:   end if
10: end for
11: 输出所得赋值  $(x_1, \dots, x_n)$ 

```

正确性证明. 我们对循环过程做归纳。初始时

$$\psi = \phi$$

是可满足的。

假设在第 i 轮开始时，当前公式

$$\psi$$

仍可满足。由于

$$\psi$$

可满足，所以在某个满足赋值下，

$$x_i$$

不是取真就是取假。因此

$$\psi \wedge x_i$$

与

$$\psi \wedge \neg x_i$$

中至少有一个可满足。

- 若

$$\psi \wedge x_i$$

可满足，则算法把 x_i 设为 1，并把 ψ 更新为

$$\psi \wedge x_i;$$

新公式仍可满足。

- 否则，

$$\psi \wedge x_i$$

不可满足。由于至少一边可满足，可知

$$\psi \wedge \neg x_i$$

必可满足。算法于是把 x_i 设为 0，并把 ψ 更新为

$$\psi \wedge \neg x_i,$$

新公式仍可满足。

所以每一步之后，当前公式始终保持可满足。经过 n 轮后，所有变量都被确定，所得赋值满足最终公式，也就满足原公式 ϕ 。

时间复杂度。 算法共调用 T 恰好 n 次。每次调用的输入公式大小至多是

$$|\phi| + O(n),$$

仍为多项式规模。因为 T 是多项式时间算法，所以总运行时间是

$$n \cdot \text{poly}(|\phi|),$$

仍是多项式时间。

因此，在假设

$$P = NP$$

之下，确实存在一个多项式时间算法，可以在输入一个可满足布尔公式时输出它的一个满足赋值。■

7.39

7.39 在库克-列文定理的证明中，定义窗口为单元的 2×3 矩形。说明如果换用 2×2 的窗口，为什么会使证明失效。

图 99

解答。 在 Cook-Levin 定理的证明里，窗口的作用是刻画“相邻两行是否对应一次合法的图灵机转移”。关键点在于：一次转移会同时影响头所在位置及其左右相邻位置，总共三个相邻单元。因此需要一个

$$2 \times 3$$

的局部视野，才能把这三列作为一个整体检查。

如果只用

$$2 \times 2$$

窗口，那么左半部分和右半部分可能分别看起来都合法，但二者并不一定来自同一次转移；也就是说，可以把两个不同合法转移的“局部碎片”拼接成一个整体上非法的相邻行，而所有

$$2 \times 2$$

窗口仍然全部通过检查。

例如，设顶行在某一段是

$$w, x, qa, y,$$

其中

$$qa$$

表示机器当前处于状态 q 且头扫描符号 a 。设机器存在两种可能的后继局部形式：

$$w, q'x, b, y \quad \text{和} \quad w, x, b, q'y.$$

前一种表示头向左移动，后一种表示头向右移动。若只检查

$$2 \times 2$$

窗口，那么下面这个相邻两行：

$$\begin{array}{cccc} w & x & qa & y \\ w & q'x & b & q'y \end{array}$$

的每一个

$$2 \times 2$$

子窗口都能在前面两种合法局部形式中找到“解释”：

$$\begin{array}{ccc} w & x & x & qa & qa & y \\ w & q'x & q'x & b & b & q'y \end{array}$$

分别来自某个合法左移或右移窗口。

但是整条下一行显然不可能是一次合法转移的结果，因为它同时含有

$$q'x \text{ 和 } q'y,$$

相当于“头同时往左、往右各走了一步”。这说明：

仅仅让所有

$$2 \times 2$$

窗口都合法，并不能保证整对相邻格局真的对应一次图灵机转移。

而

$$2 \times 3$$

窗口正好把“左邻、当前、右邻”三列同时纳入检查，能阻止这种把两个不同局部转移拼接起来的伪造情形。因此，若改用

$$2 \times 2$$

窗口，Cook-Levin 证明中的局部一致性条件就不再足够，整个证明会失效。■

7.40

7.40 考虑下面的算法 *MINIMIZE*，以 DFA M 为输入，输出 DFA M' ：
MINIMIZE = “对输入 $\langle M \rangle$ ，其中 $M = (Q, \Sigma, \delta, q_0, A)$ 是 DFA：

图 100

- 1) 把 M 的从起始状态出发不可达的状态全部删除。
 - 2) 以 M 的状态为结点, 构造下面的无向图 G 。
 - 3) 在 G 中的每一个接受状态和每一个非接受状态之间连一条边。另外如下添加补充的边。
 - 4) 反复执行下面的步骤, 直到 G 中无新边加入为止:
 - 5) 对于 M 中每一对不同的状态 q 和 r , 以及每一个 $a \in \Sigma$:
 - 6) 如果 $(\delta(q, a), \delta(r, a))$ 是 G 的一条边, 就把边 (q, r) 加入 G 中。
 - 7) 对于每个状态 q , 令 $[q]$ 代表状态的集合: $[q] = \{r \in Q \mid q \text{ 和 } r \text{ 在 } G \text{ 中无边相连, 包括 } r=q\}$ 。
 - 8) 形成一个新的 DFA $M' = (Q', \Sigma, \delta', q'_0, A')$, 其中 $Q' = \{[q] \mid q \in Q\}$, (若 $[q] = [r]$, 则 Q' 中只保留其中一个); 对每个 $q \in Q$ 和 $a \in \Sigma$ 有 $\delta'([q], a) = [\delta(q, a)]$; $q'_0 = [q_0]$, $A' = \{[q] \mid q \in A\}$ 。
 - 9) 输出 $\langle M' \rangle$ 。”
- a. 证明 M 与 M' 等价。
 - b. 证明 M' 极小, 即没有更少状态的 DFA 能识别同样的语言。可以无须证明就使用问题 1.52 的结果。
 - c. 证明 MINIMIZE 在多项式时间内运行。

图 101

解答. 设输入 DFA 为

$$M = (Q, \Sigma, \delta, q_0, A),$$

算法输出

$$M' = (Q', \Sigma, \delta', q'_0, A').$$

为叙述方便, 先把第 1 步删除不可达状态之后剩余的状态集仍记为 Q 。

(a) 证明 M 与 M' 等价. 定义两个状态

$$q, r \in Q$$

是可区分的, 如果存在某个字符串

$$w \in \Sigma^*$$

使得

$$\delta^*(q, w) \in A \quad \text{而} \quad \delta^*(r, w) \notin A,$$

或反过来。

算法构造图 G 的本质, 就是把所有可区分状态对都连边。我们先证明算法结束时:

$$\{q, r\} \text{ 在 } G \text{ 中有边} \iff q, r \text{ 可区分}.$$

“有边 $\Rightarrow$ 可区分”. 按边被加入的次序归纳。

- 初始加入的边, 恰好连接一个接受状态和一个非接受状态。此时空串

就能区分这两个状态。

- 后续若因为某个

$$a \in \Sigma$$

且

$$(\delta(q, a), \delta(r, a))$$

已有边，而把

$$(q, r)$$

加入图中，那么根据归纳假设，存在字符串 w 区分

$$\delta(q, a), \delta(r, a).$$

于是字符串

$$aw$$

区分 q, r 。

“可区分 $\Rightarrow$ 有边” . 对区分字符串的最短长度归纳。

- 若最短区分字符串长度为 0，则该字符串只能是

$$\varepsilon,$$

所以 q, r 一接受一不接受，初始时就连边。

- 若最短区分字符串为

$$aw, \quad a \in \Sigma,$$

则

$$\delta(q, a), \delta(r, a)$$

被更短的字符串 w 区分。按归纳假设，它们在图中已有边。因此算法最终会把

$$(q, r)$$

加入图中。

所以最终无边相连的状态恰好是不可区分状态。算法定义

$$[q] = \{r \in Q \mid q, r \text{ 在 } G \text{ 中无边相连}\},$$

因此

$$[q]$$

正是与 q 等价的不可区分类。

现在证明 M' 与 M 识别同一语言。首先，若

$$q \sim r,$$

则对任意

$$a \in \Sigma$$

都有

$$\delta(q, a) \sim \delta(r, a),$$

否则若

$$\delta(q, a), \delta(r, a)$$

可区分, 则 q, r 会被字符串

$$aw$$

区分, 矛盾。因此

$$\delta'([q], a) = [\delta(q, a)]$$

是良定义的。

再对输入字符串 w 长度归纳, 可得

$$\delta'^*([q_0], w) = [\delta^*(q_0, w)].$$

而同一不可区分类中的状态不可能一接受一不接受, 否则会被

$$\varepsilon$$

区分。所以

$$\delta^*(q_0, w) \in A \iff [\delta^*(q_0, w)] \in A'.$$

于是

$$w \in L(M) \iff w \in L(M').$$

故

$$L(M) = L(M').$$

(b) 证明 M' 极小. 由上一步可知, M' 的状态正是删除不可达状态后, M 的不可区分类。任意两个不同的类

$$[q] \neq [r]$$

都对应一对可区分状态, 因此存在某个字符串把这两个类区分开。

现在设 N 是任何一个与 M 等价的 DFA。若 N 把两个不同的类

$$[q], [r]$$

合并到同一个状态中, 那么读入区分它们的那个字符串后, N 就无法同时模仿 M 对这两个类的不同接受行为, 矛盾。因此任何识别同一语言的 DFA 至少都要有和这些不可区分类一样多的状态。

而 M' 恰好每个类取一个状态, 所以没有更少状态的 DFA 能识别同一语言。故 M' 是极小 DFA。

若愿意, 也可以直接引用第 1 章 1.52 的结论: 最小 DFA 的状态数正好等于 Myhill–Nerode 等价类的个数, 而这里的

$$[q]$$

正是这些等价类。

(c) 证明 MINIMIZE 在多项式时间内运行. 设删除不可达状态后剩余状态数为

$$n = |Q|, \quad s = |\Sigma|.$$

- 第 1 步删除不可达状态, 可在

$$O(ns)$$

或

$$O(n^2)$$

时间内完成;

- 图 G 最多有

$$\binom{n}{2} = O(n^2)$$

条无向边;

- 初始化“接受态-非接受态”之间的边, 需要至多

$$O(n^2)$$

时间;

- 之后每一轮都扫描每对状态和每个字母, 检查是否应加边, 这一步是

$$O(n^2s).$$

每一轮若未结束, 则至少新增一条边, 所以轮数至多为

$$O(n^2).$$

因此总时间至多为

$$O(n^4s),$$

当然这是一个比较粗的上界, 但已经足够说明它是多项式时间算法。最后形成等价类并构造 M' 也只需多项式时间。

所以

$$MINIMIZE$$

在多项式时间内运行。■

7.42

* 7.42 **2cnf** 公式是子句的 AND, 其中每个子句是至多两个文字的 OR。令 $2SAT = \{\langle \phi \rangle \mid \phi \text{ 是可满足的 2cnf 公式}\}$ 。证明 $2SAT \in P$ 。

图 102

解答. 设

$$\phi$$

是一个 2CNF 公式。也就是说,

$$\phi = C_1 \wedge \cdots \wedge C_m,$$

其中每个子句 C_i 至多含有两个文字。

我们给出一个多项式时间判定算法。核心工具是蕴含图。

1. 构造蕴含图. 对每个变量

$$x$$

建立两个顶点

$$x, \quad \neg x.$$

若子句为

$$(\alpha \vee \beta),$$

则加入两条有向边

$$\neg\alpha \rightarrow \beta, \quad \neg\beta \rightarrow \alpha.$$

因为要使

$$(\alpha \vee \beta)$$

为真, 只要

$$\alpha$$

为假, 就必须令

$$\beta$$

为真; 反之亦然。

若子句只有一个文字

$$(\alpha),$$

可把它看作

$$(\alpha \vee \alpha),$$

于是加入边

$$\neg\alpha \rightarrow \alpha.$$

2. 关键判据. 我们 claim:

$$\phi \text{ 可满足} \iff \text{对每个变量 } x, x \text{ 与 } \neg x \text{ 不在同一个强连通分量中.}$$

必要性. 若某个变量 x 满足

$$x \rightsquigarrow \neg x \quad \text{且} \quad \neg x \rightsquigarrow x,$$

那么若令 x 为真, 由路径上的蕴含关系会推出

$$\neg x$$

也为真; 若令 x 为假, 则同样推出

$$x$$

为真。于是无论怎样赋值都会矛盾, 所以公式不可满足。

充分性. 若对每个变量 x ,

$$x, \neg x$$

都不在同一个强连通分量中, 则把强连通分量缩成 DAG。按该 DAG 的逆拓扑序依次赋值: 对于每个变量 x , 若

$$SCC(x)$$

排在

$$SCC(\neg x)$$

之后，就令 $x = \text{true}$ ；否则令

$$x = \text{false}.$$

这是标准的 2SAT 赋值法。它保证：若存在边

$$\alpha \rightarrow \beta,$$

则不可能出现

$$\alpha = \text{true} \quad \text{但} \quad \beta = \text{false}$$

的情形。否则会导致

$$SCC(\alpha)$$

在 DAG 中不晚于

$$SCC(\beta),$$

与赋值规则矛盾。于是所有蕴含都被满足，故每个子句也都被满足。

3. 算法与复杂度. 因此算法是：

Algorithm 10 判定 2CNF 公式是否可满足的算法

Require: 一个 2CNF 公式 ϕ

Ensure: 若 ϕ 可满足则接受，否则拒绝

- 1: 构造 ϕ 的蕴含图 H
 - 2: 求 H 的所有强连通分量
 - 3: **for all** 变量 x **do**
 - 4: **if** x 与 $\neg x$ 在同一个强连通分量中 **then**
 - 5: **拒绝**
 - 6: **end if**
 - 7: **end for**
 - 8: **接受**
-

设公式共有 n 个变量、 m 个子句，则蕴含图共有

$$2n$$

个顶点、至多

$$2m$$

条边。求强连通分量可在线性时间

$$O(n + m)$$

内完成。因此整个算法是多项式时间的。

所以

$$2SAT \in P.$$

■

Chapter 8

8.1

8.1 证明对于任意函数 $f: \mathcal{N} \rightarrow \mathcal{R}^+$, 其中 $f(n) \geq n$, 不论用单带 TM 模型还是用两带只读输入 TM 模型, 所定义的空间复杂性类 $\text{SPACE}(f(n))$ 总是相同的。

图 103

证明

设 $\mathcal{C}_1$ 是用单带 TM 定义得到的空间复杂性类 $\text{SPACE}(f(n))$, $\mathcal{C}_2$ 是用“两带且输入带只读”的 TM 定义得到的 $\text{SPACE}(f(n))$ 。我们证明

$$\mathcal{C}_1 = \mathcal{C}_2, \quad \text{其中 } f(n) \geq n.$$

$\mathcal{C}_1 \subseteq \mathcal{C}_2$ 设单带机 M 在长度为 n 的输入上使用至多 $f(n)$ 个带格。构造两带只读输入机 N :

$N =$ “对于输入 w :

1. 先把只读输入带上的 w 复制到工作带上。
2. 然后在工作带上逐步模拟 M 对 w 的计算。”

更细地说, 复制后的工作带就是 M 的单带磁带副本, 而不是在副本之外再另开一条长度为 $f(n)$ 的磁带。因为 M 在整个计算过程中只会访问至多 $f(n)$ 个带格, 且初始输入占用的 n 个带格已经包含在这 $f(n)$ 个带格中, 所以 N 的工作带只需保存这些被访问格的内容、一个工作带头标记和有限控制状态。带头标记至多给每个格增加常数个符号选择, 不改变渐近空间。

复制输入本身只需 n 个工作带格, 而题设有 $f(n) \geq n$, 故初始化阶段使用 $O(f(n))$ 空间。随后模拟 M 时, 工作带上保存的是 M 当前格局的磁带部分, 长度始终不超过 $f(n)$ 的常数倍。因此 N 也属于 $\text{SPACE}(f(n))$ 。因此

$$\mathcal{C}_1 \subseteq \mathcal{C}_2.$$

$\mathcal{C}_2 \subseteq \mathcal{C}_1$ 设两带只读输入机 M 在长度为 n 的输入上使用至多 $f(n)$ 个工作带格。构造单带机 N 如下:

$N =$ “对于输入 w :

1. 保留前 n 个带格上的输入 w 不改写, 把它当作原来的只读输入带。
2. 在其右侧开辟工作区, 存放 M 的工作带内容。
3. 用标记记录 M 的输入头位置和工作头位置, 并逐步模拟 M 。”

这里也需要说明单带机不会因为合并两条带而多出超过常数倍的空间。 N 的单带可以分成两段: 前 n 格保存原输入, 并只在模拟时读取而不改写; 右侧工作区保存 M 的工作带内容。为了模拟 M 的两个带头, N 可以在输入区的某一格上加输入头标记, 在工作区的某一格上加工作头标记; 若两个标记需要和原符号同时保存, 只需把带字母表扩大为常数倍大小。

因此, 任一时刻 N 访问过的格数至多是“输入区 n 格”加上“工作区至多 $f(n)$ 格”, 再加常数个分隔符和标记, 总共不超过

$$n + f(n) = O(f(n)),$$

因为 $f(n) \geq n$ 。故 N 仍在 $\text{SPACE}(f(n))$ 内工作，从而

$$\mathcal{C}_2 \subseteq \mathcal{C}_1.$$

综上，

$$\text{SPACE}(f(n))$$

在这两种机器模型下定义相同。■

解释. 这题的核心是：空间复杂度对机器模型的变化具有一定稳健性。只读输入带模型把输入和工作空间分开，而单带模型把它们放在同一条带上；只要 $f(n) \geq n$ ，保存原输入所需的 n 个格子本来就在允许空间量级内。带头位置和分隔标记只会让带字母表扩大常数倍，不会把 $f(n)$ 变成更高阶的函数。因此两种模型虽然模拟时间可能不同，但定义出来的空间类相同。

8.3

8.3 考虑下图所示的广义地理学游戏，其中起始结点就是由无源箭头指向的结点。选手 I 有必胜策略吗？选手 II 呢？给出理由。

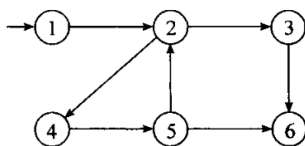


图 104

解答

选手 II 有必胜策略，选手 I 没有必胜策略。

在广义地理学游戏中，已经经过的结点不能再次使用；当前选手若无法沿一条合法有向边移动，则失败。由题图可读出边为

$$1 \rightarrow 2, \quad 2 \rightarrow 3, \quad 2 \rightarrow 4, \quad 3 \rightarrow 6, \quad 4 \rightarrow 5, \quad 5 \rightarrow 2, \quad 5 \rightarrow 6.$$

起点是 1，所以选手 I 第一手只能走

$$1 \rightarrow 2.$$

此时轮到选手 II。若选手 II 走 $2 \rightarrow 3$ ，则选手 I 走 $3 \rightarrow 6$ ，选手 II 在结点 6 无路可走而输。因此选手 II 不应走向 3。

选手 II 走

$$2 \rightarrow 4$$

则选手 I 被迫走 $4 \rightarrow 5$ 。此时结点 2 已经访问过，所以边 $5 \rightarrow 2$ 不再合法；选手 II 只需走

$$5 \rightarrow 6.$$

于是选手 I 在结点 6 无合法移动而失败。故选手 II 的必胜策略是在到达结点 2 后走向结点 4。

解释. 广义地理学游戏的关键不是单纯看当前结点有几条出边，而是看走完这条边后哪些结点会被禁止再次使用。选手 II 若走到 3，会把主动权交给选手 I，使选手 I 直接走到 6 并让 II 无路可走；走到 4 则迫使 I 到 5，同时由于 2 已访问过， $5 \rightarrow 2$ 被封锁，II 只能也是正好走到 6，把无路可走的位置留给 I。

8.6

8.6 证明 PSPACE 难的语言也是 NP 难的。

图 105

证明

设语言 B 是 PSPACE 难的。按定义，对任意 $A \in \text{PSPACE}$ ，都有多项式时间多对一归约

$$A \leq_m^p B.$$

又因为

$$\text{NP} \subseteq \text{PSPACE},$$

所以对任意 $A \in \text{NP}$ ，也有 $A \in \text{PSPACE}$ ，从而同样满足

$$A \leq_m^p B.$$

这正是 B 为 NP 难的定义。因此任意 PSPACE 难语言也是 NP 难语言。■

解释. “PSPACE 难”表示所有 PSPACE 问题都能归约到 B 。而 NP 是 PSPACE 的子类，所以所有 NP 问题也自动包含在这批“能归约到 B ”的问题里。因此 PSPACE 难比 NP 难要求更强；一旦满足前者，就必然满足后者。

8.10

8.10 五子棋游戏由两名选手“X”和“O”在 19×19 的网格上比赛。选手们轮流放棋子，第一个把自己的 5 个棋子连续地放在一行、一列或者一条对角线上的选手就是赢家。考虑把该游戏推广到 $n \times n$ 棋盘上。令

$$GM = \{\langle B \rangle \mid B \text{ 是推广的五子棋的棋局, 其中选手“X”有必胜策略}\}$$

这里的棋局是指放有一些棋子的棋盘，它可能在下棋的过程中出现。

证明 $GM \in \text{PSPACE}$ 。

图 106

证明

给定一个 $n \times n$ 的推广五子棋局面 B ，棋盘上至多还有 n^2 个空位。因此从任意局面继续下去，游戏长度至多为 n^2 步；不存在无限对局。

构造如下递归判定过程 $\text{Win}(B, P)$ ，其中 $P \in \{X, O\}$ 是当前要落子的选手。

- (1) 若当前局面已经有 X 的五子连线，则返回真。
- (2) 若当前局面已经有 O 的五子连线，则返回假。
- (3) 若无空位，则返回假。
- (4) 若 $P = X$ ，枚举所有合法落子位置；只要存在一步使得递归调用返回真，就返回真，否则返回假。
- (5) 若 $P = O$ ，枚举所有合法落子位置；只要存在一步使得递归调用返回假，就返回假，否则返回真。

这个过程精确表达了“ X 是否有必胜策略”： X 的结点取存在分支， O 的结点取所有分支。

空间复杂度方面，关键是算法做深度优先搜索，而不是把整棵博弈树同时存下来。博弈树的分支数可能很大，结点总数可能指数级，但一次递归调用只沿着一条当前路径向下搜索；搜索完一个落子分支后，回溯并继续尝试下一个分支。

递归深度至多为 n^2 ，因为每下一步至少填入一个原来为空的棋盘格。每一层递归需要保存的信息包括：当前枚举到的落子位置，其编号在 $1, \dots, n^2$ 之间，用 $O(\log n)$ 位即可保存；当前轮到谁，只需常数空间；以及递归返回后继续枚举所需的少量控制信息。棋盘本身可以用一个 $n \times n$ 数组表示，每个格只需记录空、 X 、 O 三种状态之一，所以整个棋盘占 $O(n^2)$ 个格。检查“五子连线”时，只需用行号、列号、方向和连续棋子数这几个计数器扫描棋盘，它们都只占 $O(\log n)$ 空间，不需要额外保存所有连线。

若采用最直接的实现，在每一层递归都保存一份棋盘副本，则空间上界为

$$n^2 \text{ 层} \times O(n^2) \text{ 格/层} = O(n^4),$$

仍然是多项式空间。若采用原地修改棋盘并在回溯时撤销落子的实现，则只需一份 $O(n^2)$ 的棋盘，加上 n^2 层递归栈中每层 $O(\log n)$ 的落子编号和控制信息，总空间甚至为 $O(n^2 \log n)$ 。无论采用哪种实现，空间都是多项式阶；指数增长只出现在搜索时间中，不出现在同时占用的空间中。

故

$$GM \in \text{PSPACE}.$$

解释. 这里最容易混淆的是“博弈树很大”和“算法占用空间很大”不是一回事。完整博弈树确实可能有指数多个结点，但深度优先搜索一次只保留当前这一条对局路径，以及回溯时需要恢复的少量信息。只要游戏长度有多项式上界，并且每个局面可以用多项式空间描述，就能用多项式空间穷尽整个博弈树；代价只是时间可能指数大。

8.13

8.13 定义 $A_{\text{LBA}} = \{\langle M, w \rangle \mid M \text{ 是一个接受输入 } w \text{ 的 LBA}\}$ ，证明 A_{LBA} 是 PSPACE 完全的。

图 107

证明

先证 $A_{\text{LBA}} \in \text{PSPACE}$ 。给定输入 $\langle M, w \rangle$ ，其中 M 是一台 LBA。设 $|w| = m$ 。LBA 的磁头不能离开输入所占的线性有界区域，所以 M 在 w 上任一时刻只使用 $O(m)$ 个带格。一个格局由三部分组成：当前状态、磁头位置以及这 $O(m)$ 个带格上的内容。因此一个格局的编码长度为

$$O(m \log |\Gamma|) + O(\log m) + O(1),$$

相对于输入 $\langle M, w \rangle$ 的长度是多项式的。合法格局总数至多为

$$|Q| \cdot O(m) \cdot |\Gamma|^{O(m)} = 2^{\text{poly}(|\langle M, w \rangle|)}.$$

构造格局图 G ：结点是 M 在 w 上的合法格局，边表示一步计算关系。这里的“构造”是概念上的；算法不需要把指数多个结点和边全部写出。给定两个格局 C, C' ，只要比较它们的编码并检查是否满足 M 的一步转移规则，就能在多项式空间内判断 $C \rightarrow C'$ 是否为边。于是

$$\langle M, w \rangle \in A_{\text{LBA}}$$

当且仅当在 G 中, 初始格局能够到达某个接受格局。

可用 Savitch 定理中的递归可达性算法判定这个可达性问题。设格局数上界为 N_G , 则若存在接受计算, 一定存在长度小于 N_G 的无重复格局路径。递归过程 $\text{Reach}(C_1, C_2, t)$ 只需保存起止格局 C_1, C_2 、中间格局 C_m 和步数界 t 。每个格局长度为多项式, $t \leq N_G$, 所以 t 的二进制表示也只有 $\log N_G = \text{poly}(|\langle M, w \rangle|)$ 位。递归深度为 $O(\log N_G)$, 每层保存多项式长度的信息, 因而总空间仍为多项式。注意时间可能很大, 但 PSPACE 只限制空间。因此 $A_{\text{LBA}} \in \text{PSPACE}$ 。

再证 PSPACE 难。设 $A \in \text{PSPACE}$, 则存在图灵机 M 和多项式 p , 使得 M 在每个长度为 n 的输入上只使用 $p(n)$ 个带格。对输入 w , 构造 LBA B_w 以及输入

$$x = 1^{p(|w|)}.$$

机器 B_w 把 w 硬编码在有限控制中, 并把输入 x 的 $p(|w|)$ 个格子当作可用的线性有界工作区, 在其中逐格保存 $M(w)$ 的模拟磁带内容、模拟磁头位置和模拟状态。由于 M 在输入 w 上最多使用 $p(|w|)$ 个带格, B_w 永远不需要越过这段输入空间; 若需要空白格, 就使用 x 中尚未用到的位置并把其内容改写为模拟磁带符号。构造出的 B_w 的描述长度只比 M 和硬编码的 w 多一个多项式因子, 所以从 w 生成 $\langle B_w, x \rangle$ 是多项式时间可完成的。若 M 接受 w , 则 B_w 接受 x ; 若 M 拒绝 w , 则 B_w 拒绝 x 。因此

$$w \in A \iff \langle B_w, x \rangle \in A_{\text{LBA}}.$$

该映射可在多项式时间内完成, 所以 $A \leq_m^p A_{\text{LBA}}$ 。由于 A 是任意 PSPACE 语言, A_{LBA} 是 PSPACE 难的。

综上,

A_{LBA} 是 PSPACE 完全的。

解释. LBA 的限制使得每个格局本身很短: 只能写在线性有界的带区域里, 所以一个格局可以用多项式长度编码。虽然所有格局的数量是指数级, 但判定可达性时不必把整个格局图画出来, 只需在需要时检查两个格局是否相差一步。PSPACE 难性的方向则是把任意多项式空间计算塞进一台 LBA 的线性有界输入空间里; 输入 $1^{p(|w|)}$ 的作用就是为模拟提供恰好足够长的工作区。

8.20

8.20 令 $\text{MULT} = \{a \# b \# c \mid a, b, c \text{ 是二进制自然数且 } a \times b = c\}$ 。证明 $\text{MULT} \in \text{L}$ 。

图 108

证明

设输入为 $a \# b \# c$, 三者均为二进制自然数。记最低位为第 0 位。我们逐位验证乘积

$$a \cdot b = c.$$

对第 k 位, 乘法竖式给出的未进位和为

$$s_k = \sum_{i+j=k} a_j b_i + \gamma_k,$$

其中 a_j 是 a 的第 j 位, b_i 是 b 的第 i 位, γ_k 是从更低位传来的进位。第 k 位乘积应为 $s_k \bmod 2$, 下一位进位为

$$\gamma_{k+1} = \lfloor s_k / 2 \rfloor.$$

一台对数空间机从 $k = 0$ 开始依次检查各位。它不把乘积 $a \cdot b$ 写出来，而是每次只计算当前第 k 位所需的信息。为此它保存：

$$k, \quad i, \quad j, \quad s_k, \quad \gamma_k$$

以及有限个状态。设整个输入长度为 n 。下标 k, i, j 都不超过 n ，所以各需 $O(\log n)$ 位。对固定的 k ，和式中至多有 n 项，每项为 0 或 1；同时乘法进位 γ_k 也不会超过 $O(n)$ ，因为每一列最多增加 n ，而进位每次至少除以 2 后进入下一列。因此 s_k 和 γ_k 都只需 $O(\log n)$ 位。

需要读取某一位 a_j 、 b_i 或 c_k 时，机器不保存这些字符串的副本，而是从输入开头重新扫描，借助计数器定位到相应分隔符和相应位。为了计算 s_k ，机器枚举所有满足 $i + j = k$ 的下标对，逐个重新扫描输入取得 a_j 与 b_i ，把乘积位加到 s_k 中。这样的反复扫描可能使时间变为多项式甚至更大，但工作带上始终只保存上述有限个计数器和进位。

逐位检查完所有可能的乘积位后，再确认高位进位与 c 的剩余高位一致，并确认输入格式合法。整个过程中没有保存长度为 n 的中间结果，所有工作信息都是 $O(\log n)$ 位的计数器或常数个状态，因此工作带空间为 $O(\log n)$ ，所以

$$MULT \in L.$$

解释. 对数空间算法常用的技巧是“少存，多扫”。乘积本身可能有线性长度，不能写在工作带上；但验证某一位时，只需要当前列号、枚举下标和进位。需要哪一位输入，就重新从只读输入带上定位读取。这样时间会增加，但空间复杂度只计算工作带用量，所以仍然是 $O(\log n)$ 。

8.21

8.21 对任意正整数 x ，令 x^R 是将 x 的二进制形式颠倒后得到的整数（假设 x 的二进制形式首位不是 0）。定义函数 $\mathcal{R}^+ : \mathcal{N} \rightarrow \mathcal{N}$ ，其中 $\mathcal{R}^+(x) = x + x^R$ 。

- 令 $A_2 = \{\langle x, y \rangle \mid \mathcal{R}^+(x) = y\}$ ，证明 $A_2 \in L$ 。
- 令 $A_3 = \{\langle x, y \rangle \mid \mathcal{R}^+(\mathcal{R}^+(x)) = y\}$ ，证明 $A_3 \in L$ 。

图 109

证明

先看 A_2 。设 x 的二进制长度为 m 。在计算

$$\mathcal{R}^+(x) = x + x^R$$

时，第 i 位只依赖于 x 的第 i 位、 x 的第 $m - 1 - i$ 位以及来自低位的进位。对数空间机不需要把 x^R 写到工作带上。它只保存当前位置 i 、输入长度 m 、一个进位位以及少量临时位；这些量都需要 $O(\log n)$ 空间，其中 n 是整个输入长度。每次需要 x_i 或 x_{m-1-i} 时，机器从输入开头重新扫描并用计数器定位该位即可。两个二进制位相加再加一个进位，新的进位仍然只是 0 或 1，所以不会产生长整数。机器逐位检查所得结果位是否与 y 的对应位相等，最后检查最高进位和 y 的长度是否一致。因此 $A_2 \in L$ 。

再看 A_3 。令

$$z = \mathcal{R}^+(x).$$

我们不能把 z 整体写在工作带上，因为 z 的长度可能与输入同阶，直接保存会超过对数空间。但可以在对数空间内按需计算 z 的任意一位：若要计算 z_i ，只需从低位到第 i 位重新执行上段的加法进位计算。这个子过程保存目标下标 i 、当前扫描下标、一个进位位和少量临时位，合计仍为 $O(\log n)$ 空间。若需要知道 z 的最高位是否存在，只需再执行一次到最高位的进位计算，从而确定 $|z|$ 是 m 还是 $m + 1$ 。

于是为了验证

$$y = R^+(z) = z + z^R,$$

机器再次逐位相加。外层计算保存当前位置、 $|z|$ 、外层进位以及用于和 y 比较的下标，都是 $O(\log n)$ 位。每当需要 z 的某一位时，就调用上述按需计算子过程；每当需要 z^R 的某一位时，把外层下标 i 转换为 $|z| - 1 - i$ ，再调用同一个子过程。这里的调用嵌套深度是常数层：外层加法只调用一次内层“求 z 的某一位”的过程，内层过程本身只保存计数器和进位，并不会保存整个 z 。因此同时存在的工作信息仍只是常数多个 $O(\log n)$ 位计数器和常数个进位位，总空间仍为 $O(\log n)$ 。

故

$$A_2 \in L \quad \text{且} \quad A_3 \in L.$$

解释. 这题的重点是不能把中间结果 $z = R^+(x)$ 真的写出来。做法是把“求 z 的第 i 位”当成一个可重复调用的小过程：每次从低位重新算到第 i 位，只保留下标和进位。外层验证 $R^+(z)$ 时反复调用这个小过程。重复计算会浪费时间，但不会增加超过对数级的空间。

8.24

- * 8.24 证明对每个 n ，存在两个长度为 $\text{poly}(n)$ 的正则表达式 R 和 S ，其中 $L(R) \neq L(S)$ ，但它们包含的第一个相异的字符串是指数长的。换言之， $L(R)$ 和 $L(S)$ 必须是不同的，然而对某些常量 $\epsilon > 0$ ，字符串的长度为 $2^{\epsilon n}$ 时二者是一样的。

图 110

证明

取字母表为 $\{a\}$ 。令 $p_1, \dots, p_n$ 为前 n 个素数，并令

$$P_n = p_1 p_2 \cdots p_n.$$

由于每个 $p_i \geq 2$ ，故 $P_n \geq 2^n$ 。

对每个素数 p ，构造正则表达式

$$D_p = (a^p)^*(a \cup a^2 \cup \cdots \cup a^{p-1})(a^p)^*.$$

它恰好生成长度不能被 p 整除的一元串。令

$$R_n = \epsilon \cup D_{p_1} \cup D_{p_2} \cup \cdots \cup D_{p_n}, \quad S_n = a^*.$$

因为第 i 个素数满足 $p_i = O(i \log i)$ ，所以

$$|R_n| = O\left(\sum_{i=1}^n p_i^2\right) = \text{poly}(n), \quad |S_n| = O(1).$$

现在比较 $L(R_n)$ 和 $L(S_n)$ 。任意长度 $1 \leq \ell < P_n$ 的字符串 a^ℓ 不可能同时被所有 $p_1, \dots, p_n$ 整除，否则 $P_n \mid \ell$ ，矛盾。因此存在某个 p_i 不整除 ℓ ，从而 $a^\ell \in L(D_{p_i}) \subseteq L(R_n)$ 。同时 $\epsilon \in L(R_n)$ ，所以二者在所有长度小于 P_n 的字符串上一致。

但是 $a^{P_n} \in L(S_n)$ ，而 P_n 被每个 p_i 整除，所以

$$a^{P_n} \notin L(R_n).$$

因此 $L(R_n) \neq L(S_n)$, 且它们第一次相异的字符串长度至少为

$$P_n \geq 2^n.$$

取任意常数 $0 < \varepsilon \leq 1$ 即得题设要求。

解释. 这道题想说明: 两个正则表达式本身可以很短, 但要找到区分它们的第一个字符串却可能很长。构造的办法是用许多素数做“模约束”。表达式 D_p 接受长度不被 p 整除的串, R_n 是这些条件的并, 所以只要长度 ℓ 不是所有 p_i 的共同倍数, 就会被 R_n 接受。小于 $P_n = p_1 \cdots p_n$ 的正整数不可能同时被所有 p_i 整除, 因此 R_n 和 a^* 在这些长度上看起来一样; 但长度正好为 P_n 时, 它被所有 p_i 整除, 于是被 R_n 排除, 而仍被 a^* 接受。这里的反差就是: 表达式大小只有 $\text{poly}(n)$, 第一次相异的长度却至少是 2^n 。

先说明为什么 $P_n \geq 2^n$ 。因为每个素数都至少为 2, 所以

$$P_n = p_1 p_2 \cdots p_n \geq \underbrace{2 \cdot 2 \cdots 2}_{n \text{ 个}} = 2^n.$$

这里要得到的是一个指数级下界; 实际上 P_n 远大于 2^n , 但 2^n 已经足够满足题目要求。

再说明为什么可以用 $p_i = O(i \log i)$ 。这是关于第 i 个素数的标准估计, 意思是存在常数 C , 使得当 i 足够大时,

$$p_i \leq C i \log i.$$

若用素数计数函数 $\pi(x)$ 表示不超过 x 的素数个数, 则素数定理给出

$$\pi(x) \sim \frac{x}{\log x}.$$

取 $x = C i \log i$ 且 C 足够大时, $\pi(x) \geq i$, 也就是说前 i 个素数都已经出现在 x 以内, 因此第 i 个素数满足 $p_i \leq x = O(i \log i)$ 。

最后, $\text{poly}(n)$ 表示“关于 n 的某个多项式上界”, 也就是说存在常数 $C > 0$ 和整数 $c > 0$, 使得当 n 足够大时, 相关数量不超过 $C n^c$ 。在本题中, 由 $p_i = O(i \log i)$ 可得

$$\sum_{i=1}^n p_i^2 = O\left(\sum_{i=1}^n i^2 \log^2 i\right) = O(n^3 \log^2 n),$$

而 $n^3 \log^2 n$ 仍可被某个多项式上界控制, 例如对足够大的 n 有 $n^3 \log^2 n \leq n^5$, 所以可记为 $\text{poly}(n)$ 。

8.28

8.28 令 $BOTH_{\text{NFA}} = \{\langle M_1, M_2 \rangle \mid M_1 \text{ 和 } M_2 \text{ 是 NFA, } L(M_1) \cap L(M_2) \neq \emptyset\}$ 。证明 $BOTH_{\text{NFA}}$ 是 NL 完全的。

图 111

证明

先证 $BOTH_{\text{NFA}} \in \text{NL}$ 。给定 $\langle M_1, M_2 \rangle$, 非确定型对数空间机只需在两个 NFA 的乘积自动机上猜一条路径。算法并不显式构造整个乘积自动机; 它只保存当前状态对

$$(q_1, q_2)$$

以及一个长度不超过 $|Q_1||Q_2|$ 的计数器。每个状态编号只需 $O(\log |Q_1| + \log |Q_2|)$ 位, 计数器也只需 $O(\log(|Q_1||Q_2|))$ 位, 所以这些信息对输入长度来说是对数空间。每一步非确定地猜下一个输入符号以及两台 NFA 的下一个状态, 然后重新扫描输入中两台 NFA 的转移表, 验证所猜转移确实存在。机器不保存已经猜出的输入串, 也不保存走过的路径; 它只保留当前状态对和步数。若在计数器耗尽前到达某个

$$q_1 \in F_1, \quad q_2 \in F_2$$

的状态对, 则接受。若交集非空, 则乘积自动机中必存在一条长度至多 $|Q_1||Q_2|$ 的简单接受路径, 因为重复的状态对可以删去; 反之, 任何接受路径都给出一个同时被两台 NFA 接受的字符串。因此该算法正确, 且只用对数空间。

再证 NL 难。把 NL 完全问题 PATH 归约到 $BOTH_{NFA}$ 。给定有向图 G 及结点 s, t , 构造一台一元 NFA M_1 : 状态集合为 G 的结点, 起始状态为 s , 唯一接受状态为 t ; 对每条边 $u \rightarrow v$, 加入转移

$$u \xrightarrow{a} v.$$

再令 M_2 为接受 a^* 的一状态 NFA。于是

$$L(M_1) \cap L(M_2) \neq \emptyset$$

当且仅当 G 中存在从 s 到 t 的有向路径。该构造可在对数空间内输出: 归约机只需用 $O(\log |V|)$ 位计数器依次扫描输入图的顶点和边, 遇到边 $u \rightarrow v$ 就输出一条 $u \xrightarrow{a} v$ 的转移; 起始状态和接受状态的编号也只需对数空间保存。输出带不计入工作空间, 所以无需把整台 NFA 先存入工作带。因此 $BOTH_{NFA}$ 是 NL 难的。

综上, $BOTH_{NFA}$ 是 NL 完全的。

解释. 两个 NFA 是否有公共字符串, 可以看成乘积自动机中是否能从初始状态对走到接受状态对。NL 机器不需要保存猜出的整个字符串, 只需要保存当前状态对和已经走了多少步。归约方向也很直接: 一台 NFA 模拟图中的路径, 另一台 NFA 接受所有一元串, 于是交集非空正好表达“存在从 s 到 t 的路径”。

8.30

8.30 证明 E_{DFA} 是 NL 完全的。

图 112

证明

这里

$$E_{DFA} = \{\langle M \rangle \mid M \text{ 是 DFA 且 } L(M) = \emptyset\}.$$

先证 $E_{DFA} \in \text{NL}$ 。DFA 的语言非空当且仅当从起始状态可达某个接受状态。判断非空性时, 非确定型对数空间机只需保存当前状态和一个不超过状态数 $|Q|$ 的步数计数器, 逐步猜输入符号并按照 DFA 转移函数更新当前状态; 每次查询转移函数时重新扫描机器描述即可。若在 $|Q|$ 步内到达接受状态, 则语言非空, 因为任何更长的最短接受路径都可以删去重复状态。因此

$$\overline{E_{DFA}}$$

就是一个有向图可达性问题, 属于 NL。由 Immerman-Szelepcsényi 定理, $\text{NL} = \text{coNL}$, 所以 $E_{DFA} \in \text{NL}$ 。

再证 NL 难。先观察到

$$\text{NONEMPTY}_{\text{DFA}} = \{\langle M \rangle \mid L(M) \neq \emptyset\}$$

是 NL 难的：给定 PATH 实例 (G, s, t) ，为图中每条边 $e = (u, v)$ 指定一个输入符号 a_e ，并构造 DFA，使其从状态 u 在符号 a_e 上转移到 v ；对其他状态读到 a_e 时进入死状态，死状态在所有符号上自环。起始状态为 s ，唯一接受状态为 t 。则该 DFA 的语言非空当且仅当 t 从 s 可达。这个构造也是对数空间可输出的：归约机只需用计数器枚举状态和边符号，逐项写出转移表，不需要在工作带上保存整个 DFA。

由于 $\text{NL} = \text{coNL}$ ，对任意 $A \in \text{NL}$ ，其补语言 $\bar{A}$ 也在 NL。把 $\bar{A}$ 对数空间归约到 $\text{NONEMPTY}_{\text{DFA}}$ ，得到 DFA M_x ，满足

$$x \in \bar{A} \iff L(M_x) \neq \emptyset.$$

于是

$$x \in A \iff L(M_x) = \emptyset \iff \langle M_x \rangle \in E_{\text{DFA}}.$$

故 $A \leq_m^L E_{\text{DFA}}$ 。这里使用的是对数空间归约的闭包性：先由 $\bar{A} \in \text{NL}$ 得到 $\text{NONEMPTY}_{\text{DFA}}$ 的对数空间归约，再把同一个 DFA 解释为 E_{DFA} 的实例；整个过程中工作带只保存归约机的对数空间配置和输出位置计数器。因为 A 任意， E_{DFA} 是 NL 难的。

综上， E_{DFA} 是 NL 完全的。

解释. DFA 的空性本质上是一个可达性问题的补问题：语言非空等价于从起始状态能到达某个接受状态。由于 $\text{NL} = \text{coNL}$ ，可达性的补问题仍在 NL 中。证明 NL 难时，先把 PATH 编码成 DFA 非空性，再利用补语言和同一个 DFA，把任意 NL 语言压到空性问题上。

8.33

A*8.33 令 $\text{CYCLE} = \{\langle G \rangle \mid G \text{ 是包含一个有向回路的有向图}\}$ 。证明 CYCLE 是 NL 完全的。

图 113

证明

先证 $\text{CYCLE} \in \text{NL}$ 。给定有向图 G ，非确定地猜一个起点 v ，再猜一条长度在 1 到 $|V|$ 之间的有向路径。机器只保存当前结点、起点 v 和步数计数器。结点编号和计数器都不超过输入规模，所以各占 $O(\log |V|)$ 空间。每一步只需非确定地猜下一个结点，并重新扫描输入边表检查当前结点到下一个结点是否有边；机器不保存整条路径，也不保存访问过的结点集合。若在至少走了一条边后回到 v ，则接受。若图中存在有向回路，则删去重复片段后存在一个长度至多 $|V|$ 的简单有向回路，所以该算法正确，并且只使用对数空间。

再证 NL 难。把 PATH 归约到 CYCLE 。给定实例 (G, s, t) ，设 $|V(G)| = n$ 。构造分层有向图 G' ：其结点为

$$(v, i), \quad v \in V(G), \quad 0 \leq i \leq n.$$

对 G 中每条边 $u \rightarrow v$ 和每个 $0 \leq i < n$ ，加入边

$$(u, i) \rightarrow (v, i+1).$$

这些边都从低层指向高层，因此本身不产生回路。最后，对每个 $1 \leq i \leq n$ ，加入一条边

$$(t, i) \rightarrow (s, 0).$$

若 G 中存在从 s 到 t 的路径，则删去重复结点后可取长度至多 n 的简单路径；它在 G' 中给出从

$(s, 0)$ 到某个 (t, i) 的路径，再接上边 $(t, i) \rightarrow (s, 0)$ ，得到有向回路。反之， G' 中任何回路都必须使用某条回边 $(t, i) \rightarrow (s, 0)$ ；而回边之前的分层路径正给出 G 中从 s 到 t 的路径。

该构造可在对数空间内完成。归约机输出结点 (v, i) 时只需保存原图结点编号 v 和层数 i ，二者都是 $O(\log n)$ 位；输出分层边时，用计数器枚举原图边 $u \rightarrow v$ 和层数 i ，写出 $(u, i) \rightarrow (v, i+1)$ ；输出回边时枚举 $i = 1, \dots, n$ ，写出 $(t, i) \rightarrow (s, 0)$ 。输出图本身可能有多项式大小，但输出带不计入工作空间。因此 $\text{PATH} \leq_m^L \text{CYCLE}$ 。故 CYCLE 是 NL 难的。

因此 CYCLE 是 NL 完全的。

解释. 证明 $\text{CYCLE} \in \text{NL}$ 时，机器只是在图中非确定地猜一条长度不超过 $|V|$ 的闭合路径；如果有回路，总能删掉重复片段得到这样的简单回路。NL 难性的分层构造则把原图中的路径变成只能向高层前进的路径，因此不会凭空产生回路；唯一能形成回路的边是从某个 (t, i) 回到 $(s, 0)$ 的回边。所以新图有回路，当且仅当原图中存在从 s 到 t 的路径。

Chapter 9

9.2

A 9.2 证明 $\text{TIME}(2^n) \subsetneq \text{TIME}(2^{2n})$ 。

图 114

证明

包含关系

$$\text{TIME}(2^n) \subseteq \text{TIME}(2^{2n})$$

是显然的, 因为 $2^n \leq 2^{2n}$ 。

下面证明包含是严格的。函数 $t(n) = 2^{2n}$ 是时间可构造的。由时间层次定理, 存在语言 A 满足

$$A \in \text{TIME}(t(n))$$

但

$$A \notin \text{TIME}\left(o\left(\frac{t(n)}{\log t(n)}\right)\right).$$

此处

$$\frac{t(n)}{\log t(n)} = \frac{2^{2n}}{2n}$$

差一个多项式因子仍远大于 2^n , 并且

$$2^n = o\left(\frac{2^{2n}}{2n}\right).$$

因此该语言 A 不属于 $\text{TIME}(2^n)$, 但属于 $\text{TIME}(2^{2n})$ 。故

$$\text{TIME}(2^n) \subsetneq \text{TIME}(2^{2n}).$$

9.7

9.7 给出带指数的正则表达式, 产生如下在字母表 $\{0, 1\}$ 上的语言:

- A a.** 所有长度为 500 的字符串。
- A b.** 所有长度不超过 500 的字符串。

图 115

- ^A c. 所有长度不少于 500 的字符串。
^A d. 所有长度不等于 500 的字符串。
 e. 所有恰好包含 500 个 1 的字符串。
 f. 所有包含至少 500 个 1 的字符串。
 g. 所有包含至多 500 个 1 的字符串。
 h. 所有长度不少于 500 并且在第 500 个位置上为一个 0 的字符串。
 i. 所有包含两个 0 并且其间至少相隔 500 个符号的字符串。

图 116

解答

令

$$\Sigma = (0 \cup 1).$$

带指数的正规表达式如下。

- (a) 所有长度为 500 的字符串：

$$\Sigma^{500}.$$

- (b) 所有长度不超过 500 的字符串：

$$(\Sigma \cup \epsilon)^{500}.$$

- (c) 所有长度不少于 500 的字符串：

$$\Sigma^{500}\Sigma^*.$$

- (d) 所有长度不等于 500 的字符串：

$$(\Sigma \cup \epsilon)^{499} \cup \Sigma^{501}\Sigma^*.$$

- (e) 所有恰好包含 500 个 1 的字符串：

$$0^*(10^*)^{500}.$$

- (f) 所有包含至少 500 个 1 的字符串：

$$\Sigma^*(1\Sigma^*)^{500}.$$

- (g) 所有包含至多 500 个 1 的字符串：

$$(0^*10^* \cup \epsilon)^{500}0^*.$$

- (h) 所有长度不少于 500 且第 500 个位置为 0 的字符串：

$$\Sigma^{499}0\Sigma^*.$$

- (i) 所有包含两个 0 且其间至少相隔 500 个符号的字符串：

$$\Sigma^*0\Sigma^{500}\Sigma^*0\Sigma^*.$$

9.8

9.8 若 R 是正则表达式，令 $R^{\{m,n\}}$ 代表表达式

$$R^m \cup R^{m+1} \cup \cdots \cup R^n$$

说明怎样利用通常的指数算子实现算子 $R^{\{m,n\}}$ ，但不许用“...”。

图 117

解答

若 $m \leq n$ ，则

$$R^{\{m,n\}} = R^m \cup R^{m+1} \cup \cdots \cup R^n$$

可以写成

$$R^m (R \cup \epsilon)^{n-m}.$$

理由是：后面的 $n-m$ 个因子中，每个因子都可以选择贡献一个 R ，也可以选择贡献 ϵ 。若其中恰有 i 个因子贡献 R ，则总共得到 R^{m+i} ，其中

$$0 \leq i \leq n-m.$$

所以该表达式恰好生成

$$\bigcup_{i=0}^{n-m} R^{m+i} = R^m \cup R^{m+1} \cup \cdots \cup R^n.$$

整个表达式只使用普通指数算子、连接、并和 ϵ ，没有使用省略号。

9.9

9.9 证明若 $\text{NP} = \text{P}^{\text{SAT}}$ ，则 $\text{NP} = \text{coNP}$ 。

图 118

证明

设

$$\text{NP} = \text{P}^{\text{SAT}}.$$

注意 P^{SAT} 是确定型多项式时间预言机类，因此对补运算封闭：若某语言由一台 SAT 预言机机器判定，则交换接受和拒绝状态即可判定其补语言。

取任意 $L \in \text{coNP}$ 。则

$$\bar{L} \in \text{NP} = \text{P}^{\text{SAT}}.$$

由 P^{SAT} 对补封闭，得到

$$L \in \text{P}^{\text{SAT}} = \text{NP}.$$

所以

$$\text{coNP} \subseteq \text{NP}.$$

反过来，若 $L \in \text{NP}$ ，则 $\bar{L} \in \text{coNP} \subseteq \text{NP}$ ，故 $L \in \text{coNP}$ 。于是

$$\text{NP} \subseteq \text{coNP}.$$

两边合并可得

$$NP = \text{coNP}.$$

9.12

- 9.12** 说明下面的关于 $P \neq NP$ 的错误“证明”错在哪里。采用反证法。假设 $P = NP$ ，则 $\text{SAT} \in P$ 。所以，对于某个 k ， $\text{SAT} \in \text{TIME}(n^k)$ 。因为 NP 中的每个语言多项式时间可归约到 SAT ，所以 $NP \subseteq \text{TIME}(n^k)$ 。因此 $P \subseteq \text{TIME}(n^k)$ 。但是，由时间层次定理， $\text{TIME}(n^{k+1})$ 包含不在 $\text{TIME}(n^k)$ 中的语言，这与 $P \subseteq \text{TIME}(n^k)$ 矛盾。所以 $P \neq NP$ 。

图 119

解答

错误出在这一步：

“因为 NP 中的每个语言都可多项式时间归约到 SAT ，所以 $NP \subseteq \text{TIME}(n^k)$ 。”

这个推理忽略了归约本身的多项式次数会随语言而变化。

具体地说，若假设 $P = NP$ ，则确实有 $\text{SAT} \in P$ ，于是存在某个固定 k 使得

$$\text{SAT} \in \text{TIME}(n^k).$$

但对某个 $A \in NP$ ，从 A 到 SAT 的归约可能需要时间 n^c ，其中 c 依赖于语言 A 。把该归约与 SAT 的 n^k 时间算法复合，只能得到

$$A \in \text{TIME}(n^{ck})$$

或类似的某个多项式时间界，而不能得到统一的

$$A \in \text{TIME}(n^k).$$

因此不能推出

$$NP \subseteq \text{TIME}(n^k),$$

更不能推出

$$P \subseteq \text{TIME}(n^k).$$

时间层次定理只说明多项式时间类内部有越来越高的时间层次；它并不排除

$$P = \bigcup_{d \geq 1} \text{TIME}(n^d)$$

这样的并集形式。因此该“证明”没有得到矛盾。

9.16

- 9.16** 证明 $TQBF \notin \text{SPACE}(n^{1/3})$ 。

图 120

证明

反设

$$\text{TQBF} \in \text{SPACE}(n^{1/3}).$$

我们使用 TQBF 的标准 PSPACE 完全性构造：若语言 A 可由一台使用 $s(n)$ 空间的图灵机判定，则可在对数空间内把输入 w 变换为一个量化布尔公式 φ_w ，使得

$$w \in A \iff \varphi_w \in \text{TQBF},$$

并且该公式长度为

$$|\varphi_w| = O(s(|w|)^2).$$

于是若 TQBF 能在 $N^{1/3}$ 空间内判定，则任意

$$A \in \text{SPACE}(s(n))$$

都可在空间

$$O(|\varphi_w|^{1/3}) = O(s(n)^{2/3})$$

内判定；归约本身只需对数空间，可以按需生成公式的被访问位置，不改变主项空间界。

取 $s(n) = n$ 。由空间层次定理，存在语言

$$A \in \text{SPACE}(n)$$

但

$$A \notin \text{SPACE}(n^{2/3}).$$

上面的推论却给出 $A \in \text{SPACE}(n^{2/3})$ ，矛盾。

故反设不成立，即

$$\text{TQBF} \notin \text{SPACE}(n^{1/3}).$$

9.19

9.19 定义唯一满足 (unique-sat) 问题为： $\text{USAT} = \{\langle \phi \rangle \mid \phi \text{ 是一个只有唯一满足赋值的布尔公式}\}$ 。
证明 $\text{USAT} \in \text{P}^{\text{SAT}}$ 。

图 121

证明

我们给出一台带 SAT 预言机的多项式时间确定型机器来判定 USAT 。

输入布尔公式 $\phi(x_1, \dots, x_n)$ 。

- (1) 先询问 SAT 预言机： ϕ 是否可满足？若不可满足，则拒绝。
- (2) 利用自归约构造一个满足赋值。依次处理变量 x_i 。在已经确定前缀赋值的条件下，询问

$$\phi \wedge x_1 = a_1 \wedge \dots \wedge x_{i-1} = a_{i-1} \wedge x_i = 0$$

是否可满足。若可满足，令 $a_i = 0$ ；否则令 $a_i = 1$ 。

(3) 得到一个满足赋值 $a = (a_1, \dots, a_n)$ 后, 再询问 SAT 预言机:

$$\phi \wedge \bigvee_{i=1}^n (x_i \neq a_i)$$

是否可满足。

(4) 若最后一个询问的答案为“不可满足”, 则说明不存在不同于 a 的第二个满足赋值, 接受; 否则拒绝。

该算法只做 $n + 2$ 次 SAT 询问, 并且每次构造的公式长度都是多项式。因此它属于 P^{SAT} 。算法接受当且仅当 ϕ 有且仅有一个满足赋值, 所以

$$USAT \in P^{SAT}.$$

9.23

9.23 如例 9.21 中那样定义函数 $parity_n$ 。证明可以用 $O(n)$ 规模的电路计算 $parity_n$ 。

图 122

证明

用常数规模电路实现二输入异或:

$$x \oplus y = (x \wedge \neg y) \vee (\neg x \wedge y).$$

这只需要常数个 $\wedge, \vee, \neg$ 门。

现在构造一条异或链。令

$$y_1 = x_1, \quad y_i = y_{i-1} \oplus x_i \quad (2 \leq i \leq n).$$

最后输出 y_n 。因为每增加一个输入变量只增加一个常数规模的异或子电路, 所以总门数为

$$O(n).$$

而 y_i 表示前 i 个输入中 1 的个数的奇偶性, 因此 y_n 正是

$$parity_n(x_1, \dots, x_n).$$

故 $parity_n$ 可由 $O(n)$ 规模的电路计算。

9.25

9.25 定义函数 $majority_n: \{0, 1\}^n \rightarrow \{0, 1\}$ 为:

$$majority_n(x_1, \dots, x_n) = \begin{cases} 0 & \sum x_i < n/2 \\ 1 & \sum x_i \geq n/2 \end{cases}$$

于是 $majority_n$ 函数返回输入中的多数派。证明 $majority_n$ 可以用下面的电路计算:

a. $O(n^2)$ 规模的电路。

b. $O(n \log n)$ 规模的电路。(提示: 递归地把输入数分成两半并利用问题 9.24 的结果。)

图 123

证明

令阈值

$$r = \left\lceil \frac{n}{2} \right\rceil.$$

题图中的定义等价于判断输入中 1 的个数是否至少为 r 。

(a) $O(n^2)$ 规模电路. 构造动态规划电路。令 $E_{i,j}$ 表示“前 i 个输入中至少有 j 个 1”。边界条件为

$$E_{i,0} = 1, \quad E_{0,j} = 0 \quad (j \geq 1).$$

递推式为

$$E_{i,j} = E_{i-1,j} \vee (E_{i-1,j-1} \wedge x_i).$$

对所有

$$1 \leq i \leq n, \quad 1 \leq j \leq r$$

各放置一个常数规模子电路即可。总门数为 $O(nr) = O(n^2)$ 。最后输出 $E_{n,r}$, 它恰好表示多数函数。

(b) $O(n \log n)$ 规模电路. 使用问题 9.24 中的加法电路: 两个 m 位二进制数可用 $O(m)$ 规模电路相加。从左到右维护输入中 1 的计数器。计数器只需要

$$\lceil \log(n+1) \rceil$$

位。每读入一个输入位 x_i , 用一个 $O(\log n)$ 规模的加法电路把当前计数器与 x_i 相加。重复 n 次, 总规模为

$$n \cdot O(\log n) = O(n \log n).$$

最后用 $O(\log n)$ 规模的比较电路判断计数结果是否至少为 r 。因此整体规模仍为

$$O(n \log n).$$

Chapter 10

10.2

10.2 证明：12 不能通过费马测试，从而不是伪素数。

图 124

证明

取费马测试的底数

$$a = 5.$$

有

$$\gcd(5, 12) = 1.$$

若 12 能通过以 5 为底的费马测试，则应满足

$$5^{12-1} = 5^{11} \equiv 1 \pmod{12}.$$

但

$$5^2 = 25 \equiv 1 \pmod{12},$$

所以

$$5^{11} = 5(5^2)^5 \equiv 5 \not\equiv 1 \pmod{12}.$$

因此 12 不能通过该费马测试。故 12 不是伪素数。

解释. 费马测试是一种随机化素性测试，用来判断整数 n 是否可能为素数。它基于费马小定理：若 p 是素数，且

$$\gcd(a, p) = 1,$$

则

$$a^{p-1} \equiv 1 \pmod{p}.$$

因此，对待测整数 n ，随机选取一个与 n 互素的底数 a ，检查

$$a^{n-1} \equiv 1 \pmod{n}$$

是否成立。若不成立，则 n 一定是合数；若成立，则只能说明 n 通过了这一次测试，因而是一个可能的素数。

这种测试可能出错，因为某些合数也会通过特定底数的检验，这样的数称为伪素数。更特殊地，卡迈克尔数会对所有与其互素的底数都通过费马测试。因此费马测试通常需要重复多次以降低错误概率。

10.4

10.4 证明：有 n 个输入的奇偶函数 parity 能用 $O(n)$ 个顶点的分支程序计算。

图 125

证明

构造一个按顺序读取 $x_1, \dots, x_n$ 的分支程序。第 i 层保存读完前 $i-1$ 个输入后的奇偶状态：

E_i ：当前1 的个数为偶数， O_i ：当前1 的个数为奇数。

起始结点为 E_1 。在第 i 层查询变量 x_i ：

$$\begin{array}{ll} E_i \xrightarrow{x_i=0} E_{i+1}, & E_i \xrightarrow{x_i=1} O_{i+1}, \\ O_i \xrightarrow{x_i=0} O_{i+1}, & O_i \xrightarrow{x_i=1} E_{i+1}. \end{array}$$

读完 x_n 后，从 O_{n+1} 连到接受结点，从 E_{n+1} 连到拒绝结点。

该分支程序每一层只有两个结点，总结点数为

$$2(n+1) + 2 = O(n).$$

它接受当且仅当输入中 1 的个数为奇数，因此计算的是 n 输入奇偶函数。

解释. 分支程序可以看成一种有向无环的判定图：每个内部结点询问某个输入变量，并根据变量取值沿不同边前进，最后到达接受或拒绝结点。分支程序的规模通常指结点数；若按层依次读 $x_1, \dots, x_n$ ，则每一层表示已经读过前若干位之后所需保存的信息。

奇偶函数

$$x_1 \oplus x_2 \oplus \dots \oplus x_n$$

只关心到目前为止见到的 1 的个数是奇数还是偶数。因此整个计算只需要两个状态，读到 0 时状态不变，读到 1 时在奇、偶之间切换。这正是上面构造中每层只有两个结点的原因。

10.7**10.7 证明： $BPP \subseteq PSPACE$ 。**

图 126

证明

设 $A \in BPP$ ，由概率多项式时间图灵机 M 判定。对长度为 n 的输入，可假设 M 恰好使用

$$r(n)$$

个随机比特，其中 r 是某个多项式。每一个随机比特串 $\rho \in \{0,1\}^{r(n)}$ 唯一确定 M 的一条计算分支。

确定型 PSPACE 机器可以枚举所有 ρ ，逐一模拟 $M(w; \rho)$ ，并统计接受分支数。枚举计数器、接受分支计数器和一次模拟所需的空间都为多项式；虽然分支数是指数个，但空间可以复用。

最后比较接受分支数是否大于相应阈值，例如是否至少为

$$\frac{2}{3} \cdot 2^{r(n)}.$$

若是则接受，否则拒绝。该判定与 M 的 BPP 判定条件一致，并且只使用多项式空间。因此

$$BPP \subseteq PSPACE.$$

解释. BPP 是有双边有界错误的概率多项式时间类：对属于语言的输入，机器以至少 $2/3$ 的概率接受；对不属于语言的输入，机器以至少 $2/3$ 的概率拒绝。固定输入和随机比特串以后，概率机器的运行就变成一条确定的计算分支。

PSPACE 限制的是空间而不是时间，所以它允许使用指数时间，只要工作带空间保持多项式。这里的关键是逐个枚举所有随机串，而不是同时保存所有分支。接受分支数最多为 $2^{r(n)}$ ，计数器只需要 $O(r(n))$ 位；一次模拟结束后空间也可以复用。因此指数多个随机分支可以在多项式空间内全部检查。

10.8

10.8 设 A 是 $\{0, 1\}$ 上的正则语言。证明 A 的规模-深度复杂度为 $(O(n), O(\log n))$ 。

图 127

证明

设正则语言 A 由 DFA

$$D = (Q, \{0, 1\}, \delta, q_0, F)$$

识别。因为 A 固定，状态集合 Q 的大小是常数。

每个输入比特 $b \in \{0, 1\}$ 都诱导一个从 Q 到 Q 的函数

$$T_b(q) = \delta(q, b).$$

一个长度为 n 的输入 $x_1 \cdots x_n$ 是否被接受，只取决于函数复合

$$T_{x_n} \circ T_{x_{n-1}} \circ \cdots \circ T_{x_1}$$

作用在 q_0 上后是否落入 F 。

用常数个比特编码一个 $Q \rightarrow Q$ 的函数。由于 $|Q|$ 是常数，两个这样的函数的复合也可由常数规模、常数深度的电路计算。于是构造一棵平衡二叉树：叶子根据输入位 x_i 输出 T_0 或 T_1 的编码，内部结点计算左右子树对应函数的复合。最后用常数规模电路检查复合函数把 q_0 映到的状态是否属于 F 。

该树有 n 个叶子和 $O(n)$ 个内部结点，所以规模为 $O(n)$ ；树高为 $O(\log n)$ ，每层只增加常数深度，所以总深度为 $O(\log n)$ 。因此 A 的规模-深度复杂度为

$$(O(n), O(\log n)).$$

解释. 这里的“规模-深度复杂度”是对布尔电路族的两个资源指标同时计量。对每个输入长度 n ，用一个电路 C_n 判断长度为 n 的输入是否属于语言；电路的**规模**是门的总数，表示总工作量，电路的**深度**是从输入到输出经过的最长门路径长度，表示并行计算时需要的层数。因此说规模-深度复杂度为 $(O(n), O(\log n))$ ，就是指存在一族电路，其门数为线性阶，最长计算链为对数阶。

正则语言由有限自动机识别，自动机读入一个字符时只是在有限状态集合 Q 上做一次状态转移。若语言 A 固定，则 $|Q|$ 是常数，所以每个输入符号对应的转移函数也可用常数长度编码。

本题利用的是函数复合的结合律。顺序运行 DFA 等价于把每个输入位诱导的转移函数依次复合；这些复合可以放到平衡二叉树上并行完成。由于每个内部结点只做常数规模的函数复合，整棵树的规模是线性的，深度是对数级。这说明固定正则语言不仅能由有限自动机线性扫描识别，也能由小规模、浅深度的布尔电路并行识别。

10.10

10.10 k 头下推自动机 (k PDA)是具有 k 个双向只读输入头和一个读写栈的确定型下推自动机。

定义语言类

$$\text{PDA}_k = \{A \mid A \text{ 被一台 } k \text{ PDA 识别}\}$$

证明: $P = \bigcup_k \text{PDA}_k$. (提示: 回忆一下, P 等于交错式对数空间可识别的语言类。)

图 128

证明

记

$$\text{PDA}_k = \{A \mid A \text{ 可由一台 } k \text{ 头下推自动机识别}\}.$$

证明分两边。

第一步: $\bigcup_k \text{PDA}_k \subseteq P$. 固定 k , 考虑一台 k 头下推自动机 M 在长度为 n 的输入上的运行。忽略栈中较深部分后, 一个表层格局由有限控制状态、 k 个输入头位置和栈顶符号决定, 因此表层格局数为

$$O(n^k).$$

对固定的栈符号 γ , 可计算一个摘要关系

$$R_\gamma(p, q),$$

表示从表层格局 p 出发、在栈顶为 γ 且不访问 γ 以下栈内容的前提下, 机器是否能最终弹出该 γ 并到达表层格局 q 。这些摘要关系可由动态规划或最小不动点饱和算法求出; 关系项总数为多项式, 因为 k 和栈字母表都是固定的。

一旦所有摘要关系求出, 就能在多项式时间内判断初始格局是否可达接受格局。因此每个 PDA_k 中的语言都属于 P 。

第二步: $P \subseteq \bigcup_k \text{PDA}_k$. 使用定理

$$P = \text{ALOGSPACE}.$$

设 $A \in P$, 取一台交替对数空间机 N 判定 A 。由于 N 只使用 $O(\log n)$ 空间, 它的一个格局可由常数多个 0 到 $n^{O(1)}$ 之间的整数编码。每个这样的整数可用常数个输入头的位置作为 n 进制计数器表示; 因此存在某个固定的 k , 使得一台 k 头下推自动机能够在其输入头位置中保存并更新 N 的当前格局编码。

可令 N 携带一个多项式时间时钟; 该时钟也只需 $O(\log n)$ 位, 因此仍可并入上述常数多个计数器。这样 N 的交替计算树深度为多项式, 且不会出现无限分支。

下推栈用于做交替计算树的深度优先求值。对存在格局, 自动机依次计算其后继格局并递归检查, 只要找到一个接受后继即可返回接受; 对全称格局, 自动机把尚未检查的后继信息压栈, 逐个验证所有后继都接受。每次递归调用只需在输入头位置中保存当前格局编码, 并用栈保存返回点和待检查分支。于是栈承担递归搜索记录的作用, 而格局本身始终由常数多个输入头表示。

于是存在某个固定 k , 使得 $A \in \text{PDA}_k$ 。因此

$$P \subseteq \bigcup_k \text{PDA}_k.$$

两边合并得到

$$P = \bigcup_k \text{PDA}_k.$$

解释. k 头下推自动机是在普通下推自动机的基础上增加 k 个输入头。输入头只能在只读输入上移动, 栈提供后进先出的辅助存储。这里的 k 对每个被识别的语言是固定常数, 不随输入长度变化。

包含 $\bigcup_k \text{PDA}_k \subseteq \text{P}$ 的直觉是: 固定 k 后, 输入头位置只有 n^k 种组合, 有限控制和栈顶符号也只有常数种。因此虽然完整栈可能很长, 但可以用关于“从某个栈顶符号开始能否弹回”的摘要关系来做动态规划, 避免逐条展开所有栈内容。

反向包含用到

$$\text{P} = \text{ALOGSPACE},$$

即多项式时间等于交替对数空间。对数空间格局可以用常数多个输入头位置编码, 而下推栈适合保存递归检查交替计算树时的返回信息。因此多头位置负责保存当前格局, 栈负责深度优先求值的调用记录, 两者合起来可以模拟任意多项式时间计算。

10.13

10.13 证明: 如果 $\text{PH} = \text{PSPACE}$, 则多项式时间层次只有有限个不同的层次。

图 129

证明

假设

$$\text{PH} = \text{PSPACE}.$$

由于 TQBF 是 PSPACE 完全语言, 且 $\text{PSPACE} = \text{PH}$, 可知

$$\text{TQBF} \in \text{PH}.$$

而

$$\text{PH} = \bigcup_{k \geq 0} \Sigma_k^P,$$

所以存在某个固定的 m , 使得

$$\text{TQBF} \in \Sigma_m^P.$$

现在取任意 $A \in \text{PSPACE}$ 。由于 TQBF 是 PSPACE 完全的,

$$A \leq_m^P \text{TQBF}.$$

又 Σ_m^P 对多项式时间多对一归约封闭, 因此

$$A \in \Sigma_m^P.$$

于是

$$\text{PSPACE} \subseteq \Sigma_m^P.$$

另一方面

$$\Sigma_m^P \subseteq \text{PH} = \text{PSPACE}.$$

故

$$\text{PH} = \text{PSPACE} = \Sigma_m^P.$$

这说明从第 m 层开始, 多项式时间层次全部坍缩到 Σ_m^P 。因此多项式时间层次至多只有有限个不同的层

次。

解释. 多项式时间层次 PH 由

$$\Sigma_0^P = \Pi_0^P = P, \quad \Sigma_1^P = NP, \quad \Pi_1^P = \text{coNP}$$

以及更高层的交替量词类组成。直观地，层数表示在多项式长度证书上允许多少轮存在量词和全称量词交替。

TQBF 是 PSPACE 完全语言，代表任意多项式空间计算都能归约到的核心问题。若假设 $PH = PSPACE$ ，那么 TQBF 必落在 PH 的某个固定层 Σ_m^P 中。由于所有 PSPACE 语言都可归约到 TQBF，整个 PSPACE 也会落入同一层 Σ_m^P ，于是 PH 不再有无限严格上升的层级，而是在第 m 层坍塌。

10.19

10.19 证明：如果 $NP \subseteq BPP$ ，则 $NP = RP$ 。

图 130

证明

总有

$$RP \subseteq NP,$$

因为 RP 机器在接受输入上存在一条接受随机分支，而在拒绝输入上没有接受分支；接受随机串可作为 NP 证书。

下面在假设

$$NP \subseteq BPP$$

下证明 $NP \subseteq RP$ 。只需证明 $SAT \in RP$ ，因为任意 NP 语言都可多项式时间归约到 SAT，而 RP 对多项式时间多对一归约封闭。

由假设， $SAT \in BPP$ 。把这个 BPP 算法放大，使其在每次调用上的错误概率小于 $1/(10n)$ ，其中 n 是公式变量数。

给定公式 $\phi(x_1, \dots, x_n)$ ，RP 算法尝试构造一个满足赋值。依次处理变量 x_i ：在已经固定前缀赋值后，调用放大后的 BPP-SAT 算法判断令 $x_i = 0$ 后的剩余公式是否仍可满足；若答案为是，则令 $x_i = 0$ ，否则令 $x_i = 1$ 。所有变量赋值完成后，确定性地检查所得赋值是否真的满足 ϕ ；满足则接受，否则拒绝。

若 ϕ 不可满足，则无论 BPP 子程序怎样回答，最终得到的赋值都不可能满足 ϕ ，所以算法一定拒绝。这给出了 RP 所需的单边错误。

若 ϕ 可满足，则只要所有 n 次 BPP 调用都回答正确，上述自归约过程就会始终保留至少一个满足赋值，最终构造出真正的满足赋值并接受。由并合界，所有调用都正确的概率至少为

$$1 - \frac{n}{10n} = \frac{9}{10},$$

特别地大于 $1/2$ 。因此 $SAT \in RP$ 。

故

$$NP \subseteq RP.$$

结合 $RP \subseteq NP$ ，得到

$$NP = RP.$$

解释. RP 是单边错误的随机多项式时间类：若输入不在语言中，机器必须总是拒绝；若输入在语言中，机器以至少某个常数概率接受。相比之下，BPP 允许双边错误，也就是“是”和“否”两类输入都可能以小概率答错。

证明中用到两个常见技巧。第一是错误放大：独立重复运行 BPP 算法并取多数，可以把每次调用的错误概率降到很小。第二是 SAT 的自归约：判断公式是否可满足不仅能回答“有没有满足赋值”，还可以逐个固定变量来构造一个满足赋值。最后再确定性检查构造出的赋值是否真的满足公式，就把可能的误接受消掉，从而得到 RP 所需的单边错误。